

CONTENTS

1	Introduction	1
1.1	Principle of Inclusive Terminology	1
1.2	UEFI Driver Model Extensions	2
1.3	Organization	2
1.4	Goals	5
1.5	Target Audience	7
1.6	UEFI Design Overview	7
1.7	UEFI Driver Model	8
1.7.1	UEFI Driver Model Goals	8
1.7.2	Legacy Option ROM Issues	9
1.8	Migration Requirements	9
1.8.1	Legacy Operating System Support	10
1.8.2	Supporting the UEFI Specification on a Legacy Platform	10
1.9	Conventions Used in this Document	10
1.9.1	Data Structure Descriptions	10
1.9.2	Protocol Descriptions	10
1.9.3	Procedure Descriptions	11
1.9.4	Instruction Descriptions	11
1.9.5	Pseudo-Code Conventions	11
1.9.6	Typographic Conventions	12
1.9.7	Number formats	12
1.9.7.1	Hexadecimal	13
1.9.7.2	Decimal	13
1.9.8	SI & Binary prefixes	13
1.9.9	Revision Numbers	14
2	Overview	15
2.1	Boot Manager	16
2.1.1	UEFI Images	16
2.1.2	UEFI Applications	17
2.1.3	UEFI OS Loaders	18
2.1.4	UEFI Drivers	18
2.2	Firmware Core	19
2.2.1	UEFI Services	19
2.2.2	Runtime Services	19
2.3	Calling Conventions	21
2.3.1	Data Types	21
2.3.2	IA-32 Platforms	23
2.3.2.1	Handoff State	25
2.3.2.2	Calling Convention	25

2.3.3	Intel®Itanium®-Based Platforms	25
2.3.3.1	Handoff State	27
2.3.3.2	Calling Convention	28
2.3.4	x64 Platforms	28
2.3.4.1	Handoff State	30
2.3.4.2	Detailed Calling Conventions	30
2.3.4.3	Enabling Paging or Alternate Translations in an Application	31
2.3.5	AArch32 Platforms	31
2.3.5.1	Handoff State	33
2.3.5.2	Enabling Paging or Alternate Translations in an Application	34
2.3.5.3	Detailed Calling Convention	34
2.3.6	AArch64 Platforms	35
2.3.6.1	Memory types	38
2.3.6.2	Handoff State	39
2.3.6.3	Enabling Paging or Alternate Translations in an Application	39
2.3.6.4	Detailed Calling Convention	40
2.3.7	RISC-V Platforms	40
2.3.7.1	Handoff State	42
2.3.7.2	Enabling Paging or Alternate Translations in an Application	42
2.3.7.3	Detailed Calling Convention	43
2.3.8	LoongArch Platforms	43
2.3.8.1	Handoff Statue	45
2.3.8.2	Detailed Calling Convention	46
2.4	Protocols	46
2.5	UEFI Driver Model	50
2.5.1	Legacy Option ROM Issues	52
2.5.1.1	32-bit/16-Bit Real Mode Binaries	52
2.5.1.2	Fixed Resources for Working with Option ROMs	52
2.5.1.3	Matching Option ROMs to their Devices	53
2.5.1.4	Ties to PC-AT System Design	53
2.5.1.5	Ambiguities in Specification and WorkaroundsBorn of Experience	53
2.5.2	Driver Initialization	54
2.5.3	Host Bus Controllers	54
2.5.4	Device Drivers	56
2.5.5	Bus Drivers	57
2.5.6	Platform Components	58
2.5.7	Hot-Plug Events	59
2.5.8	EFI Services Binding	59
2.6	Requirements	60
2.6.1	Required Elements	61
2.6.2	Platform-Specific Elements	61
2.6.3	Driver-Specific Elements	64
2.6.4	Extensions to this Specification Published Elsewhere	66
2.6.5	Cryptographic Algorithm Requirement	67
3	Boot Manager	68
3.1	Firmware Boot Manager	68
3.1.1	Boot Manager Programming	69
3.1.2	Load Option Processing	70
3.1.3	Load Options	71
3.1.4	Boot Manager Capabilities	73
3.1.5	Launching Boot#### Applications	73
3.1.6	Launching Boot#### Load Options Using Hot Keys	73
3.1.7	Required System Preparation Applications	75

3.2	Boot Manager Policy Protocol	75
3.2.1	EFI_BOOT_MANAGER_POLICY_PROTOCOL	75
3.2.2	EFI_BOOT_MANAGER_POLICY_PROTOCOL.ConnectDevicePath()	76
3.2.3	EFI_BOOT_MANAGER_POLICY_PROTOCOL.ConnectDeviceClass()	77
3.3	Globally Defined Variables	78
3.4	Boot Option Recovery	83
3.4.1	OS-Defined Boot Option Recovery	83
3.4.2	Platform-Defined Boot Option Recovery	83
3.4.3	Boot Option Variables Default Boot Behavior	84
3.5	Boot Mechanisms	84
3.5.1	Boot via the Simple File Protocol	84
3.5.1.1	Removable Media Boot Behavior	85
3.5.2	Boot via the Load File Protocol	85
3.5.2.1	Network Booting	86
3.5.2.2	Future Boot Media	86
4	EFI System Table	87
4.1	UEFI Image Entry Point	87
4.1.1	EFI_IMAGE_ENTRY_POINT	87
4.2	EFI Table Header	89
4.2.1	EFI_TABLE_HEADER	89
4.3	EFI System Table	90
4.3.1	EFI_SYSTEM_TABLE	90
4.4	EFI Boot Services Table	92
4.4.1	EFI_BOOT_SERVICES	92
4.5	EFI Runtime Services Table	96
4.5.1	EFI_RUNTIME_SERVICES	96
4.6	EFI Configuration Table & Properties Table	98
4.6.1	EFI_CONFIGURATION_TABLE	98
4.6.1.1	Industry Standard Configuration Tables	99
4.6.1.2	JSON Configuration Tables	100
4.6.1.3	Devicetree Tables	100
4.6.2	EFI_RT_PROPERTIES_TABLE	101
4.6.3	EFI_MEMORY_ATTRIBUTES_TABLE	102
4.6.4	EFI_CONFORMANCE_PROFILE_TABLE	103
4.6.5	Other Configuration Tables	104
4.7	Image Entry Point Examples	104
4.7.1	Image Entry Point Examples	104
4.7.2	UEFI Driver Model Example	106
4.7.3	UEFI Driver Model Example (Unloadable)	107
4.7.4	EFI Driver Model Example (Multiple Instances)	109
5	GUID Partition Table (GPT) Disk Layout	111
5.1	GPT and MBR disk layout comparison	111
5.2	LBA 0 Format	111
5.2.1	Legacy Master Boot Record (MBR)	111
5.2.2	OS Types	113
5.2.3	Protective MBR	114
5.2.4	Partition Information	115
5.3	GUID Partition Table (GPT) Disk Layout	115
5.3.1	GPT overview	115
5.3.2	GPT Header	117
5.3.3	GPT Partition Entry Array	119

6	Block Translation Table (BTT) Layout	121
6.1	Block Translation Table (BTT) Background	121
6.2	Block Translation Table (BTT) Data Structures	123
6.2.1	BTT Info Block	123
6.2.2	BTT Map Entry	125
6.2.3	BTT Flog	126
6.2.4	BTT Data Area	127
6.2.5	NVDIMM Label Protocol Address Abstraction Guid	127
6.3	BTT Theory of Operation	127
6.3.1	BTT Arenas	128
6.3.2	Atomicity of Data Blocks in an Arena	129
6.3.3	Atomicity of BTT Data Structures	129
6.3.4	Writing the Initial BTT layout	130
6.3.5	Validating BTT Arenas at start-up	130
6.3.6	Validating the Flog entries at start-up	131
6.3.7	Read Path	132
6.3.8	Write Path	132
7	Services — Boot Services	135
7.1	Event, Timer, and Task Priority Services	136
7.1.1	EFI_BOOT_SERVICES.CreateEvent()	139
7.1.2	EFI_BOOT_SERVICES.CreateEventEx()	142
7.1.3	EFI_BOOT_SERVICES.CloseEvent()	145
7.1.4	EFI_BOOT_SERVICES.SignalEvent()	146
7.1.5	EFI_BOOT_SERVICES.WaitForEvent()	147
7.1.6	EFI_BOOT_SERVICES.CheckEvent()	148
7.1.7	EFI_BOOT_SERVICES.SetTimer()	148
7.1.8	EFI_BOOT_SERVICES.RaiseTPL()	149
7.1.9	EFI_BOOT_SERVICES.RestoreTPL()	151
7.2	Memory Allocation Services	151
7.2.1	EFI_BOOT_SERVICES.AllocatePages()	154
7.2.2	EFI_BOOT_SERVICES.FreePages()	156
7.2.3	EFI_BOOT_SERVICES.GetMemoryMap()	156
7.2.4	EFI_BOOT_SERVICES.AllocatePool()	160
7.2.5	EFI_BOOT_SERVICES.FreePool()	161
7.3	Protocol Handler Services	162
7.3.1	Driver Model Boot Services	164
7.3.2	EFI_BOOT_SERVICES.InstallProtocolInterface()	166
7.3.3	EFI_BOOT_SERVICES.UninstallProtocolInterface()	167
7.3.4	EFI_BOOT_SERVICES.ReinstallProtocolInterface()	169
7.3.5	EFI_BOOT_SERVICES.RegisterProtocolNotify()	170
7.3.6	EFI_BOOT_SERVICES.LocateHandle()	171
7.3.7	EFI_BOOT_SERVICES.HandleProtocol()	172
7.3.8	EFI_BOOT_SERVICES.LocateDevicePath()	174
7.3.9	EFI_BOOT_SERVICES.OpenProtocol()	175
7.3.10	EFI_BOOT_SERVICES.CloseProtocol()	180
7.3.11	EFI_BOOT_SERVICES.OpenProtocolInformation()	182
7.3.12	EFI_BOOT_SERVICES.ConnectController()	183
7.3.13	EFI_BOOT_SERVICES.DisconnectController()	187
7.3.14	EFI_BOOT_SERVICES.ProtocolsPerHandle()	189
7.3.15	EFI_BOOT_SERVICES.LocateHandleBuffer()	190
7.3.16	EFI_BOOT_SERVICES.LocateProtocol()	193
7.3.17	EFI_BOOT_SERVICES.InstallMultipleProtocolInterfaces()	194
7.3.18	EFI_BOOT_SERVICES.UninstallMultipleProtocolInterfaces()	195

7.4	Image Services	196
7.4.1	EFI_BOOT_SERVICES.LoadImage()	197
7.4.2	EFI_BOOT_SERVICES.StartImage()	199
7.4.3	EFI_BOOT_SERVICES.UnloadImage()	200
7.4.4	EFI_IMAGE_ENTRY_POINT	201
7.4.5	EFI_BOOT_SERVICES.Exit()	202
7.4.6	EFI_BOOT_SERVICES.ExitBootServices()	203
7.5	Miscellaneous Boot Services	204
7.5.1	EFI_BOOT_SERVICES.SetWatchdogTimer()	205
7.5.2	EFI_BOOT_SERVICES.Stall()	206
7.5.3	EFI_BOOT_SERVICES.CopyMem()	206
7.5.4	EFI_BOOT_SERVICES.SetMem()	207
7.5.5	EFI_BOOT_SERVICES.GetNextMonotonicCount()	208
7.5.6	EFI_BOOT_SERVICES.InstallConfigurationTable()	208
7.5.7	EFI_BOOT_SERVICES.CalculateCrc32()	209
8	Services — Runtime Services	211
8.1	Runtime Services Rules and Restrictions	212
8.1.1	Exception for Machine Check, INIT, and NMI	213
8.2	Variable Services	213
8.2.1	GetVariable()	214
8.2.2	GetNextVariableName()	216
8.2.3	SetVariable()	218
8.2.4	QueryVariableInfo()	223
8.2.5	Using the EFI_VARIABLE_AUTHENTICATION_3 descriptor	224
8.2.6	Using the EFI_VARIABLE_AUTHENTICATION_2 descriptor	227
8.2.7	Hardware Error Record Persistence	229
8.2.7.1	Hardware Error Record Non-Volatile Store	229
8.2.7.2	Hardware Error Record Variables	229
8.2.7.3	Common Platform Error Record Format	230
8.3	Time Services	230
8.3.1	GetTime()	230
8.3.2	SetTime()	233
8.3.3	GetWakeuptime()	234
8.3.4	SetWakeuptime()	235
8.4	Virtual Memory Services	236
8.4.1	SetVirtualAddressMap()	236
8.4.2	ConvertPointer()	237
8.5	Miscellaneous Runtime Services	239
8.5.1	Reset System	239
8.5.1.1	ResetSystem()	239
8.5.2	Get Next High Monotonic Count	240
8.5.2.1	GetNextHighMonotonicCount()	240
8.5.3	Update Capsule	241
8.5.3.1	UpdateCapsule()	242
8.5.3.2	Capsule Definition	245
8.5.3.3	EFI_MEMORY_RANGE_CAPSULE_GUID	246
8.5.3.4	QueryCapsuleCapabilities()	248
8.5.4	Exchanging information between the OS and Firmware	249
8.5.5	Delivery of Capsules via file on Mass Storage Device	250
8.5.6	UEFI variable reporting on the Success or any Errors encountered in processing of capsules after restart	251
8.5.6.1	EFI_CAPSULE_REPORT_GUID	252

9	Protocols - EFI Loaded Image	255
9.1	EFI Loaded Image Protocol	255
9.1.1	EFI_LOADED_IMAGE_PROTOCOL	255
9.1.2	EFI_LOADED_IMAGE_PROTOCOL.Unload()	257
9.2	EFI Loaded Image Device Path Protocol	257
9.2.1	EFI_LOADED_IMAGE_DEVICE_PATH_PROTOCOL	257
10	Protocols – Device Path Protocol	259
10.1	Device Path Overview	259
10.2	EFI Device Path Protocol	259
10.3	Device Path Nodes	260
10.3.1	Generic Device Path Structures	261
10.3.2	Hardware Device Path	262
10.3.2.1	PCI Device Path	262
10.3.2.2	PCCARD Device Path	262
10.3.2.3	Memory Mapped Device Path	263
10.3.2.4	Vendor Device Path	263
10.3.2.5	Controller Device Path	263
10.3.2.6	BMC Device Path	264
10.3.3	ACPI Device Path	264
10.3.3.1	ACPI_ADR Device Path	266
10.3.3.2	NVDIMM Device Path	266
10.3.4	Messaging Device Path	267
10.3.4.1	ATAPI Device Path	267
10.3.4.2	SCSI Device Path	267
10.3.4.3	Fibre Channel Device Path	267
10.3.4.4	1394 Device Path	269
10.3.4.5	USB Device Paths	270
10.3.4.6	SATA Device Path	271
10.3.4.7	USB Device Paths (WWID)	272
10.3.4.8	Device Logical Unit	272
10.3.4.9	I ₂ O Device Path	273
10.3.4.10	MAC Address Device Path	273
10.3.4.11	IPv4 Device Path	274
10.3.4.12	IPv6 Device Path	274
10.3.4.13	2. VLAN device path node	275
10.3.4.14	InfiniBand Device Path	275
10.3.4.15	UART Device Path	275
10.3.4.16	Vendor-Defined Messaging Device Path	276
10.3.4.17	UART Flow Control Messaging Path	277
10.3.4.18	Serial Attached SCSI (SAS) Device Path	277
10.3.4.19	Serial Attached SCSI (SAS) Extended Device Path	279
10.3.4.20	iSCSI Device Path	280
10.3.4.21	NVM Express namespace messaging device path node	288
10.3.4.22	Uniform Resource Identifiers (URI) Device Path	289
10.3.4.23	UFS (Universal Flash Storage) device messaging devicepath node	289
10.3.4.24	SD (Secure Digital) Device Path	289
10.3.4.25	EFI Bluetooth Device Path	290
10.3.4.26	Wireless Device Path	290
10.3.4.27	eMMC (Embedded Multi-Media Card) Device Path	290
10.3.4.28	EFI BluetoothLE Device Path	290
10.3.4.29	DNS Device Path	291
10.3.4.30	NVDIMM Namespace path	291
10.3.4.31	REST Service Device Path	291

10.3.4.32	NVMe over Fabric (NVMe-oF) Namespace Device Path	292
10.3.4.33	NVMe over Fabric (NVMe-oF) Namespace Device Path Example	293
10.3.5	Media Device Path	295
10.3.5.1	Hard Drive	295
10.3.5.2	CD-ROM Media Device Path	296
10.3.5.3	Vendor-Defined Media Device Path	296
10.3.5.4	File Path Media Device Path	296
10.3.5.5	Media Protocol Device Path	297
10.3.5.6	PIWG Firmware File	297
10.3.5.7	PIWG Firmware Volume	298
10.3.5.8	Relative Offset Range	298
10.3.5.9	RAM Disk	298
10.3.6	BIOS Boot Specification Device Path	299
10.4	Device Path Generation Rules	300
10.4.1	Housekeeping Rules	300
10.4.2	Rules with ACPI _HID and _UID	300
10.4.3	Rules with ACPI _ADR	301
10.4.4	Hardware vs. Messaging Device Path Rules	301
10.4.5	Media Device Path Rules	302
10.4.6	Other Rules	302
10.5	Device Path Utilities Protocol	302
10.5.1	EFI_DEVICE_PATH_UTILITIES_PROTOCOL	302
10.5.2	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.GetDevicePathSize()	303
10.5.3	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.DuplicateDevicePath()	304
10.5.4	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.AppendDevicePath()	304
10.5.5	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.AppendDeviceNode()	305
10.5.6	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.AppendDevicePathInstance()	306
10.5.7	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.GetNextDevicePathInstance()	306
10.5.8	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.CreateDeviceNode()	307
10.5.9	EFI_DEVICE_PATH_UTILITIES_PROTOCOL.IsDevicePathMultiInstance()	308
10.6	EFI Device Path Display Format Overview	308
10.6.1	Design Discussion	309
10.6.1.1	Standardized Display Format	309
10.6.1.2	Readability	309
10.6.1.3	Round-Trip Conversion	309
10.6.1.4	Command-Line Parsing	310
10.6.1.5	Text Representation Basics	310
10.6.1.6	Text Device Node Reference	311
10.6.2	Device Path to Text Protocol	330
10.6.2.1	EFI_DEVICE_PATH_TO_TEXT_PROTOCOL	330
10.6.3	EFI_DEVICE_PATH_TO_TEXT_PROTOCOL.ConvertDeviceNodeToText()	331
10.6.4	EFI_DEVICE_PATH_TO_TEXT_PROTOCOL.ConvertDevicePathToText()	332
10.6.5	Device Path from Text Protocol	333
10.6.5.1	EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL	333
10.6.5.2	EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL.ConvertTextToDeviceNode()	333
10.6.5.3	EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL.ConvertTextToDevicePath()	334
11	Protocols — UEFI Driver Model	335
11.1	EFI Driver Binding Protocol	335
11.1.1	EFI_DRIVER_BINDING_PROTOCOL	335
11.1.2	EFI_DRIVER_BINDING_PROTOCOL.Supported()	337
11.1.3	EFI_DRIVER_BINDING_PROTOCOL.Start()	341
11.1.4	EFI_DRIVER_BINDING_PROTOCOL.Stop()	350
11.2	EFI Platform Driver Override Protocol	354

11.2.1	EFI_PLATFORM_DRIVER_OVERRIDE_PROTOCOL.GetDriver()	355
11.2.2	EFI_PLATFORM_DRIVER_OVERRIDE_PROTOCOL.GetDriverPath()	356
11.2.3	EFI_PLATFORM_DRIVER_OVERRIDE_PROTOCOL.DriverLoaded()	357
11.3	EFI Bus Specific Driver Override Protocol	358
11.3.1	EFI_BUS_SPECIFIC_DRIVER_OVERRIDE_PROTOCOL	358
11.3.2	EFI_BUS_SPECIFIC_DRIVER_OVERRIDE_PROTOCOL.GetDriver()	359
11.4	EFI Driver Diagnostics Protocol	360
11.4.1	EFI_DRIVER_DIAGNOSTICS2_PROTOCOL	360
11.4.2	EFI_DRIVER_DIAGNOSTICS2_PROTOCOL.RunDiagnostics()	361
11.5	EFI Component Name Protocol	363
11.5.1	EFI_COMPONENT_NAME2_PROTOCOL	363
11.5.2	EFI_COMPONENT_NAME2_PROTOCOL.GetDriverName()	364
11.5.3	EFI_COMPONENT_NAME2_PROTOCOL.GetControllerName()	365
11.6	EFI Service Binding Protocol	366
11.6.1	EFI_SERVICE_BINDING_PROTOCOL	366
11.6.2	EFI_SERVICE_BINDING_PROTOCOL.CreateChild()	367
11.6.3	EFI_SERVICE_BINDING_PROTOCOL.DestroyChild()	370
11.7	EFI Platform to Driver Configuration Protocol	374
11.7.1	EFI_PLATFORM_TO_DRIVER_CONFIGURATION_PROTOCOL	374
11.7.2	EFI_PLATFORM_TO_DRIVER_CONFIGURATION_PROTOCOL.Query()	375
11.7.3	EFI_PLATFORM_TO_DRIVER_CONFIGURATION_PROTOCOL.Response()	376
11.7.4	DMTF SM CLP ParameterTypeGuid	378
11.8	EFI Driver Supported EFI Version Protocol	379
11.8.1	EFI_DRIVER_SUPPORTED_EFI_VERSION_PROTOCOL	379
11.9	EFI Driver Family Override Protocol	380
11.9.1	Overview	380
11.9.1.1	EFI_DRIVER_FAMILY_OVERRIDE_PROTOCOL	381
11.9.1.2	EFI_DRIVER_FAMILY_OVERRIDE_PROTOCOL.GetVersion()	382
11.10	EFI Driver Health Protocol	382
11.10.1	EFI_DRIVER_HEALTH_PROTOCOL	382
11.10.2	EFI_DRIVER_HEALTH_PROTOCOL.GetHealthStatus()	384
11.10.3	EFI_DRIVER_HEALTH_PROTOCOL.Repair()	387
11.10.4	UEFI Boot Manager Algorithms	389
11.10.4.1	All Controllers Healthy	389
11.10.4.2	Process a Controller Until Terminal StateReached	390
11.10.4.3	Repair Notification Function	391
11.10.4.4	Process Message List	391
11.10.4.5	Process HII Form	392
11.10.5	UEFI Driver Algorithms	392
11.10.5.1	Driver Entry Point Updates	392
11.10.5.2	Add global variable	392
11.10.5.3	Update private context structure	392
11.10.5.4	Implement GetHealthStatus() service	393
11.10.5.5	Implement Repair() service	393
11.11	EFI Adapter Information Protocol	393
11.11.1	EFI_ADAPTER_INFORMATION_PROTOCOL	393
11.11.2	EFI_ADAPTER_INFORMATION_PROTOCOL.EFI_ADAPTER_GET_INFO()	394
11.11.3	EFI_ADAPTER_INFORMATION_PROTOCOL.EFI_ADAPTER_INFO_SET_INFO()	395
11.11.4	EFI_ADAPTER_INFORMATION_PROTOCOL.EFI_ADAPTER_INFO_GET_SUPPORTED_TYPES()	396
11.12	EFI Adapter Information Protocol Information Types	397
11.12.1	Network Media State	397
11.12.2	Network Boot	397
11.12.3	SAN MAC Address	398
11.12.4	IPv6 Support from UNDI	399

11.12.5	Network Media Type	399
11.12.6	Coherent Device Attribute Table (CDAT) Type	400
12	Protocols — Console Support	401
12.1	Console I/O Protocol	401
12.1.1	Overview	401
12.1.2	ConsoleIn Definition	402
12.2	Simple Text Input Ex Protocol	402
12.2.1	EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL	402
12.2.2	EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL.Reset()	403
12.2.3	EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL.ReadKeyStrokeEx()	404
12.2.4	EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL.SetState()	406
12.2.5	EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL.RegisterKeyNotify()	407
12.2.6	EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL.UnregisterKeyNotify()	408
12.3	Simple Text Input Protocol	408
12.3.1	EFI_SIMPLE_TEXT_INPUT_PROTOCOL	409
12.3.2	EFI_SIMPLE_TEXT_INPUT_PROTOCOL.Reset()	409
12.3.3	EFI_SIMPLE_TEXT_INPUT_PROTOCOL.ReadKeyStroke()	410
12.3.4	ConsoleOut or StandardError	411
12.4	Simple Text Output Protocol	411
12.4.1	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL	411
12.4.2	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.Reset()	413
12.4.3	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.OutputString()	414
12.4.4	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.TestString()	417
12.4.5	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.QueryMode()	417
12.4.6	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.SetMode()	418
12.4.7	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.SetAttribute()	419
12.4.8	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.ClearScreen()	421
12.4.9	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.SetCursorPosition()	421
12.4.10	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL.EnableCursor()	422
12.5	Simple Pointer Protocol	423
12.5.1	EFI_SIMPLE_POINTER_PROTOCOL	423
12.5.2	EFI_SIMPLE_POINTER_PROTOCOL.Reset()	424
12.5.3	EFI_SIMPLE_POINTER_PROTOCOL.GetState()	425
12.6	EFI Simple Pointer Device Paths	426
12.7	Absolute Pointer Protocol	429
12.7.1	EFI_ABSOLUTE_POINTER_PROTOCOL	429
12.7.2	EFI_ABSOLUTE_POINTER_PROTOCOL.Reset()	431
12.7.3	EFI_ABSOLUTE_POINTER_PROTOCOL.GetState()	432
12.8	Serial I/O Protocol	433
12.8.1	EFI_SERIAL_IO_PROTOCOL	433
12.8.2	Serial Device Identification	436
12.8.3	Serial Device Type GUIDs	436
12.8.3.1	EFI_SERIAL_IO_PROTOCOL.Reset()	437
12.8.3.2	EFI_SERIAL_IO_PROTOCOL.SetAttributes()	438
12.8.3.3	EFI_SERIAL_IO_PROTOCOL.SetControl()	439
12.8.3.4	EFI_SERIAL_IO_PROTOCOL.GetControl()	440
12.8.3.5	EFI_SERIAL_IO_PROTOCOL.Write()	441
12.8.3.6	EFI_SERIAL_IO_PROTOCOL.Read()	442
12.9	Graphics Output Protocol	443
12.9.1	Blt Buffer	443
12.9.2	EFI_GRAPHICS_OUTPUT_PROTOCOL	445
12.9.2.1	EFI_GRAPHICS_OUTPUT_PROTOCOL.QueryMode()	449
12.9.2.2	EFI_GRAPHICS_OUTPUT_PROTOCOL.SetMode()	450

12.9.2.3	EFI_GRAPHICS_OUTPUT_PROTOCOL.Blt()	450
12.9.2.4	EFI_EDID_DISCOVERED_PROTOCOL	452
12.9.2.5	EFI_EDID_ACTIVE_PROTOCOL	453
12.9.2.6	EFI_EDID_OVERRIDE_PROTOCOL	454
12.9.2.7	EFI_EDID_OVERRIDE_PROTOCOL.GetEdid()	454
12.10	Rules for PCI/AGP Devices	456
13	Protocols – Media Access	458
13.1	Load File Protocol	458
13.1.1	EFI_LOAD_FILE_PROTOCOL	458
13.1.2	EFI_LOAD_FILE_PROTOCOL.LoadFile()	459
13.2	Load File 2 Protocol	460
13.2.1	EFI_LOAD_FILE2_PROTOCOL	460
13.2.2	EFI_LOAD_FILE2_PROTOCOL.LoadFile()	461
13.3	File System Format	462
13.3.1	System Partition	462
13.3.1.1	File System Format	462
13.3.1.2	File Names	463
13.3.1.3	Directory Structure	463
13.3.2	Partition Discovery	464
13.3.2.1	ISO-9660 and El Torito	465
13.3.3	Number and Location of System Partitions	466
13.3.4	Media Formats	466
13.3.4.1	Removable Media	466
13.3.4.2	Diskette	466
13.3.4.3	Hard Drive	467
13.3.4.4	CD-ROM and DVD-ROM	467
13.3.4.5	Network	467
13.4	Simple File System Protocol	467
13.4.1	EFI_SIMPLE_FILE_SYSTEM_PROTOCOL	467
13.4.2	EFI_SIMPLE_FILE_SYSTEM_PROTOCOL.OpenVolume()	468
13.5	File Protocol	469
13.5.1	EFI_FILE_PROTOCOL	469
13.5.2	EFI_FILE_PROTOCOL.Open()	471
13.5.3	EFI_FILE_PROTOCOL.Close()	473
13.5.4	EFI_FILE_PROTOCOL.Delete()	473
13.5.5	EFI_FILE_PROTOCOL.Read()	474
13.5.6	EFI_FILE_PROTOCOL.Write()	475
13.5.7	EFI_FILE_PROTOCOL.OpenEx()	476
13.5.8	EFI_FILE_PROTOCOL.ReadEx()	478
13.5.9	EFI_FILE_PROTOCOL.WriteEx()	479
13.5.10	EFI_FILE_PROTOCOL.FlushEx()	480
13.5.11	EFI_FILE_PROTOCOL.SetPosition()	481
13.5.12	EFI_FILE_PROTOCOL.GetPosition()	482
13.5.13	EFI_FILE_PROTOCOL.GetInfo()	483
13.5.14	EFI_FILE_PROTOCOL.SetInfo()	484
13.5.15	EFI_FILE_PROTOCOL.Flush()	485
13.5.16	EFI_FILE_INFO	485
13.5.17	EFI_FILE_SYSTEM_INFO	487
13.5.18	EFI_FILE_SYSTEM_VOLUME_LABEL	488
13.6	Tape Boot Support	488
13.6.1	Tape I/O Support	488
13.6.2	Tape I/O Protocol	488
13.6.2.1	EFI_TAPE_IO_PROTOCOL	489

13.6.2.2	EFI_TAPE_IO_PROTOCOL.TapeRead()	490
13.6.2.3	EFI_TAPE_IO_PROTOCOL.TapeWrite()	491
13.6.2.4	EFI_TAPE_IO_PROTOCOL.TapeRewind()	492
13.6.2.5	EFI_TAPE_IO_PROTOCOL.TapeSpace()	493
13.6.2.6	EFI_TAPE_IO_PROTOCOL.TapeWriteFM()	494
13.6.2.7	EFI_TAPE_IO_PROTOCOL.TapeReset()	495
13.6.3	Tape Header Format	496
13.7	Disk I/O Protocol	497
13.7.1	EFI_DISK_IO_PROTOCOL	497
13.7.2	EFI_DISK_IO_PROTOCOL.ReadDisk()	498
13.7.3	EFI_DISK_IO_PROTOCOL.WriteDisk()	499
13.8	Disk I/O 2 Protocol	500
13.8.1	EFI_DISK_IO2_PROTOCOL	500
13.8.2	EFI_DISK_IO2_PROTOCOL.Cancel()	501
13.8.3	EFI_DISK_IO2_PROTOCOL.ReadDiskEx()	502
13.8.4	EFI_DISK_IO2_PROTOCOL.WriteDiskEx()	503
13.8.5	EFI_DISK_IO2_PROTOCOL.FlushDiskEx()	504
13.9	Block I/O Protocol	506
13.9.1	EFI_BLOCK_IO_PROTOCOL	506
13.9.2	EFI_BLOCK_IO_PROTOCOL.Reset()	508
13.9.3	EFI_BLOCK_IO_PROTOCOL.ReadBlocks()	509
13.9.4	EFI_BLOCK_IO_PROTOCOL.WriteBlocks()	510
13.9.5	EFI_BLOCK_IO_PROTOCOL.FlushBlocks()	511
13.10	Block I/O 2 Protocol	512
13.10.1	EFI_BLOCK_IO2_PROTOCOL	512
13.10.2	EFI_BLOCK_IO2_PROTOCOL.Reset()	513
13.10.3	EFI_BLOCK_IO2_PROTOCOL.ReadBlocksEx()	513
13.10.4	EFI_BLOCK_IO2_PROTOCOL.WriteBlocksEx()	515
13.10.5	EFI_BLOCK_IO2_PROTOCOL.FlushBlocksEx()	516
13.11	Inline Cryptographic Interface Protocol	517
13.11.1	EFI_BLOCK_IO_CRYPTO_PROTOCOL	517
13.11.2	EFI_BLOCK_IO_CRYPTO_PROTOCOL.Reset()	521
13.11.3	EFI_BLOCK_IO_CRYPTO_PROTOCOL.GetCapabilities()	522
13.11.4	EFI_BLOCK_IO_CRYPTO_PROTOCOL.SetConfiguration()	523
13.11.5	EFI_BLOCK_IO_CRYPTO_PROTOCOL.GetConfiguration()	524
13.11.6	EFI_BLOCK_IO_CRYPTO_PROTOCOL.ReadExtended()	525
13.11.7	EFI_BLOCK_IO_CRYPTO_PROTOCOL.WriteExtended()	527
13.11.8	EFI_BLOCK_IO_CRYPTO_PROTOCOL.FlushBlocks()	528
13.12	Erase Block Protocol	529
13.12.1	EFI_ERASE_BLOCK_PROTOCOL	529
13.12.2	EFI_ERASE_BLOCK_PROTOCOL.EraseBlocks()	530
13.13	ATA Pass Thru Protocol	531
13.13.1	EFI_ATA_PASS_THRU_PROTOCOL	531
13.13.2	EFI_ATA_PASS_THRU_PROTOCOL.PassThru()	534
13.13.3	EFI_ATA_PASS_THRU_PROTOCOL.GetNextPort()	539
13.13.4	EFI_ATA_PASS_THRU_PROTOCOL.GetNextDevice()	540
13.13.5	EFI_ATA_PASS_THRU_PROTOCOL.BuildDevicePath()	541
13.13.6	EFI_ATA_PASS_THRU_PROTOCOL.GetDevice()	542
13.13.7	EFI_ATA_PASS_THRU_PROTOCOL.ResetPort()	543
13.13.8	EFI_ATA_PASS_THRU_PROTOCOL.ResetDevice()	544
13.14	Storage Security Command Protocol	545
13.14.1	EFI_STORAGE_SECURITY_COMMAND_PROTOCOL	545
13.14.2	EFI_STORAGE_SECURITY_COMMAND_PROTOCOL.ReceiveData()	546
13.14.3	EFI_STORAGE_SECURITY_COMMAND_PROTOCOL.SendData()	547

13.15	NVM Express Pass Through Protocol	549
13.15.1	EFI_NVM_EXPRESS_PASS_THRU_PROTOCOL	549
13.15.2	EFI_NVM_EXPRESS_PASS_THRU_PROTOCOL.PassThru()	551
13.15.3	EFI_NVM_EXPRESS_PASS_THRU_PROTOCOL.GetNextNamespace()	555
13.15.4	EFI_NVM_EXPRESS_PASS_THRU_PROTOCOL.BuildDevicePath()	556
13.15.5	EFI_NVM_EXPRESS_PASS_THRU_PROTOCOL.GetNamespace()	557
13.16	SD MMC Pass Thru Protocol	558
13.16.1	EFI_SD_MMC_PASS_THRU_PROTOCOL	558
13.16.2	EFI_SD_MMC_PASS_THRU_PROTOCOL.PassThru()	559
13.16.3	EFI_SD_MMC_PASS_THRU_PROTOCOL.GetNextSlot()	561
13.16.4	EFI_SD_MMC_PASS_THRU_PROTOCOL.BuildDevicePath()	562
13.16.5	EFI_SD_MMC_PASS_THRU_PROTOCOL.GetSlotNumber()	563
13.16.6	EFI_SD_MMC_PASS_THRU_PROTOCOL.ResetDevice()	564
13.17	RAM Disk Protocol	564
13.17.1	EFI_RAM_DISK_PROTOCOL	564
13.17.2	EFI_RAM_DISK_PROTOCOL.Register()	565
13.17.3	EFI_RAM_DISK_PROTOCOL.Unregister()	566
13.18	Partition Information Protocol	567
13.19	NVDIMM Label Protocol	568
13.19.1	EFI_NVDIMM_LABEL_PROTOCOL	568
13.19.2	EFI_NVDIMM_LABEL_PROTOCOL.LabelStorageInformation()	569
13.19.3	EFI_NVDIMM_LABEL_PROTOCOL.LabelStorageRead()	570
13.19.4	EFI_NVDIMM_LABEL_PROTOCOL.LabelStorageWrite()	571
13.19.5	Label Storage Area Description	577
13.19.5.1	Updating the Name of a Namespace Description	582
13.20	EFI UFS Device Config Protocol	583
13.20.1	EFI_UFS_DEVICE_CONFIG_PROTOCOL	583
13.20.2	EFI_UFS_DEVICE_CONFIG_PROTOCOL.RwUfsDescriptor()	583
13.20.3	EFI_UFS_DEVICE_CONFIG_PROTOCOL.RwUfsFlag()	584
13.20.4	EFI_UFS_DEVICE_CONFIG_PROTOCOL.RwUfsAttribute()	585
14	Protocols — PCI Bus Support	587
14.1	PCI Root Bridge I/O Support	587
14.1.1	PCI Root Bridge I/O Overview	587
14.1.2	Sample PCI Architectures	589
14.2	PCI Root Bridge I/O Protocol	591
14.2.1	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL	591
14.2.2	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.PollMem()	598
14.2.3	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.PollIo()	599
14.2.4	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Mem.Read()	600
14.2.5	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Mem.Write()	600
14.2.6	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Io.Read()	602
14.2.7	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Io.Write()	602
14.2.8	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Pci.Read()	603
14.2.9	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Pci.Write()	603
14.2.10	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.CopyMem()	604
14.2.11	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Map()	606
14.2.12	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Unmap()	607
14.2.13	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.AllocateBuffer()	608
14.2.14	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.FreeBuffer()	609
14.2.15	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Flush()	610
14.2.16	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.GetAttributes()	611
14.2.17	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.SetAttributes()	612
14.2.18	EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Configuration()	613

14.2.19	PCI Root Bridge Device Paths	615
14.3	PCI Driver Model	617
14.3.1	PCI Driver Initialization	618
14.3.2	Driver Diagnostics Protocol	619
14.3.3	Component Name Protocol	619
14.3.4	Driver Family Override Protocol	619
14.3.5	PCI Bus Drivers	619
14.3.6	Driver Binding Protocol for PCI Bus Drivers	620
14.3.7	PCI Enumeration	623
14.3.8	PCI Device Drivers	623
14.3.9	Driver Binding Protocol for PCI Device Drivers	623
14.4	EFI PCI I/O Protocol	623
14.4.1	EFI_PCI_IO_PROTOCOL	625
14.4.2	EFI_PCI_IO_PROTOCOL.PollMem()	632
14.4.3	EFI_PCI_IO_PROTOCOL.PollIo()	633
14.4.4	EFI_PCI_IO_PROTOCOL.Mem.Read()	635
14.4.5	EFI_PCI_IO_PROTOCOL.Mem.Write()	635
14.4.6	EFI_PCI_IO_PROTOCOL.Io.Read()	636
14.4.7	EFI_PCI_IO_PROTOCOL.Io.Write()	636
14.4.8	EFI_PCI_IO_PROTOCOL.Pci.Read()	638
14.4.9	EFI_PCI_IO_PROTOCOL.Pci.Write()	638
14.4.10	EFI_PCI_IO_PROTOCOL.CopyMem()	639
14.4.11	EFI_PCI_IO_PROTOCOL.Map()	641
14.4.12	EFI-PCI-IO-PROTOCOL-Unmap()	642
14.4.13	EFI_PCI_IO_PROTOCOL.AllocateBuffer()	643
14.4.14	EFI_PCI_IO_PROTOCOL.FreeBuffer()	644
14.4.15	EFI_PCI_IO_PROTOCOL.Flush()	645
14.4.16	EFI_PCI_IO_PROTOCOL.GetLocation()	646
14.4.17	EFI_PCI_IO_PROTOCOL.Attributes()	647
14.4.18	EFI_PCI_IO_PROTOCOL.GetBarAttributes()	649
14.4.19	EFI_PCI_IO_PROTOCOL.SetBarAttributes()	651
14.4.20	PCI Device Paths	652
14.4.21	PCI Option ROMs	653
14.4.22	PCI Bus Driver Responsibilities	656
14.4.23	PCI Device Driver Responsibilities	657
14.4.24	Nonvolatile Storage	659
14.4.25	PCI Hot-Plug Events	659
15	Protocols — SCSI Driver Models and Bus Support	663
15.1	SCSI Driver Model Overview	663
15.2	SCSI Bus Drivers	664
15.2.1	Driver Binding Protocol for SCSI Bus Drivers	664
15.2.2	SCSI Enumeration	665
15.3	SCSI Device Drivers	665
15.3.1	Driver Binding Protocol for SCSI Device Drivers	665
15.4	EFI SCSI I/O Protocol	666
15.4.1	EFI_SCSI_IO_PROTOCOL	666
15.4.2	EFI_SCSI_IO_PROTOCOL.GetDeviceType()	667
15.4.3	EFI_SCSI_IO_PROTOCOL.GetDeviceLocation()	668
15.4.4	EFI_SCSI_IO_PROTOCOL.ResetBus()	669
15.4.5	EFI_SCSI_IO_PROTOCOL.ResetDevice()	670
15.4.6	EFI_SCSI_IO_PROTOCOL.ExecuteScsiCommand()	670
15.5	SCSI Device Paths	674
15.5.1	SCSI Device Path Example	674

15.5.2	ATAPI Device Path Example	675
15.5.3	Fibre Channel Device Path Example	676
15.5.4	InfiniBand Device Path Example	676
15.6	SCSI Pass Thru Device Paths	677
15.7	Extended SCSI Pass Thru Protocol	680
15.7.1	EFI_EXT_SCSI_PASS_THRU_PROTOCOL	680
15.7.2	EFI_EXT_SCSI_PASS_THRU_PROTOCOL.PassThru()	682
15.7.3	EFI_EXT_SCSI_PASS_THRU_PROTOCOL.GetNextTargetLun()	687
15.7.4	EFI_EXT_SCSI_PASS_THRU_PROTOCOL.BuildDevicePath()	688
15.7.5	EFI_EXT_SCSI_PASS_THRU_PROTOCOL.GetTargetLun()	689
15.7.6	EFI_EXT_SCSI_PASS_THRU_PROTOCOL.ResetChannel()	690
15.7.7	EFI_EXT_SCSI_PASS_THRU_PROTOCOL.ResetTargetLun()	690
15.7.8	EFI_EXT_SCSI_PASS_THRU_PROTOCOL.GetNextTarget()	691
16	Protocols — iSCSI Boot	693
16.1	Overview	693
16.1.1	iSCSI UEFI Driver Layering	693
16.2	EFI iSCSI Initiator Name Protocol	693
16.2.1	EFI_ISCSI_INITIATOR_NAME_PROTOCOL	694
16.2.2	EFI_ISCSI_INITIATOR_NAME_PROTOCOL.Get()	694
16.2.3	EFI_ISCSI_INITIATOR_NAME_PROTOCOL.Set()	695
17	Protocols — USB Support	697
17.1	USB Host Controller Protocol	697
17.1.1	USB Host Controller Protocol Overview	697
17.1.2	EFI_USB2_HC_PROTOCOL	697
17.1.3	EFI_USB2_HC_PROTOCOL.GetCapability()	699
17.1.4	EFI_USB2_HC_PROTOCOL.Reset()	700
17.1.5	EFI_USB2_HC_PROTOCOL.GetState()	702
17.1.6	EFI_USB2_HC_PROTOCOL.SetState()	703
17.1.7	EFI_USB2_HC_PROTOCOL.ControlTransfer()	704
17.1.8	EFI_USB2_HC_PROTOCOL.BulkTransfer()	706
17.1.9	EFI_USB2_HC_PROTOCOL.AsyncInterruptTransfer()	708
17.1.10	EFI_USB2_HC_PROTOCOL.SyncInterruptTransfer()	710
17.1.11	EFI_USB2_HC_PROTOCOL.IsochronousTransfer()	712
17.1.12	EFI_USB2_HC_PROTOCOL.AsyncIsochronousTransfer()	714
17.1.13	EFI_USB2_HC_PROTOCOL.GetRootHubPortStatus()	716
17.1.14	EFI_USB2_HC_PROTOCOL.SetRootHubPortFeature()	720
17.1.15	EFI_USB2_HC_PROTOCOL.ClearRootHubPortFeature()	722
17.2	USB Driver Model	723
17.2.1	Scope	723
17.2.2	USB Bus Driver	723
17.2.2.1	USB Bus Driver Entry Point	724
17.2.2.2	Driver Binding Protocol for USB Bus Drivers	724
17.2.2.3	USB Hot-Plug Event	724
17.2.2.4	USB Bus Enumeration	724
17.2.3	USB Device Driver	725
17.2.3.1	USB Device Driver Entry Point	725
17.2.3.2	Driver Binding Protocol for USB Device Drivers	725
17.2.4	USB I/O Protocol	726
17.2.5	EFI_USB_IO_PROTOCOL	726
17.2.6	EFI_USB_IO_PROTOCOL.UsbControlTransfer()	728
17.2.7	EFI_USB_IO_PROTOCOL.UsbBulkTransfer()	730
17.2.8	EFI_USB_IO_PROTOCOL.UsbAsyncInterruptTransfer()	731

17.2.9	EFI_USB_IO_PROTOCOL.UsbSyncInterruptTransfer()	734
17.2.10	EFI_USB_IO_PROTOCOL.UsbIsochronousTransfer()	735
17.2.11	EFI_USB_IO_PROTOCOL.UsbAsyncIsochronousTransfer()	736
17.2.12	EFI_USB_IO_PROTOCOL.UsbGetDeviceDescriptor()	737
17.2.13	EFI_USB_IO_PROTOCOL.UsbGetConfigDescriptor()	738
17.2.14	EFI_USB_IO_PROTOCOL.UsbGetInterfaceDescriptor()	739
17.2.15	EFI_USB_IO_PROTOCOL.UsbGetEndpointDescriptor()	740
17.2.16	EFI_USB_IO_PROTOCOL.UsbGetStringDescriptor()	742
17.2.17	EFI_USB_IO_PROTOCOL.UsbGetSupportedLanguages()	743
17.2.18	EFI_USB_IO_PROTOCOL.UsbPortReset()	743
17.3	USB Function Protocol	744
17.3.1	EFI_USBFN_IO_PROTOCOL	744
17.3.2	EFI_USBFN_IO_PROTOCOL.DetectPort()	746
17.3.3	EFI_USBFN_IO_PROTOCOL.ConfigureEnableEndpoints()	748
17.3.4	EFI_USBFN_IO_PROTOCOL.GetEndpointMaxPacketSize()	749
17.3.5	EFI_USBFN_IO_PROTOCOL.GetDeviceInfo()	750
17.3.6	EFI_USBFN_IO_PROTOCOL.GetVendorIdProductId()	751
17.3.7	EFI_USBFN_IO_PROTOCOL.AbortTransfer()	752
17.3.8	EFI_USBFN_IO_PROTOCOL.GetEndpointStallState()	753
17.3.9	EFI_USBFN_IO_PROTOCOL.SetEndpointStallState()	753
17.3.10	EFI_USBFN_IO_PROTOCOL.EventHandler()	754
17.3.11	EFI_USBFN_IO_PROTOCOL.Transfer()	758
17.3.12	EFI_USBFN_IO_PROTOCOL.GetMaxTransferSize()	759
17.3.13	EFI_USBFN_IO_PROTOCOL.AllocateTransferBuffer()	760
17.3.14	EFI_USBFN_IO_PROTOCOL.FreeTransferBuffer()	761
17.3.15	EFI_USBFN_IO_PROTOCOL.StartController()	761
17.3.16	EFI_USBFN_IO_PROTOCOL.StopController()	762
17.3.16.1	Description	762
17.3.17	EFI_USBFN_IO_PROTOCOL.SetEndpointPolicy()	762
17.3.18	EFI_USBFN_IO_PROTOCOL.GetEndpointPolicy()	765
17.3.19	USB Function Sequence Diagram	766
18	Protocols — Debugger Support	768
18.1	Overview	768
18.2	EFI Debug Support Protocol	769
18.2.1	EFI Debug Support Protocol Overview	769
18.2.2	EFI_DEBUG_SUPPORT_PROTOCOL	769
18.2.3	EFI_DEBUG_SUPPORT_PROTOCOL.GetMaximumProcessorIndex()	771
18.2.4	EFI_DEBUG_SUPPORT_PROTOCOL.RegisterPeriodicCallback()	771
18.2.5	EFI_DEBUG_SUPPORT_PROTOCOL.RegisterExceptionCallback()	780
18.2.6	EFI_DEBUG_SUPPORT_PROTOCOL.InvalidateInstructionCache()	785
18.3	EFI Debugport Protocol	786
18.3.1	EFI Debugport Overview	786
18.3.2	EFI_DEBUGPORT_PROTOCOL	786
18.3.3	EFI_DEBUGPORT_PROTOCOL.Reset()	787
18.3.4	EFI_DEBUGPORT_PROTOCOL.Write()	788
18.3.5	EFI_DEBUGPORT_PROTOCOL.Read()	788
18.3.6	EFI_DEBUGPORT_PROTOCOL.Poll()	789
18.3.7	Debugport Device Path	790
18.3.8	EFI Debugport Variable	790
18.4	EFI Debug Support Table	791
18.4.1	Overview	791
18.4.2	EFI System Table Location	792
18.4.3	EFI Image Info	793

19	Protocols — Compression Algorithm Specification	795
19.1	Algorithm Overview	795
19.2	Data Format	796
19.2.1	Bit Order	796
19.2.2	Overall Structure	797
19.2.3	Block Structure	797
19.2.3.1	Block Header	798
19.2.3.2	Block Body	803
19.3	Compressor Design	803
19.3.1	Overall Process	803
19.3.2	String Info Log	805
19.3.2.1	Data Structures	805
19.3.2.2	Searching the Tree	805
19.3.2.3	Adding String Info	806
19.3.2.4	Deleting String Info	806
19.3.3	Huffman Code Generation	808
19.3.3.1	Huffman Tree Generation	808
19.3.3.2	Code Length Adjustment	808
19.3.3.3	Code Generation	809
19.4	Decompressor Design	809
19.5	Decompress Protocol	810
19.5.1	EFI_DECOMPRESS_PROTOCOL	810
19.5.2	EFI_DECOMPRESS_PROTOCOL.GetInfo()	810
19.5.3	EFI_DECOMPRESS_PROTOCOL.Decompress()	812
20	Protocols — ACPI Protocols	814
20.1	EFI_ACPI_TABLE_PROTOCOL	814
20.2	EFI_ACPI_TABLE_PROTOCOL.InstallAcpiTable()	814
20.3	EFI_ACPI_TABLE_PROTOCOL.UninstallAcpiTable()	816
21	Protocols — String Services	817
21.1	Unicode Collation Protocol	817
21.1.1	EFI_UNICODE_COLLATION_PROTOCOL	817
21.1.2	EFI_UNICODE_COLLATION_PROTOCOL.StriColl()	818
21.1.3	EFI_UNICODE_COLLATION_PROTOCOL.MetaiMatch()	819
21.1.4	EFI_UNICODE_COLLATION_PROTOCOL.StrLwr()	820
21.1.5	EFI_UNICODE_COLLATION_PROTOCOL.StrUp()	821
21.1.6	EFI_UNICODE_COLLATION_PROTOCOL.FatToStr()	821
21.1.7	EFI_UNICODE_COLLATION_PROTOCOL.StrToFat()	822
21.2	Regular Expression Protocol	823
21.2.1	EFI_REGULAR_EXPRESSION_PROTOCOL	823
21.2.2	EFI_REGULAR_EXPRESSION_PROTOCOL.MatchString()	823
21.2.3	EFI_REGULAR_EXPRESSION_PROTOCOL.GetInfo()	825
21.2.4	EFI Regular Expression Syntax Type Definitions	826
22	EFI Byte Code Virtual Machine	827
22.1	Overview	827
22.1.1	Processor Architecture Independence	827
22.1.2	OS Independent	828
22.1.3	EFI Compliant	828
22.1.4	Coexistence of Legacy Option ROMs	828
22.1.5	Relocatable Image	828
22.1.6	Size Restrictions Based on Memory Available	829
22.2	Memory Ordering	829

22.3	Virtual Machine Registers	829
22.4	Natural Indexing	830
22.4.1	Sign Bit	831
22.4.2	Bits Assigned to Natural Units	831
22.4.3	Constant	831
22.4.4	Natural Units	831
22.5	EBC Instruction Operands	832
22.5.1	Direct Operands	832
22.5.2	Indirect Operands	832
22.5.3	Indirect with Index Operands	832
22.5.4	Immediate Operands	833
22.6	EBC Instruction Syntax	833
22.7	Instruction Encoding	834
22.7.1	Instruction Opcode Byte Encoding	834
22.7.2	Instruction Operands Byte Encoding	834
22.7.3	Index/Immediate Data Encoding	835
22.8	EBC Instruction Set	835
22.8.1	ADD	835
22.8.2	AND	836
22.8.3	ASHR	837
22.8.4	BREAK	838
22.8.5	CALL	840
22.8.6	CMP	842
22.8.7	CMPI	843
22.8.8	DIV	844
22.8.9	DIVU	845
22.8.10	EXTNDB	846
22.8.11	EXTNDD	847
22.8.12	EXTNDW	848
22.8.13	JMP	849
22.8.14	JMP8	851
22.8.15	LOADSP	852
22.8.16	MOD	852
22.8.17	MODU	853
22.8.18	MOV	854
22.8.19	MOVI	856
22.8.20	MOVIn	857
22.8.21	MOVn	858
22.8.22	MOVREL	859
22.8.23	MOVsn	860
22.8.24	MUL	862
22.8.25	MULU	863
22.8.26	NEG	864
22.8.27	NOT	865
22.8.28	OR	866
22.8.29	POP	867
22.8.30	POPn	868
22.8.31	PUSH	869
22.8.32	PUSHn	870
22.8.33	RET	870
22.8.34	SHL	871
22.8.35	SHR	872
22.8.36	STORESP	873
22.8.37	SUB	874

22.8.38 XOR	875
22.9 Runtime and Software Conventions	876
22.9.1 Calling Outside VM	876
22.9.2 Calling Inside VM	876
22.9.3 Parameter Passing	876
22.9.4 Return Values	876
22.9.5 Binary Format	876
22.10 Architectural Requirements	876
22.10.1 EBC Image Requirements	877
22.10.2 EBC Execution Interfacing Requirements	877
22.10.3 Interfacing Function Parameters Requirements	877
22.10.4 Function Return Requirements	877
22.10.5 Function Return Values Requirements	877
22.11 EBC Interpreter Protocol	878
22.11.1 EFI_EBC_PROTOCOL	878
22.11.2 EFI_EBC_PROTOCOL.CreateThunk()	879
22.11.3 EFI_EBC_PROTOCOL.UnloadImage()	879
22.11.4 EFI_EBC_PROTOCOL.RegisterICacheFlush()	880
22.11.5 EFI_EBC_PROTOCOL.GetVersion()	881
22.12 EBC Tools	882
22.12.1 EBC C Compiler	882
22.12.2 C Coding Convention	882
22.12.3 EBC Interface Assembly Instructions	882
22.12.4 Stack Maintenance and Argument Passing	882
22.12.5 Native to EBC Arguments Calling Convention	883
22.12.6 EBC to Native Arguments Calling Convention	883
22.12.7 EBC to EBC Arguments Calling Convention	883
22.12.8 Function Returns	883
22.12.9 Function Return Values	883
22.12.10 Thunking	884
22.12.10.1 Thunking EBC to Native Code	884
22.12.10.2 Thunking Native Code to EBC	884
22.12.10.3 Thunking EBC to EBC	885
22.12.11 EBC Linker	885
22.12.12 Image Loader	886
22.12.13 Debug Support	886
22.13 VM Exception Handling	886
22.13.1 Divide By 0 Exception	887
22.13.2 Debug Break Exception	887
22.13.3 Invalid Opcode Exception	887
22.13.4 Stack Fault Exception	887
22.13.5 Alignment Exception	887
22.13.6 Instruction Encoding Exception	887
22.13.7 Bad Break Exception	887
22.13.8 Undefined Exception	887
22.14 Option ROM Formats	888
22.14.1 EFI Drivers for PCI Add-in Cards	888
22.14.2 Non-PCI Bus Support	888
23 Firmware Update and Reporting	889
23.1 Firmware Management Protocol	889
23.1.1 EFI_FIRMWARE_MANAGEMENT_PROTOCOL	889
23.1.2 EFI_FIRMWARE_MANAGEMENT_PROTOCOL.GetImageInfo()	890
23.1.3 EFI_FIRMWARE_MANAGEMENT_PROTOCOL.GetImage()	896

23.1.4	EFI_FIRMWARE_MANAGEMENT_PROTOCOL.SetImage()	897
23.1.5	EFI_FIRMWARE_MANAGEMENT_PROTOCOL.CheckImage()	899
23.1.6	EFI_FIRMWARE_MANAGEMENT_PROTOCOL.GetPackageInfo()	900
23.1.7	EFI_FIRMWARE_MANAGEMENT_PROTOCOL.SetPackageInfo()	902
23.2	Dependency Expression Instruction Set	903
23.2.1	PUSH_GUID	904
23.2.2	PUSH_VERSION	904
23.2.3	DECLARE_VERSION_NAME	905
23.2.4	AND	905
23.2.5	OR	906
23.2.6	NOT	906
23.2.7	TRUE	907
23.2.8	FALSE	907
23.2.9	EQ	908
23.2.10	GT	908
23.2.11	GTE	909
23.2.12	LT	909
23.2.13	LTE	910
23.2.14	END	910
23.2.15	DECLARE_LENGTH	911
23.3	Delivering Capsules Containing Updates to Firmware Management Protocol	911
23.3.1	EFI_FIRMWARE_MANAGEMENT_CAPSULE_ID_GUID	911
23.3.2	DEFINED FIRMWARE MANAGEMENT PROTOCOL DATA CAPSULE STRUCTURE	912
23.3.3	Firmware Processing of the Capsule Identified by EFI_FIRMWARE_MANAGEMENT_CAPSULE_ID_GUID	915
23.4	EFI System Resource Table	917
23.4.1	EFI_SYSTEM_RESOURCE_TABLE	917
23.4.2	Adding and Removing Devices from the ESRT	919
23.4.3	ESRT and Firmware Management Protocol	919
23.4.4	Mapping Firmware Management Protocol Descriptors to ESRT Entries	919
23.5	Delivering Capsule Containing JSON payload	920
23.5.1	EFI_JSON_CAPSULE_ID_GUID	920
23.5.2	Defined JSON Capsule Data Structure	921
23.5.3	Firmware Processing of the Capsule Identified by EFI_JSON_CAPSULE_ID_GUID	922
24	Network Protocols — SNP, PXE, BIS and HTTP Boot	924
24.1	Simple Network Protocol	924
24.1.1	EFI_SIMPLE_NETWORK_PROTOCOL	924
24.1.2	EFI_SIMPLE_NETWORK.Start()	928
24.1.3	EFI_SIMPLE_NETWORK.Stop()	929
24.1.4	EFI_SIMPLE_NETWORK.Initialize()	929
24.1.5	EFI_SIMPLE_NETWORK.Reset()	930
24.1.6	EFI_SIMPLE_NETWORK.Shutdown()	931
24.1.7	EFI_SIMPLE_NETWORK.ReceiveFilters()	932
24.1.8	EFI_SIMPLE_NETWORK.StationAddress()	934
24.1.9	EFI_SIMPLE_NETWORK.Statistics()	935
24.1.10	EFI_SIMPLE_NETWORK.MCastIPtoMAC()	938
24.1.11	EFI_SIMPLE_NETWORK.NvData()	939
24.1.12	EFI_SIMPLE_NETWORK.GetStatus()	940
24.1.13	EFI_SIMPLE_NETWORK.Transmit()	941
24.1.14	EFI_SIMPLE_NETWORK.Receive()	943
24.2	Network Interface Identifier Protocol	944
24.2.1	EFI_NETWORK_INTERFACE_IDENTIFIER_PROTOCOL	944
24.3	PXE Base Code Protocol	946
24.3.1	EFI_PXE_BASE_CODE_PROTOCOL	947

24.3.2	DHCP Packet Data Types	952
24.3.3	IP Receive Filter Settings	954
24.3.4	ARP Cache Entries	954
24.3.5	Filter Operations for UDP Read/Write Functions	955
24.3.6	EFI_PXE_BASE_CODE_PROTOCOL.Start()	957
24.3.7	EFI_PXE_BASE_CODE_PROTOCOL.Stop()	958
24.3.8	EFI_PXE_BASE_CODE_PROTOCOL.Dhcp()	959
24.3.9	EFI_PXE_BASE_CODE_PROTOCOL.Discover()	960
24.3.10	EFI_PXE_BASE_CODE_PROTOCOL.Mtftp()	963
24.3.11	EFI_PXE_BASE_CODE_PROTOCOL.UdpWrite()	966
24.3.12	EFI_PXE_BASE_CODE_PROTOCOL.UdpRead()	968
24.3.13	EFI_PXE_BASE_CODE_PROTOCOL.SetIpFilter()	970
24.3.14	EFI_PXE_BASE_CODE_PROTOCOL.Arp()	971
24.3.15	EFI_PXE_BASE_CODE_PROTOCOL.SetParameters()	972
24.3.16	EFI_PXE_BASE_CODE_PROTOCOL.SetStationIp()	974
24.3.17	EFI_PXE_BASE_CODE_PROTOCOL.SetPackets()	975
24.3.18	Netboot6	976
	24.3.18.1 DHCP6 options for PXE	977
	24.3.18.2 IPv6-based PXE boot	978
	24.3.18.3 Proxy DHCP6	980
24.4	PXE Base Code Callback Protocol	980
24.4.1	EFI_PXE_BASE_CODE_CALLBACK_PROTOCOL	980
24.4.2	EFI_PXE_BASE_CODE_CALLBACK.Callback()	983
24.5	Boot Integrity Services Protocol	984
24.5.1	EFI_BIS_PROTOCOL	985
24.5.2	EFI_BIS_PROTOCOL.Initialize()	986
24.5.3	EFI_BIS_PROTOCOL.Shutdown()	989
24.5.4	EFI_BIS_PROTOCOL.Free()	990
24.5.5	EFI_BIS_PROTOCOL.GetBootObjectAuthorizationCertificate()	991
24.5.6	EFI_BIS_PROTOCOL.GetBootObjectAuthorizationCheckFlag()	991
24.5.7	EFI_BIS_PROTOCOL.GetBootObjectAuthorizationUpdateToken()	992
24.5.8	EFI_BIS_PROTOCOL.GetSignatureInfo()	993
24.5.9	EFI_BIS_PROTOCOL.UpdateBootObjectAuthorization()	996
24.5.10	EFI_BIS_PROTOCOL.VerifyBootObject()	1003
24.5.11	EFI_BIS_PROTOCOL.VerifyObjectWithCredential()	1009
24.6	DHCP options for iSCSI on IPV6	1014
24.7	HTTP Boot	1014
24.7.1	Boot from URL	1014
24.7.2	Concept configuration for a typical HTTP Bootscenario	1015
	24.7.2.1 Use in Corporate environment	1015
	24.7.2.2 Use case in Home environment	1016
24.7.3	Protocol Layout for UEFI HTTP Boot Clientconcept configuration for a typical HTTP Boot scenario	1017
	24.7.3.1 Device Path	1018
24.7.4	Concept of Message Exchange in a typical HTTPBoot scenario (IPv4 in Corporate Environment)	1019
	24.7.4.1 Message exchange between EFI Client and DHCPserver using DHCP Client Extensions	1020
24.7.5	Priority1	1021
24.7.6	Priority2	1021
24.7.7	Priority3	1022
24.7.8	Priority4	1022
	24.7.8.1 Message exchange between UEFI Client and DHCP server not using DHCP Client Extensions	1022
	24.7.8.2 Message in DNS Query/Reply	1023
	24.7.8.3 Message in HTTP Download	1023

24.7.9	Concept of Message Exchange in HTTP Boot scenario (IPv6)	1023
24.7.9.1	Message exchange between EFI Client and DHCPv6 server with DHCP Client extensions	1023
24.7.9.2	Message exchange between UEFI Client and DHCPv6 server not using DHCP Client Extensions	1024
24.7.9.3	Message exchange between UEFI Client and DNS6 server	1024
24.7.9.4	Message in HTTP Download	1024
24.7.10	Concept of Message Exchange in HTTP Boot scenario (with Proxy Host)	1024
24.7.11	EFI HTTP Boot Callback Protocol	1025
24.7.12	EFI_HTTP_BOOT_CALLBACK_PROTOCOL	1025
24.7.13	EFI_HTTP_BOOT_CALLBACK_PROTOCOL.Callback()	1026
25	Network Protocols - Managed Network	1028
25.1	EFI Managed Network Protocol	1028
25.1.1	EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL	1028
25.1.2	EFI_MANAGED_NETWORK_PROTOCOL	1029
25.1.3	EFI_MANAGED_NETWORK_PROTOCOL.GetModeData()	1030
25.1.4	EFI_MANAGED_NETWORK_PROTOCOL.Configure()	1032
25.1.5	EFI_MANAGED_NETWORK_PROTOCOL.McastIpToMac()	1033
25.1.6	EFI_MANAGED_NETWORK_PROTOCOL.Groups()	1034
25.1.7	EFI_MANAGED_NETWORK_PROTOCOL.Transmit()	1035
25.1.8	EFI_MANAGED_NETWORK_PROTOCOL.Receive()	1040
25.1.9	EFI_MANAGED_NETWORK_PROTOCOL.Cancel()	1041
25.1.10	EFI_MANAGED_NETWORK_PROTOCOL.Poll()	1042
26	Network Protocols — Bluetooth	1043
26.1	EFI Bluetooth Host Controller Protocol	1043
26.1.1	EFI_BLUETOOTH_HC_PROTOCOL	1043
26.1.2	BLUETOOTH_HC_PROTOCOL.SendCommand()	1044
26.1.3	BLUETOOTH_HC_PROTOCOL.ReceiveEvent()	1045
26.1.4	BLUETOOTH_HC_PROTOCOL.AsyncReceiveEvent()	1046
26.1.5	BLUETOOTH_HC_PROTOCOL.SendACLData()	1047
26.1.6	BLUETOOTH_HC_PROTOCOL.ReceiveACLData()	1048
26.1.7	BLUETOOTH_HC_PROTOCOL.AsyncReceiveACLData()	1049
26.1.8	BLUETOOTH_HC_PROTOCOL.SendSCODData()	1050
26.1.9	BLUETOOTH_HC_PROTOCOL.ReceiveSCODData()	1051
26.1.10	BLUETOOTH_HC_PROTOCOL.AsyncReceiveSCODData()	1052
26.2	EFI Bluetooth Bus Protocol	1053
26.2.1	EFI_BLUETOOTH_IO_SERVICE_BINDING_PROTOCOL	1053
26.2.2	EFI_BLUETOOTH_IO_PROTOCOL	1053
26.2.3	BLUETOOTH_IO_PROTOCOL.GetDeviceInfo	1055
26.2.4	BLUETOOTH_IO_PROTOCOL.GetSdpInfo	1056
26.2.5	BLUETOOTH_IO_PROTOCOL.L2CapRawSend	1057
26.2.6	BLUETOOTH_IO_PROTOCOL.L2CapRawReceive	1058
26.2.7	BLUETOOTH_IO_PROTOCOL.L2CapRawAsyncReceive	1059
26.2.8	BLUETOOTH_IO_PROTOCOL.L2CapSend	1060
26.2.9	BLUETOOTH_IO_PROTOCOL.L2CapReceive	1061
26.2.10	BLUETOOTH_IO_PROTOCOL.L2CapAsyncReceive	1062
26.2.11	BLUETOOTH_IO_PROTOCOL.L2CapConnect	1063
26.2.12	BLUETOOTH_IO_PROTOCOL.L2CapDisconnect	1064
26.2.13	BLUETOOTH_IO_PROTOCOL.L2CapRegisterService	1065
26.3	EFI Bluetooth Configuration Protocol	1066
26.3.1	EFI_BLUETOOTH_CONFIG_PROTOCOL	1066
26.3.2	BLUETOOTH_CONFIG_PROTOCOL.Init	1067

26.3.3	BLUETOOTH_CONFIG_PROTOCOL.Scan	1068
26.3.4	BLUETOOTH_CONFIG_PROTOCOL.Connect	1069
26.3.5	BLUETOOTH_CONFIG_PROTOCOL.Disconnect	1070
26.3.6	BLUETOOTH_CONFIG_PROTOCOL.GetData	1070
26.3.7	BLUETOOTH_CONFIG_PROTOCOL.SetData	1072
26.3.8	BLUETOOTH_CONFIG_PROTOCOL.GetRemoteData	1073
26.3.9	BLUETOOTH_CONFIG_PROTOCOL.RegisterPinCallback	1074
26.3.10	BLUETOOTH_CONFIG_PROTOCOL.RegisterGetLinkKeyCallback	1076
26.3.11	BLUETOOTH_CONFIG_PROTOCOL.RegisterSetLinkKeyCallback	1077
26.3.12	BLUETOOTH_CONFIG_PROTOCOL.RegisterLinkConnectCompleteCallback	1078
26.4	EFI Bluetooth Attribute Protocol	1080
26.4.1	EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL	1080
26.4.2	BLUETOOTH_ATTRIBUTE_PROTOCOL.SendRequest	1081
26.4.3	BLUETOOTH_ATTRIBUTE_PROTOCOL.RegisterForServerNotification	1082
26.4.4	BLUETOOTH_ATTRIBUTE_PROTOCOL.GetServiceInfo	1084
26.4.5	BLUETOOTH_ATTRIBUTE_PROTOCOL.GetDeviceInfo	1087
26.4.6	EFI_BLUETOOTH_ATTRIBUTE_SERVICE_BINDING_PROTOCOL	1088
26.5	EFI Bluetooth LE Configuration Protocol	1089
26.5.1	EFI_BLUETOOTH_LE_CONFIG_PROTOCOL	1089
26.5.2	BLUETOOTH_LE_CONFIG_PROTOCOL.Init	1090
26.5.3	BLUETOOTH_LE_CONFIG_PROTOCOL.Scan	1091
26.5.4	BLUETOOTH_LE_CONFIG_PROTOCOL.Connect	1093
26.5.5	BLUETOOTH_LE_CONFIG_PROTOCOL.Disconnect	1094
26.5.6	BLUETOOTH_LE_CONFIG_PROTOCOL.GetData	1095
26.5.7	BLUETOOTH_LE_CONFIG_PROTOCOL.SetData	1096
26.5.8	BLUETOOTH_LE_CONFIG_PROTOCOL.GetRemoteData	1097
26.5.9	BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterSmpAuthCallback	1098
26.5.10	BLUETOOTH_LE_CONFIG_PROTOCOL.SendSmpAuthData	1100
26.5.11	BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterSmpGetDataCallback	1100
26.5.12	BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterSmpSetDataCallback	1104
26.5.13	BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterLinkConnectCompleteCallback	1105
27	Network Protocols — VLAN, EAP, Wi-Fi and Supplicant	1107
27.1	VLAN Configuration Protocol	1107
27.1.1	EFI_VLAN_CONFIG_PROTOCOL	1107
27.1.2	EFI_VLAN_CONFIG_PROTOCOL.Set()	1108
27.1.3	EFI_VLAN_CONFIG_PROTOCOL.Find()	1109
27.1.4	EFI_VLAN_CONFIG_PROTOCOL.Remove()	1110
27.2	EAP Protocol	1111
27.2.1	EFI_EAP_PROTOCOL	1111
27.2.2	EFI_EAP.SetDesiredAuthMethod()	1112
27.2.3	EFI_EAP.RegisterAuthMethod()	1112
27.2.4	EAPManagement Protocol	1114
27.2.5	EFI_EAP_MANAGEMENT_PROTOCOL	1114
27.2.6	EFI_EAP_MANAGEMENT.GetSystemConfiguration()	1115
27.2.7	EFI_EAP_MANAGEMENT.SetSystemConfiguration()	1116
27.2.8	EFI_EAP_MANAGEMENT.InitializePort()	1117
27.2.9	EFI_EAP_MANAGEMENT.UserLogon()	1117
27.2.10	EFI_EAP_MANAGEMENT.UserLogoff()	1118
27.2.11	EFI_EAP_MANAGEMENT.GetSupplicantStatus()	1118
27.2.12	EFI_EAP_MANAGEMENT.SetSupplicantConfiguration()	1120
27.2.13	EFI_EAP_MANAGEMENT.GetSupplicantStatistics()	1121
27.2.14	EFI EAP Management2 Protocol	1122
27.2.14.1	EFI_EAP_MANAGEMENT2_PROTOCOL	1122

27.2.15	EFI_EAP_MANAGEMENT2_PROTOCOL.GetKey()	1123
27.2.16	EFI EAP Configuration Protocol	1124
27.2.16.1	EFI_EAP_CONFIGURATION_PROTOCOL	1124
27.2.17	EFI_EAP_CONFIGURATION_PROTOCOL.SetData()	1125
27.2.18	EFI_EAP_CONFIGURATION_PROTOCOL.GetData()	1126
27.3	EFI Wireless MAC Connection Protocol	1127
27.3.1	EFI_WIRELESS_MAC_CONNECTION_PROTOCOL	1127
27.3.2	EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Scan()	1128
27.3.3	EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Associate()	1139
27.3.4	EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Disassociate()	1143
27.3.5	EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Authenticate()	1145
27.3.6	EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Deauthenticate()	1148
27.4	EFI Wireless MAC Connection II Protocol	1150
27.4.1	EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL	1150
27.4.2	EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL.GetNetworks()	1151
27.4.3	EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL.ConnectNetwork()	1155
27.4.4	EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL.DisconnectNetwork()	1157
27.5	EFI Supplicant Protocol	1158
27.5.1	Supplicant Service Binding Protocol	1159
27.5.2	EFI_SUPPLICANT_SERVICE_BINDING_PROTOCOL	1159
27.5.3	Supplicant Protocol	1159
27.5.4	EFI_SUPPLICANT_PROTOCOL	1159
27.5.5	EFI_SUPPLICANT_PROTOCOL.BuildResponsePacket()	1160
27.5.6	EFI_SUPPLICANT_PROTOCOL.ProcessPacket()	1161
27.5.7	EFI_SUPPLICANT_PROTOCOL.SetData()	1162
27.5.8	EFI_SUPPLICANT_PROTOCOL.GetData()	1167
28	Network Protocols — TCP, IP, IPsec, FTP, TLS and Configurations	1169
28.1	EFI TCPv4 Protocol	1169
28.1.1	TCPv4 Service Binding Protocol	1169
28.1.2	EFI_TCP4_SERVICE_BINDING_PROTOCOL	1169
28.1.3	TCPv4 Protocol	1170
28.1.4	EFI_TCP4_PROTOCOL	1170
28.1.5	EFI_TCP4_PROTOCOL.GetModeData()	1171
28.1.6	EFI_TCP4_PROTOCOL.Configure()	1175
28.1.7	EFI_TCP4_PROTOCOL.Routes()	1176
28.1.8	EFI_TCP4_PROTOCOL.Connect()	1178
28.1.9	EFI_TCP4_PROTOCOL.Accept()	1180
28.1.10	EFI_TCP4_PROTOCOL.Transmit()	1182
28.1.10.1	EFI_TCP4_PROTOCOL.Receive()	1186
28.1.11	EFI_TCP4_PROTOCOL.Close()	1187
28.1.12	EFI_TCP4_PROTOCOL.Cancel()	1189
28.1.13	EFI_TCP4_PROTOCOL.Poll()	1190
28.2	EFI TCPv6 Protocol	1190
28.2.1	TCPv6 Service Binding Protocol	1190
28.2.2	EFI_TCP6_SERVICE_BINDING_PROTOCOL	1190
28.2.3	TCPv6 Protocol	1191
28.2.4	EFI_TCP6_PROTOCOL	1191
28.2.5	EFI_TCP6_PROTOCOL.GetModeData()	1192
28.2.6	EFI_TCP6_PROTOCOL.Configure()	1196
28.2.7	EFI_TCP6_PROTOCOL.Connect()	1197
28.2.8	EFI_TCP6_PROTOCOL.Accept()	1199
28.2.9	EFI_TCP6_PROTOCOL.Transmit()	1201
28.2.10	EFI_TCP6_PROTOCOL.Receive()	1205

28.2.11	EFI_TCP6_PROTOCOL.Close()	1206
28.2.12	EFI_TCP6_PROTOCOL.Cancel()	1207
28.2.13	EFI_TCP6_PROTOCOL.Poll()	1208
28.3	EFI IPv4 Protocol	1209
28.3.1	IP4 Service Binding Protocol	1209
28.3.2	EFI_IP4_SERVICE_BINDING_PROTOCOL	1209
28.3.3	IP4 Protocol	1210
28.3.4	EFI_IP4_PROTOCOL	1210
28.3.5	EFI_IP4_PROTOCOL.GetModeData()	1211
28.3.6	EFI_IP4_PROTOCOL.Configure()	1215
28.3.7	EFI_IP4_PROTOCOL.Groups()	1216
28.3.8	EFI_IP4_PROTOCOL.Routes()	1217
28.3.9	EFI_IP4_PROTOCOL.Transmit()	1219
28.3.10	EFI_IP4_PROTOCOL.Receive()	1224
28.3.11	EFI_IP4_PROTOCOL.Cancel()	1225
28.3.12	EFI_IP4_PROTOCOL.Poll()	1226
28.4	EFI IPv4 Configuration II Protocol	1226
28.4.1	EFI_IP4_CONFIG2_PROTOCOL	1226
28.4.2	EFI_IP4_CONFIG2_PROTOCOL.SetData()	1227
28.4.3	EFI_IP4_CONFIG2_PROTOCOL.GetData()	1231
28.4.4	EFI_IP4_CONFIG2_PROTOCOL.RegisterDataNotify ()	1232
28.4.5	EFI_IP4_CONFIG2_PROTOCOL.UnregisterDataNotify ()	1233
28.5	EFI IPv6 Protocol	1234
28.5.1	IPv6 Service Binding Protocol	1234
28.5.2	EFI_IP6_SERVICE_BINDING_PROTOCOL	1234
28.5.3	IPv6 Protocol	1235
28.5.4	EFI_IP6_PROTOCOL	1235
28.5.5	EFI_IP6_PROTOCOL.GetModeData()	1236
28.5.6	EFI_IP6_PROTOCOL.Configure()	1242
28.5.7	EFI_IP6_PROTOCOL.Groups()	1243
28.5.8	EFI_IP6_PROTOCOL.Routes()	1244
28.5.9	EFI_IP6_PROTOCOL.Neighbors()	1246
28.5.10	EFI_IP6_PROTOCOL.Transmit()	1247
28.5.11	EFI_IP6_PROTOCOL.Receive()	1252
28.5.12	EFI_IP6_PROTOCOL.Cancel()	1253
28.5.13	EFI_IP6_PROTOCOL.Poll()	1254
28.6	EFI IPv6 Configuration Protocol	1255
28.6.1	EFI_IP6_CONFIG_PROTOCOL	1255
28.6.2	EFI_IP6_CONFIG_PROTOCOL.SetData()	1256
28.6.3	EFI_IP6_CONFIG_PROTOCOL.GetData()	1260
28.6.4	EFI_IP6_CONFIG_PROTOCOL.RegisterDataNotify ()	1261
28.6.5	EFI_IP6_CONFIG_PROTOCOL.UnregisterDataNotify()	1262
28.7	IPsec	1263
28.7.1	IPsec Overview	1263
28.7.2	EFI IPsec Configuration Protocol	1263
28.7.3	EFI_IPSEC_CONFIG_PROTOCOL	1263
28.7.4	EFI_IPSEC_CONFIG_PROTOCOL.SetData()	1264
28.7.5	EFI_IPSEC_CONFIG_PROTOCOL.GetData()	1275
28.7.6	EFI_IPSEC_CONFIG_PROTOCOL.GetNextSelector()	1276
28.7.7	EFI_IPSEC_CONFIG_PROTOCOL.RegisterDataNotify ()	1277
28.7.8	EFI_IPSEC_CONFIG_PROTOCOL.UnregisterDataNotify ()	1278
28.7.9	EFI IPsec Protocol	1279
28.7.10	EFI_IPSEC_PROTOCOL	1279
28.7.11	EFI_IPSEC_PROTOCOL.Process()	1280

28.7.12	EFI IPsec2 Protocol	1281
28.7.13	EFI_IPSEC2_PROTOCOL	1281
28.7.14	EFI_IPSEC2_PROTOCOL.ProcessExt()	1282
28.8	Network Protocol - EFI FTP Protocol	1284
28.8.1	EFI_FTP4_SERVICE_BINDING_PROTOCOL Summary	1284
28.8.2	EFI_FTP4_PROTOCOL	1284
28.8.3	EFI_FTP4_PROTOCOL.GetModeData()	1285
28.8.4	EFI_FTP4_PROTOCOL.Connect()	1286
28.8.5	EFI_FTP4_PROTOCOL.Close()	1288
28.8.6	EFI_FTP4_PROTOCOL.Configure()	1289
28.8.7	EFI_FTP4_PROTOCOL.ReadFile()	1291
28.8.8	EFI_FTP4_PROTOCOL.WriteFile()	1293
28.8.9	EFI_FTP4_PROTOCOL.ReadDirectory()	1295
28.8.10	EFI_FTP4_PROTOCOL.Poll()	1296
28.9	EFI TLS Protocols	1296
28.9.1	EFI TLS Service Binding Protocol	1296
28.9.1.1	EFI_TLS_SERVICE_BINDING_PROTOCOL	1296
28.9.2	EFI TLS Protocol	1297
28.9.2.1	EFI_TLS_PROTOCOL	1297
28.9.3	EFI_TLS_PROTOCOL.SetSessionData ()	1298
28.9.4	EFI_TLS_PROTOCOL.GetSessionData ()	1303
28.9.5	EFI_TLS_PROTOCOL.BuildResponsePacket ()	1304
28.9.6	EFI_TLS_PROTOCOL.ProcessPacket ()	1305
28.9.7	EFI TLS Configuration Protocol	1307
28.9.7.1	EFI_TLS_CONFIGURATION_PROTOCOL	1307
28.9.8	EFI_TLS_CONFIGURATION_PROTOCOL.SetData()	1307
28.9.9	EFI_TLS_CONFIGURATION_PROTOCOL.GetData()	1309
29	Network Protocols — ARP, DHCP, DNS, HTTP and REST	1310
29.1	ARP Protocol	1310
29.1.1	EFI_ARP_SERVICE_BINDING_PROTOCOL	1310
29.1.2	EFI_ARP_PROTOCOL	1311
29.1.3	EFI_ARP_PROTOCOL.Configure()	1312
29.1.4	EFI_ARP_PROTOCOL.Add()	1313
29.1.5	EFI_ARP_PROTOCOL.Find()	1315
29.1.6	EFI_ARP_PROTOCOL.Delete()	1317
29.1.7	EFI_ARP_PROTOCOL.Flush()	1318
29.1.8	EFI_ARP_PROTOCOL.Request()	1318
29.1.9	EFI_ARP_PROTOCOL.Cancel()	1319
29.2	EFI DHCPv4 Protocol	1320
29.2.1	EFI_DHCP4_SERVICE_BINDING_PROTOCOL	1320
29.2.2	EFI_DHCP4_PROTOCOL	1321
29.2.3	EFI_DHCP4_PROTOCOL.GetModeData()	1322
29.2.4	EFI_DHCP4_PROTOCOL.Configure()	1325
29.2.5	EFI_DHCP4_PROTOCOL.Start()	1331
29.2.6	EFI_DHCP4_PROTOCOL.RenewRebind()	1332
29.2.7	EFI_DHCP4_PROTOCOL.Release()	1333
29.2.8	EFI_DHCP4_PROTOCOL.Stop()	1334
29.2.9	EFI_DHCP4_PROTOCOL.Build()	1334
29.2.10	EFI_DHCP4_PROTOCOL.TransmitReceive()	1336
29.2.11	EFI_DHCP4_PROTOCOL.Parse()	1338
29.3	EFI DHCP6 Protocol	1339
29.3.1	DHCP6 Service Binding Protocol	1339
29.3.2	EFI_DHCP6_SERVICE_BINDING_PROTOCOL	1339

29.3.3	DHCP6 Protocol	1340
29.3.4	EFI_DHCP6_PROTOCOL	1340
29.3.5	EFI_DHCP6_PROTOCOL.GetModeData ()	1341
29.3.6	EFI_DHCP6_PROTOCOL.Configure ()	1345
29.3.7	EFI_DHCP6_PROTOCOL.Start ()	1350
29.3.8	EFI_DHCP6_PROTOCOL.InfoRequest ()	1351
29.3.9	EFI_DHCP6_PROTOCOL.RenewRebind ()	1353
29.3.10	EFI_DHCP6_PROTOCOL.Decline ()	1354
29.3.11	EFI_DHCP6_PROTOCOL.Release ()	1355
29.3.12	EFI_DHCP6_PROTOCOL.Stop ()	1356
29.3.13	EFI_DHCP6_PROTOCOL.Parse ()	1357
29.4	EFI DNSv4 Protocol	1358
29.4.1	EFI_DNS4_SERVICE_BINDING_PROTOCOL	1358
29.4.2	EFI_DNS4_PROTOCOL	1359
29.4.3	EFI_DNS4_PROTOCOL.GetModeData()	1360
29.4.4	EFI_DNS4_PROTOCOL.Configure()	1362
29.4.5	EFI_DNS4_PROTOCOL.HostNameToIp()	1363
29.4.6	EFI_DNS4_PROTOCOL.IpToHostName()	1366
29.4.7	EFI_DNS4_PROTOCOL.GeneralLookUp()	1367
29.4.8	EFI_DNS4_PROTOCOL.UpdateDnsCache()	1368
29.4.9	EFI_DNS4_PROTOCOL.Poll()	1369
29.4.10	EFI_DNS4_PROTOCOL.Cancel()	1370
29.5	EFI DNSv6 Protocol	1371
29.5.1	DNS6 Service Binding Protocol	1371
29.5.2	EFI_DNS6_SERVICE_BINDING_PROTOCOL	1371
29.5.3	DNS6 Protocol	1371
29.5.4	EFI_DNS6_PROTOCOL	1371
29.5.5	EFI_DNS6_PROTOCOL.GetModeData()	1372
29.5.6	EFI_DNS6_PROTOCOL.Configure()	1375
29.5.7	EFI_DNS6_PROTOCOL.HostNameToIp()	1376
29.5.8	EFI_DNS6_PROTOCOL.IpToHostName()	1379
29.5.9	EFI_DNS6_PROTOCOL.GeneralLookUp()	1380
29.5.10	EFI_DNS6_PROTOCOL.UpdateDnsCache()	1381
29.5.11	EFI_DNS6_PROTOCOL.POLL()	1382
29.5.12	EFI_DNS6_PROTOCOL.Cancel()	1383
29.6	EFI HTTP Protocols	1384
29.6.1	HTTP Service Binding Protocol	1384
29.6.1.1	EFI_HTTP_SERVICE_BINDING_PROTOCOL	1384
29.6.2	EFI HTTP Protocol Specific Definitions	1384
29.6.3	EFI_HTTP_PROTOCOL	1384
29.6.4	EFI_HTTP_PROTOCOL.GetModeData()	1385
29.6.5	EFI_HTTP_PROTOCOL.Configure()	1388
29.6.6	EFI_HTTP_PROTOCOL.Request()	1389
29.6.7	EFI_HTTP_PROTOCOL.Cancel()	1393
29.6.8	EFI_HTTP_PROTOCOL.Response()	1394
29.6.9	EFI_HTTP_PROTOCOL.Poll()	1395
29.6.9.1	Usage Examples	1396
29.6.10	HTTP Utilities Protocol	1402
29.6.11	EFI_HTTP_UTILITIES_PROTOCOL	1402
29.6.12	EFI_HTTP_UTILITIES_PROTOCOL.Build()	1402
29.6.13	EFI_HTTP_UTILITIES_PROTOCOL.Parse()	1403
29.7	EFI REST Support Overview	1404
29.7.1	EFI REST Support Scenario 1 (PlatformManagement)	1405
29.7.2	EFI REST Support Scenario 2 (PlatformManagement)	1408

29.7.3	EFI REST Protocol	1409
29.7.3.1	EFI REST Protocol Definitions	1409
29.7.4	EFI_REST_PROTOCOL	1409
29.7.5	EFI_REST_PROTOCOL.SendReceive()	1410
29.7.6	EFI_REST_PROTOCOL.GetServiceTime()	1411
29.7.7	EFI REST EX Protocol	1411
29.7.7.1	REST EX Service Binding Protocol	1411
29.7.8	EFI_REST_EX_SERVICE_BINDING_PROTOCOL	1411
29.7.8.1	REST EX Protocol Specific Definitions	1412
29.7.9	EFI_REST_EX_PROTOCOL	1412
29.7.10	EFI_REST_EX_PROTOCOL.SendReceive()	1413
29.7.11	EFI_REST_EX_PROTOCOL.GetService()	1414
29.7.12	EFI_REST_EX_PROTOCOL.GetModeData()	1418
29.7.13	EFI_REST_EX_PROTOCOL.Configure()	1419
29.7.14	EFI_REST_EX_PROTOCOL.AsyncSendReceive()	1420
29.7.15	EFI_REST_EX_PROTOCOL.EventService()	1422
29.7.15.1	Usage Example (HTTP-aware REST EX Protocol DriverInstance)	1424
29.7.16	EFI_REST_EX_PROTOCOL.EventService()	1428
29.7.17	EFI REST JSON Resource to C Structure Converter	1431
29.7.17.1	Overview	1431
29.7.17.2	EFI REST JSON Structure Protocol	1431
29.7.18	EFI_REST_JSON_STRUCTURE.Register ()	1432
29.7.19	EFI_REST_JSON_STRUCTURE.ToStructure ()	1434
29.7.20	EFI_REST_JSON_STRUCTURE.ToJson ()	1436
29.7.21	EFI_REST_JSON_STRUCTURE.DestroyStructure ()	1437
29.7.21.1	EFI Redfish JSON Structure Converter	1438
30	Network Protocols — UDP and MTFTP	1439
30.1	EFI UDP Protocol	1439
30.1.1	UDP4 Service Binding Protocol	1439
30.1.1.1	EFI_UDP4_SERVICE_BINDING_PROTOCOL	1439
30.1.2	UDP4 Protocol	1440
30.1.2.1	EFI_UDP4_PROTOCOL	1440
30.1.2.2	EFI_UDP4_PROTOCOL.GetModeData()	1441
30.1.2.3	EFI_UDP4_PROTOCOL.Configure()	1443
30.1.2.4	EFI_UDP4_PROTOCOL.Groups()	1444
30.1.2.5	EFI_UDP4_PROTOCOL.Routes()	1445
30.1.2.6	EFI_UDP4_PROTOCOL.Transmit()	1447
30.1.2.7	EFI_UDP4_PROTOCOL.Receive()	1451
30.1.2.8	EFI_UDP4_PROTOCOL.Cancel()	1452
30.1.2.9	EFI_UDP4_PROTOCOL.Poll()	1453
30.2	EFI UDPv6 Protocol	1454
30.2.1	UDP6 Service Binding Protocol	1454
30.2.1.1	EFI_UDP6_SERVICE_BINDING_PROTOCOL	1454
30.2.2	EFI UDP6 Protocol	1454
30.2.2.1	EFI_UDP6_PROTOCOL	1454
30.2.2.2	EFI_UDP6_PROTOCOL.GetModeData()	1456
30.2.2.3	EFI_UDP6_PROTOCOL.Configure()	1458
30.2.2.4	EFI_UDP6_PROTOCOL.Groups()	1459
30.2.2.5	EFI_UDP6_PROTOCOL.Transmit()	1460
30.2.2.6	EFI_UDP6_PROTOCOL.Receive()	1464
30.2.2.7	EFI_UDP6_PROTOCOL.Cancel()	1465
30.2.2.8	EFI_UDP6_PROTOCOL.Poll()	1466
30.3	EFI MTFTIPv4 Protocol	1467

30.3.1	EFI_MTFTP4_SERVICE_BINDING_PROTOCOL	1467
30.3.2	EFI_MTFTP4_PROTOCOL	1467
30.3.3	EFI_MTFTP4_PROTOCOL.GetModeData()	1468
30.3.4	EFI_MTFTP4_PROTOCOL.Configure()	1470
30.3.5	EFI_MTFTP4_PROTOCOL.GetInfo()	1471
30.3.6	EFI_MTFTP4_PROTOCOL.ParseOptions()	1479
30.3.7	EFI_MTFTP4_PROTOCOL.ReadFile()	1481
30.3.8	EFI_MTFTP4_PROTOCOL.WriteFile()	1485
30.3.9	EFI_MTFTP4_PROTOCOL.ReadDirectory()	1487
30.3.10	EFI_MTFTP4_PROTOCOL.POLL()	1488
30.4	EFI MTFTPv6 Protocol	1489
30.4.1	MTFTP6 Service Binding Protocol	1489
30.4.1.1	EFI_MTFTP6_SERVICE_BINDING_PROTOCOL	1489
30.4.2	MTFTP6 Protocol	1490
30.4.2.1	EFI_MTFTP6_PROTOCOL	1490
30.4.2.2	EFI_MTFTP6_PROTOCOL.GetModeData()	1491
30.4.2.3	EFI_MTFTP6_PROTOCOL.Configure()	1492
30.4.2.4	EFI_MTFTP6_PROTOCOL.GetInfo()	1493
30.4.2.5	EFI_MTFTP6_PROTOCOL.ParseOptions()	1500
30.4.2.6	EFI_MTFTP6_PROTOCOL.ReadFile()	1501
30.4.2.7	EFI_MTFTP6_PROTOCOL.WriteFile()	1506
30.4.2.8	EFI_MTFTP6_PROTOCOL.ReadDirectory()	1507
30.4.2.9	EFI_MTFTP6_PROTOCOL.Poll()	1509
31	EFI Redfish Service Support	1510
31.1	EFI Redfish Discover Protocol	1510
31.1.1	Overview	1510
31.1.2	EFI Redfish Discover Driver	1511
31.1.3	EFI Redfish Discover Client	1511
31.1.4	EFI Redfish Discover Protocol	1513
31.1.4.1	EFI_REDFISH_DISCOVER_PROTOCOL.GetNetworkInterfaceList()	1514
31.1.4.2	EFI_REDFISH_DISCOVER_PROTOCOL.AcquireRedfishService()	1515
31.1.4.3	EFI_REDFISH_DISCOVER_PROTOCOL.AbortAcquireRedfishService()	1520
31.1.4.4	EFI_REDFISH_DISCOVER_PROTOCOL.ReleaseRedfishService()	1521
31.1.5	Implementation Examples	1522
31.1.5.1	Processes to Discover Redfish Services	1522
31.1.5.2	Network Interface Configuration	1522
31.2	EFI Redfish JSON Structure Converter	1523
31.2.1	The Guidance of Writing EFI Redfish JSONStructure Converter	1523
31.2.2	The Guidance of Using EFI Redfish JSON Structure Converter	1525
32	Secure Boot and Driver Signing	1526
32.1	Secure Boot	1526
32.1.1	EFI_AUTHENTICATION_INFO_PROTOCOL	1526
32.1.2	EFI_AUTHENTICATION_INFO_PROTOCOL.Get()	1527
32.1.3	EFI_AUTHENTICATION_INFO_PROTOCOL.Set()	1527
32.1.4	Authentication Nodes	1528
32.1.5	Generic Authentication Node Structures	1528
32.1.6	CHAP (using RADIUS) Authentication Node	1529
32.2	UEFI Driver Signing Overview	1531
32.2.1	Digital Signatures	1531
32.2.2	Embedded Signatures	1532
32.2.3	Creating Image Digests from Images	1533
32.2.4	Code Definitions	1533

32.2.4.1	WIN_CERTIFICATE	1533
32.2.4.2	WIN_CERTIFICATE_EFI_PKCS1_15	1535
32.2.4.3	WIN_CERTIFICATE_UEFI_GUID	1536
32.3	Firmware/OS Key Exchange: Creating Trust Relationships	1537
32.3.1	Enrolling The Platform Key	1538
32.3.2	Clearing The Platform Key	1540
32.3.3	Transitioning to Audit Mode	1540
32.3.4	Transitioning to Deployed Mode	1540
32.3.5	Enrolling Key Exchange Keys	1540
32.3.6	Platform Firmware Key Storage Requirements	1541
32.4	Firmware/OS Key Exchange: Passing Public Keys	1541
32.4.1	Signature Database	1541
32.4.1.1	EFI_SIGNATURE_DATA	1541
32.4.2	Image Execution Information Table	1546
32.5	UEFI Image Validation	1549
32.5.1	Overview	1549
32.5.2	Authorized User	1550
32.5.3	Signature Database Update	1550
32.5.3.1	Using The Image Execution Information Table	1551
32.5.3.2	Firmware Policy	1551
32.5.3.3	Authorization Process	1551
32.6	Device Authentication	1554
32.6.1	Overview	1554
32.6.2	Authorized User	1556
32.6.3	Device Signature Database Update	1556
32.7	Code Definitions	1556
32.7.1	UEFI Image Variable GUID & Variable Name	1556
32.7.2	UEFI Device Signature Variable GUID and Variable Name	1557
33	Human Interface Infrastructure Overview	1558
33.1	Goals	1558
33.2	Design Discussion	1559
33.2.1	Drivers And Applications	1560
33.2.1.1	Platform and Driver Configuration	1560
33.2.1.2	Pre-O/S applications	1560
33.2.1.3	Description of User Interface Components	1560
33.2.1.4	Forms	1560
33.2.1.5	Strings	1564
33.2.1.6	Images/Fonts	1564
33.2.1.7	Consumers of the user interface data	1564
33.2.1.8	Connected forms browser/processor	1564
33.2.1.9	Disconnected Forms Browser/Processor	1564
33.2.1.10	O/S-Present Forms Browser/Processor	1565
33.2.1.11	Where are the Results Stored	1565
33.2.2	Localization	1565
33.2.3	User Input	1566
33.2.4	Keyboard Layout	1568
33.2.4.1	Keyboard Mapping	1568
33.2.4.2	Modifier Keys	1569
33.2.4.3	Non-Spacing Keys	1569
33.2.5	Forms	1570
33.2.5.1	Form Sets	1571
33.2.5.2	Forms	1572
33.2.5.3	Statements	1574

33.2.5.4	Questions	1576
33.2.5.5	Options	1584
33.2.5.6	Storage	1585
33.2.5.7	Expressions	1587
33.2.5.8	Defaults	1589
33.2.5.9	Validation	1591
33.2.5.10	Forms Processing	1591
33.2.5.11	Forms Editing	1592
33.2.5.12	Forms Processing & Security Privileges	1593
33.2.6	Strings	1594
33.2.6.1	Configuration Language Paradigm	1595
33.2.6.2	Unicode Usage	1595
33.2.7	Fonts	1598
33.2.7.1	Font Attributes	1598
33.2.7.2	Limiting Glyphs	1598
33.2.7.3	Fixed Font Description	1599
33.2.7.4	Proportional Fonts Description	1602
33.2.8	Images	1603
33.2.8.1	Converting to a 32-bit Display	1603
33.2.8.2	Non-TrueColor Displays	1604
33.2.9	HII Database	1604
33.2.10	Forms Browser	1604
33.2.10.1	User Interaction	1605
33.2.11	Configuration Settings	1608
33.2.11.1	OS Runtime Utilization	1609
33.2.11.2	Working with a UEFI Configuration Language	1610
33.2.12	Form Callback Logic	1610
33.2.13	Driver Model Interaction	1611
33.2.14	Human Interface Component Interactions	1611
33.2.15	Standards Map Forms	1611
33.2.15.1	Create A Question's Value By Combing Multiple Configuration Settings	1617
33.2.15.2	Changing Multiple Configuration Settings From One Question's Value	1618
33.2.15.3	Value Shifting	1618
33.2.15.4	Prompts	1619
33.3	Code Definitions	1619
33.3.1	Package Lists and Package Headers	1619
33.3.1.1	EFI_HII_PACKAGE_HEADER	1619
33.3.1.2	EFI_HII_PACKAGE_LIST_HEADER	1620
33.3.2	Simplified Font Package	1621
33.3.2.1	EFI_HII_SIMPLE_FONT_PACKAGE_HDR	1621
33.3.2.2	EFI_NARROW_GLYPH	1622
33.3.2.3	EFI_WIDE_GLYPH	1623
33.3.3	Font Package	1623
33.3.3.1	Fixed Header	1623
33.3.3.2	Glyph Information	1624
33.3.4	Device Path Package	1634
33.3.5	GUID Package	1635
33.3.6	String Package	1635
33.3.6.1	Fixed Header	1635
33.3.6.2	String Information	1636
33.3.6.3	String Encoding	1648
33.3.7	Image Package	1648
33.3.7.1	Fixed Header	1649
33.3.7.2	Image Information	1649

33.3.7.3	Palette Information	1661
33.3.8	Forms Package	1664
33.3.8.1	Binary Encoding	1664
33.3.8.2	Standard Headers	1665
33.3.8.3	Opcode Reference	1667
33.3.9	Keyboard Package	1734
33.3.10	Animations Package	1735
33.3.10.1	Animated Images Package	1735
33.3.10.2	Animation Information	1735
34	HII Protocols	1746
34.1	Font Protocol	1746
34.1.1	EFI_HII_FONT_PROTOCOL	1746
34.1.2	EFI_HII_FONT_PROTOCOL.StringToImage()	1747
34.1.3	EFI_HII_FONT_PROTOCOL.StringIdToImage()	1749
34.1.4	EFI_HII_FONT_PROTOCOL.GetGlyph()	1751
34.1.5	EFI_HII_FONT_PROTOCOL.GetFontInfo()	1752
34.2	EFI HII Font Ex Protocol	1753
34.2.1	EFI_HII_FONT_EX_PROTOCOL	1753
34.2.2	EFI_HII_FONT_EX_PROTOCOL.StringToImageEx()	1754
34.2.3	EFI_HII_FONT_EX_PROTOCOL.StringIdToImageEx()	1754
34.2.4	EFI_HII_FONT_EX_PROTOCOL.GetGlyphEx()	1755
34.2.5	EFI_HII_FONT_EX_PROTOCOL.GetFontInfoEx()	1756
34.2.6	EFI_HII_FONT_EX_PROTOCOL.GetGlyphInfo()	1756
34.2.7	Code Definitions	1757
34.2.7.1	EFI_FONT_DISPLAY_INFO	1757
34.2.7.2	EFI_IMAGE_OUTPUT	1759
34.3	String Protocol	1759
34.3.1	EFI_HII_STRING_PROTOCOL	1759
34.3.2	EFI_HII_STRING_PROTOCOL.NewString()	1760
34.3.3	EFI_HII_STRING_PROTOCOL.GetString()	1761
34.3.4	EFI_HII_STRING_PROTOCOL.SetString()	1763
34.3.5	EFI_HII_STRING_PROTOCOL.GetLanguages()	1764
34.3.6	EFI_HII_STRING_PROTOCOL.GetSecondaryLanguages()	1764
34.4	Image Protocol	1766
34.4.1	EFI_HII_IMAGE_PROTOCOL	1766
34.4.2	EFI_HII_IMAGE_PROTOCOL.NewImage()	1766
34.4.3	EFI_HII_IMAGE_PROTOCOL.GetImage()	1767
34.4.4	EFI_HII_IMAGE_PROTOCOL.SetImage()	1768
34.4.5	EFI_HII_IMAGE_PROTOCOL.DrawImage()	1769
34.4.6	EFI_HII_IMAGE_PROTOCOL.DrawImageId()	1770
34.5	EFI HII Image Ex Protocol	1772
34.5.1	EFI_HII_IMAGE_EX_PROTOCOL	1772
34.5.2	EFI_HII_IMAGE_EX_PROTOCOL.NewImageEx()	1773
34.5.3	EFI_HII_IMAGE_EX_PROTOCOL.GetImageEx()	1773
34.5.4	EFI_HII_IMAGE_EX_PROTOCOL.SetImageEx()	1774
34.5.5	EFI_HII_IMAGE_EX_PROTOCOL.DrawImageEx()	1774
34.5.6	EFI_HII_IMAGE_EX_PROTOCOL.DrawImageIdEx()	1775
34.5.7	EFI_HII_IMAGE_EX_PROTOCOL.GetImageInfo()	1775
34.6	EFI HII Image Decoder Protocol	1776
34.6.1	EFI_HII_IMAGE_DECODER_PROTOCOL.DecodeImage()	1778
34.6.2	EFI_HII_IMAGE_DECODER_PROTOCOL.GetImageDecoderName()	1778
34.6.3	EFI_HII_IMAGE_DECODER_PROTOCOL.GetImageInfo()	1779
34.6.4	EFI_HII_IMAGE_DECODER_PROTOCOL.Decode()	1782

34.7	Font Glyph Generator Protocol	1783
34.7.1	EFI_HII_FONT_GLYPH_GENERATOR_PROTOCOL	1783
34.7.2	EFI_HII_FONT_GLYPH_GENERATOR_PROTOCOL.GenerateGlyph()	1783
34.7.3	EFI_HII_FONT_GLYPH_GENERATOR_PROTOCOL.GenerateGlyphImage()	1784
34.8	Database Protocol	1786
34.8.1	EFI_HII_DATABASE_PROTOCOL	1786
34.8.2	EFI_HII_DATABASE_PROTOCOL.NewPackageList()	1787
34.8.3	EFI_HII_DATABASE_PROTOCOL.RemovePackageList()	1788
34.8.4	EFI_HII_DATABASE_PROTOCOL.UpdatePackageList()	1788
34.8.5	EFI_HII_DATABASE_PROTOCOL.ListPackageLists()	1789
34.8.6	EFI_HII_DATABASE_PROTOCOL.ExportPackageLists()	1790
34.8.7	EFI_HII_DATABASE_PROTOCOL.RegisterPackageNotify()	1791
34.8.8	EFI_HII_DATABASE_PROTOCOL.UnregisterPackageNotify()	1792
34.8.9	EFI_HII_DATABASE_PROTOCOL.FindKeyboardLayouts()	1793
34.8.10	EFI_HII_DATABASE_PROTOCOL.GetKeyboardLayout()	1794
34.8.11	EFI_HII_DATABASE_PROTOCOL.SetKeyboardLayout()	1798
34.8.12	EFI_HII_DATABASE_PROTOCOL.GetPackageListHandle()	1799
34.8.13	Database Structures	1800
34.8.13.1	EFI_HII_DATABASE_NOTIFY	1800
34.8.14	EFI_HII_DATABASE_NOTIFY_TYPE	1801
35	HII Configuration Processing and Browser Protocol	1802
35.1	Introduction	1802
35.1.1	Common Configuration Data Format	1802
35.1.2	Data Flow	1802
35.2	Configuration Strings	1802
35.2.1	String Syntax	1803
35.2.1.1	Basic forms	1803
35.2.1.2	Types	1803
35.2.1.3	Routing elements	1803
35.2.1.4	Body elements	1804
35.2.1.5	Configuration strings	1804
35.2.1.6	Keyword strings	1805
35.2.2	String Types	1808
35.3	EFI Configuration Keyword Handler Protocol	1808
35.3.1	EFI_CONFIG_KEYWORD_HANDLER_PROTOCOL	1809
35.3.2	EFI_KEYWORD_HANDLER_PROTOCOL.SetData()	1809
35.3.3	EFI_KEYWORD_HANDLER_PROTOCOL.GetData()	1811
35.4	EFI HII Configuration Routing Protocol	1812
35.4.1	EFI_HII_CONFIG_ROUTING_PROTOCOL	1812
35.4.2	EFI_HII_CONFIG_ROUTING_PROTOCOL.ExtractConfig()	1813
35.4.3	EFI_HII_CONFIG_ROUTING_PROTOCOL.ExportConfig()	1815
35.4.4	EFI_HII_CONFIG_ROUTING_PROTOCOL.RouteConfig()	1815
35.4.5	EFI_HII_CONFIG_ROUTING_PROTOCOL.BlockToConfig()	1816
35.4.6	EFI_HII_CONFIG_ROUTING_PROTOCOL.ConfigToBlock()	1817
35.4.7	EFI_HII_CONFIG_ROUTING_PROTOCOL.GetAltCfg()	1819
35.5	EFI HII Configuration Access Protocol	1820
35.5.1	EFI_HII_CONFIG_ACCESS_PROTOCOL	1820
35.5.2	EFI_HII_CONFIG_ACCESS_PROTOCOL.ExtractConfig()	1821
35.5.3	EFI_HII_CONFIG_ACCESS_PROTOCOL.RouteConfig()	1823
35.5.4	EFI_HII_CONFIG_ACCESS_PROTOCOL.CallBack()	1824
35.6	Form Browser Protocol	1828
35.6.1	EFI_FORM_BROWSER2_PROTOCOL	1828
35.6.2	EFI_FORM_BROWSER2_PROTOCOL.SendForm()	1829

35.6.3	EFI_FORM_BROWSER2_PROTOCOL.BrowserCallback()	1831
35.7	HII Popup Protocol	1832
35.7.1	EFI_HII_POPUP_PROTOCOL	1832
35.7.2	EFI_HII_POPUP_PROTOCOL.CreatePopup()	1833
36	User Identification	1835
36.1	User Identification Overview	1835
36.1.1	User Identify	1836
36.1.2	User Profiles	1837
36.1.2.1	User Profile Database	1837
36.1.2.2	User Identification Policy	1838
36.1.3	Credential Providers	1838
36.1.4	Security Considerations	1839
36.1.5	Deferred Execution	1840
36.2	User Identification Process	1840
36.2.1	User Identification Process	1840
36.2.2	Changing The Current User Profile	1841
36.2.3	Ready To Boot	1841
36.3	Code Definitions	1842
36.3.1	User Manager Protocol	1842
36.3.1.1	EFI_USER_MANAGER_PROTOCOL	1842
36.3.1.2	EFI_USER_MANAGER_PROTOCOL.Create()	1843
36.3.1.3	EFI_USER_MANAGER_PROTOCOL.Delete()	1844
36.3.1.4	EFI_USER_MANAGER_PROTOCOL.GetNext()	1844
36.3.1.5	EFI_USER_MANAGER_PROTOCOL.Current()	1845
36.3.1.6	EFI_USER_MANAGER_PROTOCOL.Identify()	1846
36.3.1.7	EFI_USER_MANAGER_PROTOCOL.Find()	1846
36.3.1.8	EFI_USER_MANAGER_PROTOCOL.Notify()	1848
36.3.1.9	EFI_USER_MANAGER_PROTOCOL.GetInfo()	1848
36.3.1.10	EFI_USER_MANAGER_PROTOCOL.SetInfo()	1850
36.3.1.11	EFI_USER_MANAGER_PROTOCOL.DeleteInfo()	1851
36.3.1.12	EFI_USER_MANAGER_PROTOCOL.GetNextInfo()	1852
36.3.2	Credential Provider Protocols	1853
36.3.2.1	EFI_USER_CREDENTIAL2_PROTOCOL	1853
36.3.2.2	EFI_USER_CREDENTIAL2_PROTOCOL.Enroll()	1855
36.3.2.3	EFI_USER_CREDENTIAL2_PROTOCOL.Form()	1856
36.3.2.4	EFI_USER_CREDENTIAL2_PROTOCOL.Tile()	1857
36.3.2.5	EFI_USER_CREDENTIAL2_PROTOCOL.Title()	1858
36.3.2.6	EFI_USER_CREDENTIAL2_PROTOCOL.User()	1858
36.3.2.7	EFI_USER_CREDENTIAL2_PROTOCOL.Select()	1859
36.3.2.8	EFI_USER_CREDENTIAL2_PROTOCOL.Deselect()	1860
36.3.2.9	EFI_USER_CREDENTIAL2_PROTOCOL.Default()	1861
36.3.2.10	EFI_USER_CREDENTIAL2_PROTOCOL.GetInfo()	1861
36.3.2.11	EFI_USER_CREDENTIAL2_PROTOCOL.GetNextInfo()	1862
36.3.2.12	EFI_USER_CREDENTIAL2_PROTOCOL.Delete()	1863
36.3.3	Deferred Image Load Protocol	1863
36.3.3.1	EFI_DEFERRED_IMAGE_LOAD_PROTOCOL	1863
36.3.3.2	EFI_DEFERRED_IMAGE_LOAD_PROTOCOL.GetImageInfo()	1864
36.4	User Information	1865
36.4.1	EFI_USER_INFO_ACCESS_POLICY_RECORD	1866
36.4.1.1	EFI_USER_INFO_ACCESS_FORBID_LOAD	1866
36.4.1.2	EFI_USER_INFO_ACCESS_PERMIT_LOAD	1867
36.4.1.3	EFI_USER_INFO_ACCESS_ENROLL_SELF	1867
36.4.1.4	EFI_USER_INFO_ACCESS_ENROLL_OTHERS	1867

36.4.1.5	EFI_USER_INFO_ACCESS_MANAGE	1868
36.4.1.6	EFI_USER_INFO_ACCESS_SETUP	1868
36.4.1.7	EFI_USER_INFO_ACCESS_FORBID_CONNECT	1869
36.4.1.8	EFI_USER_INFO_ACCESS_PERMIT_CONNECT	1869
36.4.1.9	EFI_USER_INFO_ACCESS_BOOT_ORDER	1869
36.4.2	EFI_USER_INFO_CBEFF_RECORD	1870
36.4.3	EFI_USER_INFO_CREATE_DATE_RECORD	1870
36.4.4	EFI_USER_INFO_CREDENTIAL_PROVIDER_RECORD	1870
36.4.5	EFI_USER_INFO_CREDENTIAL_PROVIDER_NAME_RECORD	1871
36.4.6	EFI_USER_INFO_CREDENTIAL_TYPE_RECORD	1871
36.4.7	EFI_USER_INFO_CREDENTIAL_TYPE_NAME_RECORD	1871
36.4.8	EFI_USER_INFO_GUID_RECORD	1872
36.4.9	EFI_USER_INFO_FAR_RECORD	1872
36.4.10	EFI_USER_INFO_IDENTIFIER_RECORD	1872
36.4.11	EFI_USER_INFO_IDENTITY_POLICY_RECORD	1873
36.4.12	EFI_USER_INFO_NAME_RECORD	1874
36.4.13	EFI_USER_INFO_PKCS11_RECORD	1874
36.4.14	EFI_USER_INFO_RETRY_RECORD	1875
36.4.15	EFI_USER_INFO_USAGE_DATE_RECORD	1875
36.4.16	EFI_USER_INFO_USAGE_COUNT_RECORD	1875
36.5	User Information Table	1876

37	Secure Technologies	1877
37.1	Hash Overview	1877
37.1.1	Hash References	1877
37.1.1.1	EFI_HASH_SERVICE_BINDING_PROTOCOL	1877
37.1.1.2	EFI_HASH_PROTOCOL	1878
37.1.1.3	EFI_HASH_PROTOCOL.GetHashSize()	1879
37.1.1.4	EFI_HASH_PROTOCOL.Hash()	1879
37.1.2	Other Code Definitions	1881
37.1.2.1	EFI_SHA1_HASH, EFI_SHA224_HASH, EFI_SHA256_HASH, EFI_SHA384_HASH, EFI_SHA512HASH, EFI_MD5_HASH	1881
37.1.2.2	EFI Hash Algorithms	1881
37.2	Hash2 Protocols	1882
37.2.1	EFI Hash2 Service Binding Protocol	1882
37.2.1.1	EFI_HASH2_SERVICE_BINDING_PROTOCOL	1882
37.2.2	EFI Hash2 Protocol	1883
37.2.2.1	EFI_HASH2_PROTOCOL	1883
37.2.2.2	EFI_HASH2_PROTOCOL.GetHashSize()	1884
37.2.2.3	EFI_HASH2_PROTOCOL.Hash()	1885
37.2.2.4	EFI_HASH2_PROTOCOL.HashInit()	1886
37.2.2.5	EFI_HASH2_PROTOCOL.HashUpdate()	1887
37.2.2.6	EFI_HASH2_PROTOCOL.HashFinal()	1888
37.2.3	Other Code Definitions	1889
37.2.3.1	EFI_HASH2_OUTPUT	1889
37.3	Key Management Service	1890
37.3.1	EFI_KEY_MANAGEMENT_SERVICE_PROTOCOL	1890
37.3.2	EFI_KMS_PROTOCOL.GetServiceStatus()	1900
37.3.2.1	EFI_KMS_PROTOCOL.RegisterClient()	1901
37.3.2.2	EFI_KMS_PROTOCOL.CreateKey()	1902
37.3.2.3	EFI_KMS_PROTOCOL.GetKey()	1904
37.3.2.4	EFI_KMS_PROTOCOL.AddKey()	1906
37.3.2.5	EFI_KMS_PROTOCOL.DeleteKey()	1908
37.3.2.6	EFI_KMS_PROTOCOL.GetKeyAttributes()	1909

37.3.2.7	EFI_KMS_PROTOCOL.AddKeyAttributes()	1911
37.3.2.8	EFI_KMS_PROTOCOL.DeleteKeyAttributes()	1913
37.3.2.9	EFI_KMS_PROTOCOL.GetKeyByAttributes()	1915
37.4	PKCS7 Verify Protocol	1917
37.4.1	EFI_PKCS7_VERIFY_PROTOCOL	1917
37.4.2	EFI_PKCS7_VERIFY_PROTOCOL.VerifyBuffer()	1919
37.4.2.1	EFI_PKCS7_VERIFY_PROTOCOL.VerifySignature()	1921
37.5	Random Number Generator Protocol	1924
37.5.1	EFI_RNG_PROTOCOL	1924
37.5.2	EFI_RNG_PROTOCOL.GetInfo	1925
37.5.3	EFI_RNG_PROTOCOL.GetRNG	1926
37.5.4	EFI RNG Algorithm Definitions	1927
37.5.5	RNG References	1927
37.6	Smart Card Reader and Smart Card Edge Protocol	1928
37.6.1	Smart Card Reader Protocol	1928
37.6.1.1	EFI_SMART_CARD_READER_PROTOCOL Summary	1928
37.6.2	EFI_SMART_CARD_READER_PROTOCOL.SCardConnect()	1929
37.6.3	EFI_SMART_CARD_READER_PROTOCOL.SCardDisconnect()	1931
37.6.4	EFI_SMART_CARD_READER_PROTOCOL.SCardStatus()	1931
37.6.5	EFI_SMART_CARD_READER_PROTOCOL.SCardTransmit()	1933
37.6.6	EFI_SMART_CARD_READER_PROTOCOL.SCardControl()	1934
37.6.7	EFI_SMART_CARD_READER_PROTOCOL.SCardGetAttrib()	1935
37.6.8	Smart Card Edge Protocol	1936
37.6.8.1	EFI_SMART_CARD_EDGE_PROTOCOL	1936
37.6.8.2	EFI_SMART_CARD_EDGE_PROTOCOL.GetContext()	1938
37.6.8.3	EFI_SMART_CARD_EDGE_PROTOCOL.Connect()	1940
37.6.8.4	EFI_SMART_CARD_EDGE_PROTOCOL.Disconnect()	1941
37.6.8.5	EFI_SMART_CARD_EDGE_PROTOCOL.GetCsn	1941
37.6.8.6	EFI_SMART_CARD_EDGE_PROTOCOL.GetReaderName	1942
37.6.8.7	EFI_SMART_CARD_EDGE_PROTOCOL.VerifyPin()	1943
37.6.8.8	EFI_SMART_CARD_EDGE_PROTOCOL.GetPinRemaining()	1944
37.6.8.9	EFI_SMART_CARD_EDGE_PROTOCOL.GetData()	1945
37.6.8.10	EFI_SMART_CARD_EDGE_PROTOCOL.GetCredentials()	1946
37.6.8.11	EFI_SMART_CARD_EDGE_PROTOCOL.SignData()	1948
37.6.8.12	EFI_SMART_CARD_EDGE_PROTOCOL.DecryptData()	1950
37.6.8.13	EFI_SMART_CARD_EDGE_PROTOCOL.BuildDHAgreement()	1952
37.7	Memory Protection	1953
37.7.1	EFI_MEMORY_ATTRIBUTE_PROTOCOL	1953
37.7.1.1	EFI_MEMORY_ATTRIBUTE_PROTOCOL.GetMemoryAttributes	1954
37.7.1.2	EFI_MEMORY_ATTRIBUTE_PROTOCOL.SetMemoryAttributes	1954
37.7.1.3	EFI_MEMORY_ATTRIBUTE_PROTOCOL.ClearMemoryAttributes	1955
38	Confidential Computing	1957
38.1	Virtual Platform CC Event Log	1957
38.2	EFI_CC_MEASUREMENT_PROTOCOL	1957
38.2.1	EFI_CC_MEASUREMENT_PROTOCOL	1957
38.2.2	EFI_CC_MEASUREMENT_PROTOCOL.GetCapability	1958
38.2.3	EFI_CC_MEASUREMENT_PROTOCOL.GetEventLog	1960
38.2.4	EFI_CC_MEASUREMENT_PROTOCOL.HashLogExtendEvent	1961
38.2.5	EFI_CC_MEASUREMENT_PROTOCOL.MapPcrToMrIndex	1963
38.3	EFI CC Final Events Table	1964
38.4	Vendor Specific Information	1965
38.4.1	Intel Trust Domain Extension	1965
38.4.2	RISC-V AP-TEE	1965

39	Miscellaneous Protocols	1966
39.1	EFI Timestamp Protocol	1966
39.1.1	EFI_TIMESTAMP_PROTOCOL	1966
39.1.2	EFI_TIMESTAMP_PROTOCOL.GetTimestamp()	1966
39.1.3	EFI_TIMESTAMP_PROTOCOL.GetProperties()	1967
39.2	Reset Notification Protocol	1968
39.2.1	EFI_RESET_NOTIFICATION_PROTOCOL	1968
39.2.2	EFI_RESET_NOTIFICATION_PROTOCOL.RegisterResetNotify()	1968
39.2.3	EFI_RESET_NOTIFICATION_PROTOCOL.UnregisterResetNotify()	1969
A	GUID and Time Formats	1970
B	Console	1972
B.1	EFI_SIMPLE_TEXT_INPUT_PROTOCOL and EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL	1972
B.2	EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL for PC ANSI or ANSI X3.64 terminals	1974
C	Device Path Examples	1978
C.1	Example Computer System	1978
C.2	Legacy Floppy	1979
C.3	IDE Disk	1980
C.4	Secondary Root PCI Bus with PCI to PCI Bridge	1981
C.5	ACPI Terms	1982
C.6	EFI Device Path as a Name Space	1983
D	Status Codes	1984
E	Universal Network Driver Interfaces	1987
E.1	Introduction	1987
E.1.1	Definitions	1987
E.1.2	Referenced Specifications	1989
E.1.3	OS Network Stacks	1992
E.2	Overview	1993
E.2.1	32/64-bit UNDI Interface	1994
E.2.1.1	Issuing UNDI Commands	1998
E.2.2	UNDI Command Format	1998
E.3	UNDI C Definitions	2000
E.3.1	Portability Macros	2002
E.3.1.1	PXE_INTEL_ORDER or PXE_NETWORK_ORDER	2002
E.3.1.2	PXE_UINT64_SUPPORT or PXE_NO_UINT64_SUPPORT	2002
E.3.1.3	PXE_BUSTYPE	2002
E.3.1.4	PXE_SWAP_UINT16	2003
E.3.1.5	PXE_SWAP_UINT32	2003
E.3.1.6	PXE_SWAP_UINT64	2003
E.3.2	Miscellaneous Macros	2004
E.3.2.1	Miscellaneous	2004
E.3.3	Portability Types	2004
E.3.3.1	PXE_CONST	2005
E.3.3.2	PXE_VOLATILE	2005
E.3.3.3	PXE_VOID	2005
E.3.3.4	PXE_UINT8	2005
E.3.3.5	PXE_UINT16	2005
E.3.3.6	PXE_UINT32	2005
E.3.3.7	PXE_UINT64	2006
E.3.3.8	PXE_UINTN	2006
E.3.4	Simple Types	2006

E.3.4.1	PXE_BOOL	2006
E.3.4.2	PXE_OPCODE	2006
E.3.4.3	PXE_OPFLAGS	2008
E.3.4.4	PXE_STATFLAGS	2012
E.3.4.5	PXE_STATCODE	2015
E.3.4.6	PXE_IFNUM	2016
E.3.4.7	PXE_CONTROL	2016
E.3.4.8	PXE_FRAME_TYPE	2017
E.3.4.9	PXE_IPV4	2017
E.3.4.10	PXE_IPV6	2017
E.3.4.11	PXE_MAC_ADDR	2017
E.3.4.12	PXE_IFTYPE	2018
E.3.4.13	PXE_MEDIA_PROTOCOL	2018
E.3.5	Compound Types	2019
E.3.5.1	PXE_HW_UNDI	2019
E.3.5.2	PXE_SW_UNDI	2020
E.3.5.3	PXE_UNDI	2021
E.3.5.4	PXE_CDB	2022
E.3.5.5	PXE_IP_ADDR	2022
E.3.5.6	PXE_DEVICE	2023
E.4	UNDI Commands	2023
E.4.1	Command Linking and Queuing	2025
E.4.2	Get State	2027
E.4.2.1	Issuing the Command	2027
E.4.2.2	Waiting for the Command to Execute	2027
E.4.2.3	Checking Command Execution Results	2028
E.4.3	Start	2028
E.4.3.1	Issuing the Command	2028
E.4.3.2	Preparing the CPB	2029
E.4.3.3	Waiting for the Command to Execute	2034
E.4.3.4	Checking Command Execution Results	2034
E.4.4	Stop	2034
E.4.4.1	Issuing the Command	2034
E.4.4.2	Waiting for the Command to Execute	2035
E.4.4.3	Checking Command Execution Results	2035
E.4.5	Get Init Info	2035
E.4.5.1	Issuing the Command	2035
E.4.5.2	Waiting for the Command to Execute	2036
E.4.5.3	Checking Command Execution Results	2036
E.4.5.4	StatFlags	2036
E.4.5.5	DB	2036
E.4.6	Get Config Info	2038
E.4.6.1	Issuing the Command	2038
E.4.6.2	Waiting for the Command to Execute	2039
E.4.6.3	Checking Command Execution Results	2039
E.4.6.4	DB	2039
E.4.7	Initialize	2040
E.4.7.1	Issuing the Command	2041
E.4.7.2	OpFlags	2041
E.4.7.3	Preparing the CPB	2041
E.4.7.4	Waiting for the Command to Execute	2042
E.4.7.5	Checking Command Execution Results	2043
E.4.7.6	StatFlags	2043
E.4.7.7	Before Using the DB	2043

E.4.8	Reset	2044
E.4.8.1	Issuing the Command	2044
E.4.8.2	OpFlags	2044
E.4.8.3	Waiting for the Command to Execute	2044
E.4.8.4	Checking Command Execution Results	2045
E.4.8.5	StatFlags	2045
E.4.9	Shutdown	2045
E.4.9.1	Issuing the Command	2045
E.4.9.2	Waiting for the Command to Execute	2046
E.4.9.3	Checking Command Execution Results	2046
E.4.10	Interrupt Enables	2046
E.4.10.1	Issuing the Command	2046
E.4.10.2	OpFlags	2047
E.4.10.3	Waiting for the Command to Execute	2047
E.4.10.4	Checking Command Execution Results	2047
E.4.10.5	StatFlags	2048
E.4.11	Receive Filters	2048
E.4.11.1	Issuing the Command	2048
E.4.11.2	OpFlags	2048
E.4.11.3	Preparing the CPB	2049
E.4.11.4	Waiting for the Command to Execute	2049
E.4.11.5	Checking Command Execution Results	2049
E.4.11.6	StatFlags	2050
E.4.11.7	DB	2050
E.4.12	Station Address	2050
E.4.12.1	Issuing the Command	2050
E.4.12.2	OpFlags	2051
E.4.12.3	Preparing the CPB	2051
E.4.12.4	Waiting for the Command to Execute	2051
E.4.12.5	Checking Command Execution Results	2051
E.4.12.6	Before Using the DB	2052
E.4.13	Statistics	2052
E.4.13.1	Issuing the Command	2052
E.4.13.2	OpFlags	2053
E.4.13.3	Waiting for the Command to Execute	2053
E.4.13.4	Checking Command Execution Results	2053
E.4.13.5	DB	2053
E.4.14	MCast IP To MAC	2055
E.4.14.1	Issuing the Command	2055
E.4.14.2	OpFlags	2056
E.4.14.3	Preparing the CPB	2056
E.4.14.4	Waiting for the Command to Execute	2056
E.4.14.5	Checking Command Execution Results	2056
E.4.14.6	Before Using the DB	2057
E.4.15	NvData	2057
E.4.15.1	Issuing the Command	2057
E.4.15.2	Preparing the CPB	2057
E.4.15.3	Waiting for the Command to Execute	2058
E.4.15.4	Checking Command Execution Results	2059
E.4.16	Get Status	2059
E.4.16.1	Issuing the Command	2060
E.4.16.2	Waiting for the Command to Execute	2060
E.4.16.3	Checking Command Execution Results	2060
E.4.16.4	StatFlags	2061

E.4.16.5	Using the DB	2061
E.4.17	Fill Header	2062
E.4.17.1	Issuing the Command	2062
E.4.17.2	OpFlags	2062
E.4.17.3	Preparing the CPB	2062
E.4.17.4	Nonfragmented Frame	2062
E.4.17.5	Fragmented Frame	2063
E.4.17.6	Waiting for the Command to Execute	2064
E.4.17.7	Checking Command Execution Results	2064
E.4.18	Transmit	2065
E.4.18.1	Issuing the Command	2065
E.4.18.2	OpFlags	2065
E.4.18.3	Preparing the CPB	2066
E.4.18.4	Nonfragmented Frame	2066
E.4.18.5	Fragmented Frame	2066
E.4.18.6	Waiting for the Command to Execute	2067
E.4.18.7	Checking Command Execution Results	2067
E.4.19	Receive	2067
E.4.19.1	Issuing the Command	2068
E.4.19.2	Preparing the CPB	2068
E.4.19.3	Waiting for the Command to Execute	2069
E.4.19.4	Checking Command Execution Results	2069
E.4.19.5	Using the DB	2069
E.4.20	PXE 2.1 specification wire protocol clarifications	2070
E.4.20.1	Issue #1-time-outs	2070
E.4.20.2	Issue #2 - siaddr/option 54 precedence	2070
E.4.20.3	Issue #3 - PXE Vendor Options Existence	2070
F	Using the Simple Pointer Protocol	2073
G	Using the EFI Extended SCSI Pass Thru Protocol	2074
H	Compression Source Code	2077
I	Decompression Source Code	2108
J	EFI Byte Code Virtual Machine Opcode List	2126
K	Alphabetic Function Lists	2128
L	EFI 1.10 Protocol Changes	2129
L.1	Protocol and GUID Name Changes from EFI 1.10	2129
M	Formats — Language Codes and Language Code Arrays	2131
M.1	Specifying individual language codes	2131
M.1.1	Specifying language code arrays:	2131
N	Common Platform Error Record (CPER)	2132
N.1	Introduction	2132
N.2	Format	2132
N.2.1	Record Header	2133
N.2.1.1	Notification Type	2136
N.2.1.2	Error Status	2138
N.2.2	Section Descriptor	2139
N.2.3	Non-standard Section Body	2142

N.2.4	Processor Error Sections	2142
N.2.4.1	Generic Processor Error Section	2142
N.2.4.2	IA32/X64 Processor Error Section	2145
N.2.4.3	IA64 Processor Error Section	2155
N.2.4.4	ARM Processor Error Section	2155
N.2.5	Memory Error Section	2168
N.2.6	Memory Error Section 2	2171
N.2.7	PCI Express Error Section	2174
N.2.8	PCI/PCI-X Bus Error Section	2176
N.2.9	PCI/PCI-X Component Error Section	2177
N.2.10	Firmware Error Record Reference	2178
N.2.11	DMAr Error Sections	2179
N.2.11.1	DMAr Generic Error Section	2179
N.2.11.2	Intel® VT for Directed I/O specific DMar Error Section	2180
N.2.11.3	IOMMU Specific DMar Error Section	2182
N.2.12	CCIX PER Log Error Section	2182
N.2.13	Compute Express Link (CXL) Protocol Error Section	2183
N.2.14	CXL Component Events Section	2186
N.2.15	FRU Memory Poison Section	2187
O	UEFI ACPI Data Table	2190
O.1	Invocation method	2190
P	Hardware Error Record Persistence Usage	2192
P.1	Determining space	2192
P.2	Saving Hardware error records	2192
P.3	Clearing error record variables	2193
Q	References	2194
Q.1	Related Information	2194
Q.2	Prerequisite Specifications	2199
Q.2.1	ACPI Specification	2199
Q.2.2	Additional Considerations for Itanium-Based Platforms	2200
R	Glossary	2201

Acknowledgments

The material contained herein is not a license, either expressly or impliedly, to any intellectual property owned or controlled by any of the authors or developers of this material or to any contribution thereto. The material contained herein is provided on an “AS IS” basis and, to the maximum extent permitted by applicable law, this information is provided AS IS AND WITH ALL FAULTS, and the authors and developers of this material hereby disclaim all other warranties and conditions, either express, implied or statutory, including, but not limited to, any (if any) implied warranties, duties or conditions of merchantability, of fitness for a particular purpose, of accuracy or completeness of responses, of results, of workmanlike effort, of lack of viruses and of lack of negligence, all with regard to this material and any contribution thereto. Designers must not rely on the absence or characteristics of any features or instructions marked “reserved” or “undefined.” The Unified EFI Forum, Inc. reserves any features or instructions so marked for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them. ALSO, THERE IS NO WARRANTY OR CONDITION OF TITLE, QUIET ENJOYMENT, QUIET POSSESSION, CORRESPONDENCE TO DESCRIPTION OR NON-INFRINGEMENT WITH REGARD TO THE SPECIFICATION AND ANY CONTRIBUTION THERETO.

IN NO EVENT WILL ANY AUTHOR OR DEVELOPER OF THIS MATERIAL OR ANY CONTRIBUTION THERETO BE LIABLE TO ANY OTHER PARTY FOR THE COST OF PROCURING SUBSTITUTE GOODS OR SERVICES, LOST PROFITS, LOSS OF USE, LOSS OF DATA, OR ANY INCIDENTAL, CONSEQUENTIAL, DIRECT, INDIRECT, OR SPECIAL DAMAGES WHETHER UNDER CONTRACT, TORT, WARRANTY, OR OTHERWISE, ARISING IN ANY WAY OUT OF THIS OR ANY OTHER AGREEMENT RELATING TO THIS DOCUMENT, WHETHER OR NOT SUCH PARTY HAD ADVANCE NOTICE OF THE POSSIBILITY OF SUCH DAMAGES.

Copyright © 2024, Unified Extensible Firmware Interface (UEFI) Forum, Inc. All Rights Reserved. The UEFI Forum is the owner of all rights and title in and to this work, including all copyright rights that may exist, and all rights to use and reproduce this work. Further to such rights, permission is hereby granted to any person implementing this specification to maintain an electronic version of this work accessible by its internal personnel, and to print a copy of this specification in hard copy form, in whole or in part, in each case solely for use by that person in connection with the implementation of this Specification, provided no modification is made to the Specification.

List of Tables

Chapter 1

- Table 1.2 *SI Prefixes*
- Table 1.3 *Binary Prefixes*

Chapter 2

- Table 2.1 *UEFI Image Memory Types*
- Table 2.2 *UEFI Runtime Services*
- Table 2.3 *Common UEFI Data Types*
- Table 2.4 *Modifiers for Common UEFI Data Types*
- Table 2.5 *Map EFI Cacheability Attributes to AArch64 Memory Types*
- Table 2.6 *Map UEFI Permission Attributes to ARM Paging Attributes*
- Table 2.7 *UEFI Protocols*
- Table 2.8 *Required UEFI Implementation Elements*

Chapter 3

- Table 3.3 *Global Variables*
- Table 3.4 *UEFI Image Types*

Chapter 4

- Table 4.1 *Usage of Memory Attribute Definitions*

Chapter 5

- Table 5.1 *Legacy MBR*
- Table 5.2 *Legacy MBR Partition Record*
- Table 5.3 *Protective MBR*
- Table 5.4 *Protective MBR Partition Record protecting the entire disk**
- Table 5.6 *GPT Partition Entry*
- Table 5.7 *Defined GPT Partition Entry — Partition Type GUIDs*
- Table 5.8 *Defined GPT Partition Entry - Attributes*

Chapter 6

Chapter 7

- Table 7.1 *Event, Timer, and Task Priority Functions*
- Table 7.2 *TPL Usage*
- Table 7.3 *TPL Restrictions*
- Table 7.8 *Memory Allocation Functions*
- Table 7.9 *Memory Type Usage before ExitBootServices()*
- Table 7.10 *Memory Type Usage after ExitBootServices()*
- Table 7.14 *Protocol Interface Functions*
- Table 7.30 *Image Type Differences Summary*

- Table 7.31 *Image Functions*
- Table 7.37 *Miscellaneous Boot Services Functions*

Chapter 8

- Table 8.1 *Rules for Reentry Into Runtime Services*
- Table 8.1 *Rules for Reentry Into Runtime Services*
- Table 8.2 *Functions that may be called after Machine Check, INIT and NMI*
- Table 8.3 *Variable Services Functions*
- Table 8.8 *Hardware Error Record Persistence Variables*
- Table 8.9 *Time Services Functions*
- Table 8.14 *Virtual Memory Services*
- Table 8.17 *Miscellaneous Runtime Services*
- Table 8.19 *Flag Firmware Behavior*
- Table 8.22 *Variables Using EFI_CAPSULE_REPORT_GUID*

Chapter 9

Chapter 10

- Table 10.1 *Generic Device Path Node Structure*
- Table 10.2 *Device Path End Structure*
- Table 10.3 *PCI Device Path*
- Table 10.4 *PCCARD Device Path*
- Table 10.5 *Memory Mapped Device Path*
- Table 10.6 *Vendor-Defined Device Path*
- Table 10.7 *Controller Device Path*
- Table 10.8 *BMC Device Path*
- Table 10.9 *ACPI Device Path*
- Table 10.10 *Expanded ACPI Device Path*
- Table 10.11 *ACPI _ADR Device Path*
- Table 10.12 *NVDIMM Device Path*
- Table 10.13 *ATAPI Device Path*
- Table 10.14 *SCSI Device Path*
- Table 10.15 *Fibre Channel Device Path*
- Table 10.16 *Fibre Channel Ex Device Path*
- Table 10.17 *Fibre Channel Ex Device Path Example*
- Table 10.18 *1394 Device Path*
- Table 10.19 *USB Device Paths*
- Table 10.20 *USB Device Path Examples*
- Table 10.21 *Another USB Device Path Example*

- Table 10.22 *SATA Device Path*
- Table 10.23 *USB WWID Device Path*
- Table 10.24 *Device Logical Unit*
- Table 10.25 *USB Class Device Path*
- Table 10.26 *I 2 O Device Path*
- Table 10.27 *MAC Address Device Path*
- Table 10.28 *IPv4 Device Path*
- Table 10.29 *IPv6 Device Path*
- Table 10.31 *InfiniBand Device Path*
- Table 10.32 *UART Device Path*
- Table 10.33 *Vendor-Defined Messaging Device Path*
- Table 10.34 *UART Flow Control Messaging Device Path*
- Table 10.35 *SAS Messaging Device Path Structure*
- Table 10.36 *SAS Extended Messaging Device Path Structure*
- Table 10.37 *iSCSI Device Path Node (Base Information)*
- Table 10.38 *IPv4 configuration*
- Table 10.39 *IPv6 configuration*
- Table 10.40 *NVM Express Namespace Device Path*
- Table 10.41 *URI Device Path*
- Table 10.42 *UFS Device Path*
- Table 10.43 *SD Device Path*
- Table 10.44 *EFI Bluetooth Device Path*
- Table 10.45 *Wi-Fi Device Path*
- Table 10.46 *eMMC Device Path*
- Table 10.47 *EFI BluetoothLE Device Path*
- Table 10.48 *DNS Device Path*
- Table 10.49 *NVDIMM Namespace Device Path*
- Table 10.54 *Hard Drive Media Device Path*
- Table 10.55 *CD-ROM Media Device Path*
- Table 10.56 *Vendor-Defined Media Device Path*
- Table 10.57 *File Path Media Device Path*
- Table 10.58 *Media Protocol Media Device Path*
- Table 10.59 *PIWG Firmware File Device Path*
- Table 10.60 *PIWG Firmware Volume Device Path*
- Table 10.61 *Relative Offset Range*
- Table 10.62 *RAM Disk Device Path*

- Table 10.63 *BIOS Boot Specification Device Path*
- Table 10.64 *ACPI_CRS to EFI Device Path Mapping*
- Table 10.66 *EFI Device Path Option Parameter Values*
- Table 10.67 *Device Node Table*

Chapter 11

Chapter 12

- Table 12.1 *Supported Unicode Control Characters*
- Table B.1 *EFI Scan Codes for EFI_SIMPLE_TEXT_INPUT_PROTOCOL*
- Table B.2 *EFI Scan Codes for EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL*
- Table 12.10 *EFI Cursor Location/Advance Rules*
- Table 12.20 *PS/2 Mouse Device Path*
- Table 12.21 *Serial Mouse Device Path*
- Table 12.22 *USB Mouse Device Path*
- Table 12.32 *Blt Operation Table*
- Table 12.34 *Attributes Definition Table*

Chapter 13

- Table 13.24 *Tape Header Formats*
- Table 13.45 *PATA device mapping to ports and portmultiplier ports*
- Table 13.46 *Special Programming Considerations*

Chapter 14

- Table 14.5 *PCI Configuration Address*
- Table 14.15 *QWORD Address Space Descriptor*
- Table 14.16 *End Tag*
- Table 14.18 *PCI Root Bridge Device Path for a Desktop System*
- Table 14.19 *PCI Root Bridge Device Path for Bridge #0 in a Server System*
- Table 14.20 *PCI Root Bridge Device Path for Bridge #1 in a Server System*
- Table 14.21 *PCI Root Bridge Device Path for Bridge #2 in a Server System*
- Table 14.22 *PCI Root Bridge Device Path for Bridge #3 in a Server System*
- Table 14.37 *QWORD Address Space Descriptor*
- Table 14.38 *End Tag*
- Table 14.41 *PCI Device 7, Function 0 on PCI Root Bridge 0*
- Table 14.42 *PCI Device 7, Function 0 behind PCI to PCI bridge*
- Table 14.43 *Standard PCI Expansion ROM Header (Example from PCI Firmware Specification 3.0)*
- Table 14.44 *PCI Expansion ROM Code Types (Example from PCI Firmware Specification 3.0)*
- Table 14.45 *EFI PCI Expansion ROM Header*
- Table 14.46 *Device Path for an EFI Driver loaded from PCIO ption ROM*

- Table 14.47 *Recommended PCI Device Driver Layout*

Chapter 15

- Table 15.4 *SCSI Device Path Examples*
- Table 15.5 *ATAPI Device Path Examples*
- Table 15.6 *Fibre Channel Device Path Examples*
- Table 15.7 *InfiniBand Device Path Examples*
- Table 15.9 *Single Channel PCI SCSI Controller Behind a PCI Bridge*
- Table 15.10 *Channel #3 of a PCI SCSI Controller behind a PCIBridge*

Chapter 16

Chapter 17

- Table 17.10 *USB Hub Port Status Bitmap*
- Table 17.11 *Hub Port Change Status Bitmap*
- Table 17.12 *USB Port Features*
- Table 17.33 *Payload Associated Messages and Descriptions*

Chapter 18

- Table 18.1 *Debugport Messaging Device Path*

Chapter 19

- Table 19.1 *Block Header Fields*

Chapter 20

Chapter 21

Chapter 22

- Table 22.1 *General Purpose VM Registers*
- Table 22.2 *Dedicated VM Registers*
- Table 22.3 *VM Flags Register*
- Table 22.4 *Index Encoding*
- Table 22.5 *Index Size in Index Encoding*
- Table 22.6 *Opcode Byte Encoding*
- Table 22.7 *Operand Byte Encoding*
- Table 22.8 *ADD Instruction Encoding*
- Table 22.9 *AND Instruction Encoding*
- Table 22.10 *ASHR Instruction Encoding*
- Table 22.11 *VM Version Format*
- Table 22.12 *BREAK Instruction Encoding*
- Table 22.13 *CALL Instruction Encoding*
- Table 22.14 *CMP Instruction Encoding*
- Table 22.15 *CMPI Instruction Encoding*

- Table 22.16 *DIV Instruction Encoding*
- Table 22.17 *DIVU Instruction Encoding*
- Table 22.18 *EXTNDB Instruction Encoding*
- Table 22.19 *EXTNDD Instruction Encoding*
- Table 22.19 *EXTNDD Instruction Encoding*
- Table 22.20 *EXTNDW Instruction Encoding*
- Table 22.21 *JMP Instruction Encoding*
- Table 22.22 *JMP8 Instruction Encoding*
- Table 22.23 *LOADSP Instruction Encoding*
- Table 22.24 *MOD Instruction Encoding*
- Table 22.25 *MODU Instruction Encoding*
- Table 22.26 *MOV Instruction Encoding*
- Table 22.27 *MOVI Instruction Encoding*
- Table 22.28 *MOVIn Instruction Encoding*
- Table 22.29 *MOVn Instruction Encoding*
- Table 22.30 *MOVREL Instruction Encoding*
- Table 22.31 *MOVsn Instruction Encoding*
- Table 22.32 *MUL Instruction Encoding*
- Table 22.33 *MULU Instruction Encoding*
- Table 22.34 *NEG Instruction Encoding*
- Table 22.35 *NOT Instruction Encoding*
- Table 22.36 *OR Instruction Encoding*
- Table 22.37 *POP Instruction Encoding*
- Table 22.38 *POPn Instruction Encoding*
- Table 22.39 *PUSH Instruction Encoding*
- Table 22.40 *PUSHn Instruction Encoding*
- Table 22.41 *RET Instruction Encoding*
- Table 22.42 *SHL Instruction Encoding*
- Table 22.43 *SHR Instruction Encoding*
- Table 22.44 *STORESP Instruction Encoding*
- Table 22.45 *SUB Instruction Encoding*
- Table 22.46 *XOR Instruction Encoding*

Chapter 23

- Table 23.5 *PUSH_GUID Instruction Encoding*
- Table 23.6 *PUSH_VERSION Instruction Encoding*
- Table 23.7 *DECLARE_VERSION_NAME Instruction Encoding*

- Table 23.8 *AND Instruction Encoding*
- Table 23.9 *OR Instruction Encoding*
- Table 23.10 *NOT Instruction Encoding*
- Table 23.11 *TRUE Instruction Encoding*
- Table 23.12 *FALSE Instruction Encoding*
- Table 23.13 *EQ Instruction Encoding*
- Table 23.14 *GT Instruction Encoding*
- Table 23.15 *GTE Instruction Encoding*
- Table 23.16 *LT Instruction Encoding*
- Table 23.17 *LTE Instruction Encoding*
- Table 23.18 *END Instruction Encoding*
- Table 23.19 *DECLARE_LENGTH Instruction Encoding*
- Table 23.20 *ESRT and FMP Fields*

Chapter 24

- Table 24.13 *PXE Tag Definitions for EFI*
- Table 24.20 *Destination IP Filter Operation*
- Table 24.21 *Destination UDP Port Filter Operation*
- Table 24.22 *Source IP Filter Operation*
- Table 24.23 *Source UDP Port Filter Operation*

Chapter 25

Chapter 26

Chapter 27

Chapter 28

Chapter 29

- Table 29.8 *DHCP4 Enumerations*
- Table 29.19 *Field Descriptions*
- Table 29.21 *Callback Return Values*

Chapter 30

- Table 30.18 *Descriptions of Parameters in MTFTPv4 PacketStructures*
- Table 30.30 *Descriptions of Parameters in MTFTPv6 PacketStructures*
- Table 30.31 *MTFTP Packet OpCode Descriptions*
- Table 30.32 *MTFTP ERROR Packet ErrorCode Descriptions*

Chapter 31

Chapter 32

- Table 32.2 *Generic Authentication Node Structure*
- Table 32.3 *CHAP Authentication Node Structure using RADIUS*

- Table 32.4 *CHAP Authentication Node Structure using Local Database*
- Table 32.5 *PE/COFF Certificates Types and UEFI Signature Database Certificate Types*
- Table 32.6 *Authentication Attempt Status Codes*

Chapter 33

- Table 33.1 *Localization Issues*
- Table 33.2 *Information for Types of Storage*
- Table 33.3 *Common Control Codes for Font Display Information*
- Table 33.11 *Guidelines for UEFI System Fonts*
- Table 33.12 *Truth Table: Mapping A Single Question To Three Configuration Settings*
- Table 33.13 *Multiple Configuration Settings Example #2*
- Table 33.14 *Values*
- Table 33.15 *Package Types*
- Table 33.18 *Block Types*
- Table 33.20 *IFR Opcodes*
- Table 33.21 *VarStoreType Descriptions*
- Table 33.22 *Animation Block Types*

Chapter 34

Chapter 35

- Table 35.11 *Callback Behavior*

Callback 36

- Table 36.15 *Record Values and Descriptions*
- Table 36.16 *Standard Values for Access to Configure the Platform*

Chapter 37

- Table 37.1 *EFI Hash Algorithms*
- Table 37.2 *Identical Hash Results*
- Table 37.8 *Algorithms that may be used with EFI_HASH2_PROTOCOL*
- Table 37.9 *Encryption Algorithm Properties*
- Table 37.22 *Details of Supported Signature Format*

Appendix A

- Table A.1 *EFI GUID Format*
- Table A.2 *Text representation relationships*

Appendix B

- Table B.1 *EFI Scan Codes for EFI_SIMPLE_TEXT_INPUT_PROTOCOL*
- Table B.2 *EFI Scan Codes for EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL*
- Table B.3 *Control Sequences to Implement EFI_SIMPLE_TEXT_INPUT_PROTOCOL*

Appendix C

- Table C.1 *Legacy Floppy Device Path*
- Table C.2 *IDE Disk Device Path*
- Table C.3 *Secondary Root PCI Bus with PCI to PCI Bridge Device Path*

Appendix D

- Table D.1 *EFI_STATUS Code Ranges*
- Table D.2 *EFI_STATUS Success Codes (High Bit Clear)*
- Table D.3 *EFI_STATUS Error Codes (High Bit Set)*
- Table D.4 *EFI_STATUS Warning Codes (High Bit Clear)*

Appendix E

- Table E.1 *Definitions*
- Table E.2 *Referenced Specifications*
- Table E.3 *Driver Types: Pros and Cons*
- Table E.4 *!PXE Structure Field Definitions*
- Table E.5 *UNDI CDB Field Definitions*

Appendix F

Appendix G

Appendix H

Appendix I

Appendix J

- Table J.1 *EBC Virtual Machine Opcode Summary*

Appendix K

Appendix L

- Table L.1 *Protocol Name changes*
- Table L.2 *Revision Identifier Name Changes*

Appendix M

- Table M.1 *Alias Codes Supported in Addition to RFC 4646*

Appendix N

- Table N.1 *Error record header*
- Table N.2 *Error Record Header Flags*
- Table N.3 *Error Status Fields*
- Table N.4 *Error Types*
- Table N.5 *Section Descriptor*
- Table N.6 *Processor Generic Error Section*
- Table N.7 *Processor Error Record*
- Table N.8 *IA32/X64 Processor Error Information Structure*
- Table N.9 *IA32/X64 Cache Check Structure*

- Table N.10 *IA32/X64 TLB Check Structure*
- Table N.11 *IA32/X64 Bus Check Structure*
- Table N.12 *IA32/X64 MS Check Field Description*
- Table N.13 *IA32/X64 Processor Context Information*
- Table N.14 *IA32 Register State*
- Table N.15 *X64 Register State*
- Table N.16 *ARM Processor Error Section*
- Table N.17 *ARM Processor Error Information Structure*
- Table N.19 *ARM TLB Error Structure*
- Table N.20 *ARM Bus Error Structure*
- Table N.21 *ARM Processor Error Context Information HeaderStructure*
- Table N.22 *ARMv8 AArch32 GPRs (Type 0)*
- Table N.23 *ARM AArch32 EL1 Context System Registers (Type 1)*
- Table N.24 *ARM AArch32 EL2 Context System Registers (Type 2)*
- Table N.25 *ARM AArch32 secure Context System Registers (Type3)*
- Table N.26 *ARMv8 AArch64 GPRs (Type 4)*
- Table N.27 *ARM AArch64 EL1 Context System Registers (Type 5)*
- Table N.28 *ARM AArch64 EL2 Context System Registers (Type 6)*
- Table N.29 *ARM AArch64 EL3 Context System Registers (Type 7)*
- Table N.30 *ARM Misc. Context System Register (Type 8) - SingleRegister Entry*
- Table N.32 *Memory Error Record*
- Table N.33 *Memory Error Record 2*
- Table N.34 *PCI Express Error Record*
- Table N.35 *PCI/PCI-X Bus Error Section*
- Table N.36 *PCI/PCI-X Component Error Table*
- Table N.37 *Firmware Error Record Reference*
- Table N.38 *DMAr Generic Errors*
- Table N.39 *Intel® VT for Directed I/O specific DMAr Errors*
- Table N.40 *IOMMU-specific DMAr Errors*
- Table N.41 *CCIX PER Log Error Record*
- Table N.42 *CXL Protocol Error Section*
- Table N.43 *CXL Component Event Log Record*

Appendix O

- Table O.1 *UEFI Table Structure*

Appendix P

Appendix Q

Appendix R

List of Figures

Chapter 1

Fig. 1.1 *UEFI Conceptual Overview*

Chapter 2

Fig. 2.1 *Booting Sequence*

Fig. 2.2 *Stack After AddressOfEntryPoint Called, IA-32*

Fig. 2.3 *Stack after AddressOfEntryPoint Called, Itanium-based Systems*

Fig. 2.4 *Construction of a Protocol*

Fig. 2.5 *Desktop System*

Fig. 2.6 *Server System*

Fig. 2.7 *Image Handle*

Fig. 2.8 *Driver Image Handle*

Fig. 2.9 *Host Bus Controllers*

Fig. 2.10 *PCI Root Bridge Device Handle*

Fig. 2.11 *Connecting Device Drivers*

Fig. 2.12 *Connecting Bus Drivers*

Fig. 2.13 *Child Device Handle with a Bus Specific Override*

Fig. 2.14 *Software Service Relationships*

Chapter 5

Fig. 5.1 *MBRDisk Layout with legacy MBR example*

Fig. 5.2 *GPT disk layout with protective MBR*

Fig. 5.3 *GPT disk layout with protective MBR on a disk with capacity > LBA 0xFFFFFFFF*

Fig. 5.4 *GUID Partition Table (GPT) example*

Chapter 6

Fig. 6.1 *The BTT Layout in a BTT Arena*

Fig. 6.2 *A BTT With Multiple Arenas in a Large Namespace*

Fig. 6.3 *Cyclic Sequence Numbers for Flog Entries*

Fig. 6.4 *BTT Read Path Overview*

Fig. 6.5 *BTT Write Path Overview*

Chapter 7

Fig. 7.1 *Device Handle to Protocol Handler Mapping*

Fig. 7.2 *Handle Database*

Chapter 8

Fig. 8.1 *Scatter-Gather List of EFI_CAPSULE_BLOCK_DESCRIPTOR Structures*

Chapter 9

Chapter 10

Chapter 11

Fig. 11.1 *Driver Health Status States*

Chapter 12

Fig. 12.1 *Serial Device Identification Driver Relationships*

Fig. 12.2 *Software BLT Buffer*

Chapter 13

Fig. 13.1 *Nesting of Legacy MBR Partition Records*

Fig. 13.2 *Cyclic Sequence Numbers in Label Index Block*

Fig. 13.3 *Organization of the Label Storage Area*

Chapter 14

Fig. 14.1 *Host Bus Controllers*

Fig. 14.2 *Device Handle for a PCI Root Bridge Controller*

Fig. 14.3 *Desktop System with One PCI Root Bridge*

Fig. 14.4 *Server System with Four PCI Root Bridges*

Fig. 14.5 *Server System with Two PCI Segments*

Fig. 14.6 *Server System with Two PCI Host Buses*

Fig. 14.7 *Image Handle*

Fig. 14.8 *PCI Driver Image Handle*

Fig. 14.9 *PCI Host Bus Controller*

Fig. 14.10 *Device Handle for a PCI Host Bus Controller*

Fig. 14.11 *Physical PCI Bus Structure*

Fig. 14.12 *Connecting a PCI Bus Driver*

Fig. 14.13 *Child Handle Created by a PCI Bus Driver*

Fig. 14.14 *Connecting a PCI Device Driver*

Fig. 14.15 *Unsigned PCI Driver Image Layout*

Fig. 14.16 *Signed and Compressed PCI Driver Image Flow*

Fig. 14.17 *Signed and Compressed PCI Driver Image Layout*

Fig. 14.18 *Signed but not Compressed PCI Driver Image Flow*

Fig. 14.19 *Signed and Uncompressed PCI Driver Image Layout*

Chapter 15

Fig. 15.1 *Device Handle for a SCSI Bus Controller*

Fig. 15.2 *Child Handle Created by a SCSI Bus Driver*

Chapter 16

Chapter 17

Fig. 17.1 *Software Triggered State Transitions of a USB Host Controller*

Fig. 17.2 *USB Bus Controller Handle*

Fig. 17.3 *Sequence of Operations with Endpoint Policy Changes*

Chapter 18

Fig. 18.1 *Debug Support Table Indirection and Pointer Usage*

Chapter 19

Fig. 19.1 *Bit Sequence of Compressed Data*

Fig. 19.2 *Compressed Data Structure*

Fig. 19.3 *Block Structure*

Fig. 19.4 *Block Body*

Fig. 19.5 *String Info Log Search Tree*

Fig. 19.6 *Node Split*

Chapter 20

Chapter 21

Chapter 22

Chapter 23

Fig. 23.1 *Firmware Image with no Authentication Support*

Fig. 23.2 *Firmware Image with Authentication Support*

Fig. 23.3 *Firmware Image with Dependency/AuthenticationSupport*

Fig. 23.4 *Optional Scatter-Gather Construction of Capsule Submitted to Update Capsule()*

Fig. 23.5 *Capsule Header and Firmware Management Capsule Header*

Fig. 23.6 *Firmware Management and Firmware Image Management headers*

Chapter 24

Fig. 24.1 *IPv6-based PXE Boot*

Fig. 24.2 *Netboot6 (DHCP6 and ProxyDHCP6 reside on the same server)*

Fig. 24.3 *IPv6-based PXE boot (DHCP6 and ProxyDHCP6reside on the different server)*

Fig. 24.4 *HTTP Boot Network Topology Concept - Corporate Environment*

Fig. 24.5 *HTTP Boot Network Topology with Proxy Concept - Corporate Environment*

Fig. 24.6 *HTTP Boot Network Topology Concept2 — Homeenvironments*

Fig. 24.7 *UEFI HTTP Boot Protocol Layout*

Fig. 24.8 *HTTP Boot Overall Flow*

Chapter 25

Chapter 26

Chapter 27

Chapter 28

Chapter 29

Fig. 29.1 *EFI REST Support, Single Protocol*

Fig. 29.2 *EFI REST Support, Multiple Protocols*

Fig. 29.3 *EFI REST Support, BMC on Board*

Fig. 29.4 *EFI REST Support, Redfish Service*

Fig. 29.5 *EFI REST Support, Protocol Usages*

Chapter 30

Chapter 31

Chapter 32

Fig. 32.1 *Creating A Digital Signature*

Fig. 32.2 *Verifying a Digital Signature*

Fig. 32.3 *Embedded Digital Certificates*

Fig. 32.4 *Secure Boot Modes*

Fig. 32.5 *Signature Lists*

Fig. 32.6 *Process for Adding a New Signature by the OS*

Fig. 32.7 *Authorization Process Flow*

Chapter 33

Fig. 33.1 *Platform Configuration Overview*

Fig. 33.2 *HII Resources In Drivers & Applications*

Fig. 33.3 *Creating UI Resources With Resource Files*

Fig. 33.4 *Creating UI Resources With Intermediate Source Representation*

Fig. 33.5 *The Platform and Standard User Interactions*

Fig. 33.6 *User and Platform Component Interaction*

Fig. 33.7 *User Interface Components*

Fig. 33.8 *Connected Forms Browser/Processor*

Fig. 33.9 *Disconnected Forms Browser/Processor*

Fig. 33.10 *O/S-Present Forms Browser/Processor*

Fig. 33.11 *Platform Data Storage*

Fig. 33.12 *Keyboard Layout*

Fig. 33.13 *Forms-based Interface Example*

Fig. 33.14 *Platform Configuration Overview*

Fig. 33.15 *Question Value Retrieval Process*

Fig. 33.16 *Question Value Change Process*

Fig. 33.17 *String Identifiers*

Fig. 33.18 *Fonts*

Fig. 33.19 *Font Description Terms*

Fig. 33.20 *16 x 19 Font Parameters*

Fig. 33.21 *Font Structure Layout*

Fig. 33.22 *Proportional Font Parameters and Byte Padding*

Fig. 33.23 *Aligning Glyphs*
Fig. 33.24 *HII Database*
Fig. 33.25 *Setup Browser*
Fig. 33.26 *Storing Configuration Settings*
Fig. 33.27 *OS Runtime Utilization*
Fig. 33.28 *Standard Application Obtaining Setting Example*
Fig. 33.29 *Typical Forms Processor Decisions Necessitating a Callback (1)*
Fig. 33.30 *Typical Forms Processor Decisions Necessitating a Callback (2)*
Fig. 33.31 *Typical Forms Processor Decisions Necessitating a Callback (3)*
Fig. 33.32 *Driver Model Interactions*
Fig. 33.33 *Managing Human Interface Components*
Fig. 33.34 *EFI IFR Form Set configuration*
Fig. 33.35 *EFI IFR Form Set Question Changes*
Fig. 33.36 *Glyph Information Encoded in Blocks*
Fig. 33.37 *Glyph Block Processing*
Fig. 33.38 *EFI_HII_GIBT_GLYPH_VARIABLITY Glyph Drawing Processing*
Fig. 33.39 *String Information Encoded in Blocks*
Fig. 33.40 *String Block Processing: Base Processing*
Fig. 33.41 *String Block Processing: SCSU Processing*
Fig. 33.42 *String Block Processing: UTF Processing*
Fig. 33.43 *Image Information Encoded in Blocks*
Fig. 33.44 *Palette Structure of a Black & White, One-BitImage*
Fig. 33.45 *Palette Structure of a Four-Bit Image*
Fig. 33.46 *Palette Structure of a Four-Bit, Six-ColorImage*
Fig. 33.47 *Simple Binary Object*
Fig. 33.48 *Password Flowchart (part one)*
Fig. 33.49 *Password Flowchart (part two)*
Fig. 33.50 *Animation Information Encoded in Blocks*

Chapter 34

Fig. 34.1 *Glyph Example*
Fig. 34.2 *How EFI_HII_IMAGE_EX_PROTOCOL uses EFI_HII_IMAGE_DECODER_PROTOCOL*
Fig. 34.3 *Keyboard Layout*

Chapter 35

Chapter 36

Fig. 36.1 *User Identity*
Fig. 36.2 *User Identity Manager*

Chapter 37

Fig. 37.1 Hash workflow

Chapter 38

Chapter 39

Appendix A

Appendix B

Appendix C

Fig. C.1 Example Computer System

Fig. C.2 Partial ACPI Name Space for Example System

Fig. C.3 EFI Device Path Displayed As a Name Space

Appendix D

Appendix E

Fig. E.1 Network Stacks with Three Classes of Drivers

Fig. E.2 !PXE Structures for H/W and S/W UNDI

Fig. E.3 Issuing UNDI Commands

Fig. E.4 UNDI Command Descriptor Block (CDB)

Fig. E.5 Storage Types

Fig. E.6 UNDI States, Transitions & Valid Commands

Fig. E.7 Linked CDBs

Fig. E.8 Queued CDBs

Appendix F

Appendix G

Appendix H

Appendix I

Appendix J

Appendix K

Appendix L

Appendix M

Appendix N

Fig. N.1 Error Record Format

Appendix O

Appendix P

Appendix Q

Appendix R

Revision History

Many people have contributed to the contents of this specification, including the following:

- * UEFI Specification Working Group (USWG)
- * Tianocore Community Members
- * Others noted in the Revision History below

Changes in this release

Revision	Issue # - Description	Modified/Added Content
2.11	2365 - New Memory Attribute for hot plug	Section 7.2.3
2.11	2371 - Correct the description of EFI_KMS_PROTOCOL.DeleteKey(), KeyDescriptors	Section 37.3.1, Section 37.3.2.5
2.11	2384 - Mandate the UEFI Arm implementation to allocate pages from the 48-bit address range first	Section 2.3.6
2.11	2386 - EFI_RNG_PROTOCOL Describe the DRBG algorithm used in the Arm RNDR instruction	Section 37.5.4
2.11	2388 - Add support for Proxy Host URI in HTTP Boot	Section 24.7.2.1, Section 24.7.3.1, Section 24.7.10, Section 29.6.6
2.11	2393 - Arm CPER processor error section, support FEAT_D128	Table N.31
2.11	2431 - CodeFirst: Adjust LoongArch exception and interrupt types	Section 18.2.5
2.11	2437 - CodeFirst: Add new HTTP Boot Callback Type	Section 24.7.13
2.11	2440 - CPER definition for FRU memory poison	Section N.2.15
2.11	2443 - Add Storage Device Class for EFI_BOOT_MANAGER_POLICY_CONNECT_DEVICE_CLASS	Section 3.2.3
2.11	2448 - update UEFI spec version to 2.11	Title page
2.11	2449 - RISC-V AP-TEE Confidential Computing Extension for UEFI	Section 38.4.2
2.11	2453 - Add SM2, SM3 crypto algorithm	Table 32.5, Section 32.4.1
2.11	2457- Code-First: Changes to CPER definition for PCIe	Table N.34, Table N.42
2.11	2462 - Fix the Type mnemonic description in the ARM Processor Error Information Structure	Section N.2.4.4.1
2.11	2468 - Elaborate on EFI_LOADED_IMAGE_PROTOCOL.Unload() usage	Section 9.1, Section 9.1.2, Section 9.2
2.11	2475 - Typo in the EFI_TABLE_HEADER “HeaderSize” field description	Section 4.2
2.11	2476 - Correct the section header for EFI_CC_MEASUREMENT_PROTOCOL.GetEventLog	Section 38.2.3
2.11	2480 - Deprecate CryptoIndications variables	Modified Section 3.3, and removed Section 32.5 Firmware/OS Crypto Algorithm Exchange
2.11	2483 - Collection of Non-Functional Fixes	Various content and references in spec

Changes in previous releases

Revision	Issue # - Description	Modified/Added Content
----------	-----------------------	------------------------

continues on next page

Table 2 – continued from previous page

2.10A	2016 - Compression/decompression clean up	Section I
2.10A	2359 - Update the PASS_THRU_PROTOCOL to Enhance the logic for AtaPassThruGetNextPort()/AtaPassThruGetNext	Section 13.13.4
2.10A	2360 - Clarify that EFI_MEMORY_WB and _WT share ability attributes on AArch64	Table 2.5
2.10A	2367 - EFI_FILE_PROTOCOL.Open() suggests that a file may be opened relative to a regular file	Section 13.5.1, Section 13.5.2, Section 13.5.3
2.10A	2368 - mixed up error codes for EFI_FILE_PROTOCOL.OpenEx()	Section 13.5.9
2.10A	2369 - EFI_TABLE_HEADER Revision field cannot represent version 2.10	Section 1.9.9, Section 4.2
2.10A	2373 - Typo in spec EFIBOOTBOOT and EFIBOOT-BOOT{machine type short-name}.EFI type	Section 3.5.1.1
2.10A	2376 - Update the RISC-V Platforms section for more concise language	Section 2.3.7, Section 2.3.7.1, Section 2.3.7.2, Section 2.3.7.3, Section 2.3.8
2.10A	2408 - Recommended PCI Device Driver Layout missing RV and LoongArch	Table 14.47, Section 14.4.25
2.10A	2411 - Fix incorrect references in the Platform-Specific Elements section.	Section 2.6.2
2.10A	2412 - Update references and remove extraneous text.	Section 14.4.21
2.10A	2415 - Update reference links in chapters 8 and 32.	Section 8, Section 32.3.2, Section 32.5.3, Section 32.6.3, Section 32.7.1
2.10A	2417 - Remove old text from the VendorTable definition for the EFI_CONFIGURATION_TABLE	Section 4.6.1
2.10A	2421 - Update DEFAULT_TTL to IANA's default value of 64.	Section 24.3.5
2.10A	2428 - Update Status Codes Returned for EFI_FIRMWARE_MANAGEMENT_PROTOCOL.GetImageInfo()	Section 23.1.2
2.10A	2438 - Wrong Text Device Node for NVDIMM Namespace path	Table 10.67
2.10A	2439 - Include ACPI Device Path Subtype 4 for NVDIMM Device	Table 10.67
2.10A	2445 - Fix typos and other cleanup	Section 3.5.1.1, Section 8.5.5, Section 18.2.2, Section 29.3.13
2.10A	2447 - Undefined behavior of SetVariable(EFI_VARIABLE_APPEND_WRITE) for non-existent variable	Section 8.2.3
2.10A	2455 - Precedence of return codes for QueryVariableInfo	Section 8.2.4
2.10A	2456 - Remove deprecated content	various content and references in spec
2.10A	2462 - Arm CPER Processor Error Type values defined incorrectly	Table N.17
2.10	2205 - Enabling SHA-384/SHA-512 signing scheme for Authenticated Variables	Section 8.2.6
2.10	2207 - EFI_SECURITY_VIOLATION can't be returned by EFI_FIRMWARE_MANAGEMENT_PROTOCOL.GetImageInfo()	Section 23.1.3
2.10	2217 - Device Authentication Signature Database	Section 3.3, Section 32.4.1, Section 32.6, Section 32.7.2
2.10	2229 - Support ISA-specific memory attributes in descriptors	Section 2.3.6.1, Section 7.2.3
2.10	2247 - Support crypto agile	Section 2.6.5, Section 8.2.5, Section 8.2.6, Section 27.2, Section 28.9, Section 32.3, Section 32.5.3.3, Section 37.4
2.10	2262 - Add Memory Protection proposal - UEFI_MEMORY_ATTRIBUTE protocol	Section 37.7.1

continues on next page

Table 2 – continued from previous page

2.10	2266 - Code First - Image Execution Table - revocations of hashes (Samer El-Haj-Mahmoud)	Section 32.4.2
2.10	2271 - Introduce UEFI Conformance Profiles	Section 2.6, Section 4.6.4
2.10	2277 - Code first - Uart() UEFI DevicePath binary/text confusion issue (Samer El-Haj-Mahmoud)	Section 10.6.1.6
2.10	2278 - AARCH64 binding requirement for an OS calling RT services on platforms with SME	Section 2.3.6, Section 2.3.6.4
2.10	2291 - Support crypto agile - Address crypto agile compatibility	[deprecated in UEFI 2.11]
2.10	2292 - Forward Control Flow Guard Instruction runtime indicator	Section 4.6.3
2.10	2313 - Add LoongArch architecture support to UEFI specification	Section 2.3.8
2.10	2315 - Add NVM Express over Fabrics messaging device path AND NVMe Trademark updates	Section 6.1, Table 10.52, Section 10.3.4.32, Section 10.3.4.33, Table 10.67, Section 13.15.2
2.10	2317 - Add confidential computing extension for UEFI	Section 38
2.10	2318 - Update boot requirement for RISC-V platform	Section 2.3.7.1
2.10	2320 - Remove EBBR Conformance profile	Section 4.6.4
2.10	2329 - Update the UEFI to Version 2.10	Section 4.3
2.10	2336 - Feedback on UEFI 2.10 draft	sections throughout
2.10	2337 - Code First -Add LoongArch to section UEFI Images Boot Manager PCI Option ROMs and Debugger Support sections (Li Chao)	sections throughout
2.10	2339 - Re-add RSA 4k support for UEFI 2.10 crypto agility	[deprecated in UEFI 2.11]
2.10	2342 - GetHealthStatus: Make the statement and table consistent for EFI_UNSUPPORTED for Controller Handle Null case	Section 11.10.2
2.9A	2225 - Clarify the specification requirements around processing Boot#### variable	Section 3.1.7, Table 3.3, Section 7.1.2
2.9A	2227 - Clarify NVMe device path EUFI-64 byte order	Section 10.3.4.21
2.9A	2235 - Clarify EFI_LOAD_OPTION.FilePathList[] device path definition	Section 3.1.3
2.9A	2243 - Removing old references to Wired for Management (WfM)	Section 3.5.2.1, Section 7.3.1, Table 24.13, <i>Referenced Specifications, Glossary</i>
2.9A	2249 - Cleanup of the SI & Binary Prefixes section	Section 1.9.8
2.9A	2251 - Clarification of DevicePath examples using 0xFF for End of HW DP	Section 10.3.4.4, Table 12.20, Table 14.18, Table 15.4, <i>Legacy Floppy</i>
2.9A	2252 - Clarify OS dependency on UEFI vs PI interfaces	Section 1.4
2.9A	2263 - CXL CPER updates	Table N.42
2.9A	2270 - Add “CPER” acronym to Appendix N	<i>Common Platform Error Record (CPER)</i>
2.9A	2286 - Section header for <i>CHAP (using RADIUS) Authentication Node</i>	Section 32.1.6
2.9A	2306 - Define Arm CPER Processor Error Types	<i>ARM Processor Error Information Structure</i>
2.9A	2311 - Define the DevicePath argument from LoadImage as optional	Section 7.4.1
2.9A	2327 - Correcting subtype value for <i>REST Service Device Path</i>	Section 10.3.4.31
2.9A	2330 - Principle of Inclusive Terminology statement	Section 1.1

Revision	Issue # - Description	Release Date
2.9	1866 GetInfo() of Adapter Information Protocol should have a provision for IHV to return no data	March 2021
2.9	1982 Clarify the PKCS#7 SignedData structure of EFI_VARIABLE_AUTHENTICATION	March 2021
2.9	1986 Need a mechanism using which browser to exit out of IHV formset silently without any popup	March 2021
2.9	1989 NVDIMM SPA Location Cookie	March 2021
2.9	2024 CXL CPER Records	March 2021
2.9	2042 New Event Group EFI_EVENT_GROUP_ AF-TER_READY_TO_BOOT	March 2021
2.9	2043 New Event Group EFI_EV-ENT_GROUP_BEFORE_EXIT_BOOT_SERVICES	March 2021
2.9	2046 Add support for Key 14 & 56 for Japanese keyboard layout	March 2021
2.9	2053 Figure/Table Numbers are Duplicated in Appendices	March 2021
2.9	2062 Table numbering to restart for each chapter	March 2021
2.9	2065 CXL proposal for CDAT table extraction from devices	March 2021
2.9	2093 UpdateCapsule ScatterGatherList cache maintenance	March 2021
2.9	2129 Add DTB Configuration Table standard GUID	March 2021
2.9	2131 Clarify Console requirements	March 2021
2.9	2134 Introduce unaccepted memory type	March 2021
2.9	2155 Typo in Arm Processor CPER Error Section	March 2021
2.9	2167 CPER for CXL Component Events	March 2021
2.9	2185 Declaration for UEFI 2.9 specification in the System Table	March 2021
2.9	2190 Misc. spec review feedback	March 2021
2.9	2199 EFI_IMAGE_EXECUTION_INFO_TABLE references	March 2021
2.9	2200 Config tables references from section 4.6	March 2021
2.9	2204 Typo in GUID definition for EFI_MANAGED_NETWORK_SERVICE_BINDING_PROTOCOL	March 2021
2.9	2212 Incorrect cross reference to User Information Table	March 2021
2.8 C	2117 - E FI_BROWSER_ACTION_REQUEST_RECONNECT - perform the action when user exits out of formset	Jan. 2021
2.8 C	2139 Update RISC-V UEFI corresponding spec to align with latest RISC-V spec	Jan. 2021
2.8 C	2155 Typo in Arm Processor CPER Error Section	Jan. 2021
2.8 C	2158 EFI_DRIVE_R_HEALTH_PROTOCOL.GetHealthStatus() - Driver not managing any controller	Jan. 2021
2.8 C	2172 Revise EFI_REDFISH_DISCOVER_PROTOCOL definitions to match the implementation.	Jan. 2021
2.8 C	2173 Question on EFI_CAPSULE_HEADER Flags definition	Jan. 2021
2.8 C	2184 FI_BOOT_MANAGER_POLICY_PROTOCOL typos	Jan. 2021
2.8 C	2190 EFI_SUCCESS misspelled in five places	Jan. 2021
2.8 B	1926 update: corrected EFI_SYSTEM_TABLE entries from 2_8 to 2_80	May 2020
2.8 B	1935 update: removed a space from several references to the EFI_JSON_CAPSULE_ID_GUID	May 2020
2.8 B	2073 Modify definition of “DPA” for use with CXL based devices	May 2020
2.8 B	2074 Memory Range typo	May 2020
2.8 B	2080 Typo in N.2.2 Section Descriptor Table 56	May 2020
2.8 B	2083 Typo in guid definition of CCIX PER Log Error Section	May 2020

continues on next page

Table 3 – continued from previous page

2.8 B	2088 Clarifications on caller-freed buffers	May 2020
2.8 B	2091 Inconsistency in description of EFI_FIRMWARE_MANAGEMENT_CAPSULE_IMAGE_HEADER structure	May 2020
2.8 B	2092 Typo in definition of PEI Notification type in Table 269. Error record header	May 2020
2.8 B	2095 PCI I/O attribute typos in section 14.4 “EFI PCI I/O Protocol”	May 2020
2.8 B	UEFI Runtime Service Table correction	May 2020
2.8 B	2096 Typo in definition of EFI_JSON_CONFIG_DATA_ITEM	May 2020
2.8 A	1970 Security Command Protocol change for OPAL RAID devices	February 2020
2.8 A	1998 Update RISC-V related spec	February 2020
2.8 A	2000 JSON Capsule clarification	February 2020
2.8 A	2002 Memory allocations between ExitBootServices calls	February 2020
2.8 A	2009 DMTF references in UEFI spec	February 2020
2.8 A	2013 Correct EFI _BOOT_SERVICES.DisconnectController contradicting info in Description	February 2020
2.8 A	2018 EFI_EDID_OVERRIDE_PROTOCOL.GET_EDID should take an EFI_HANDLE as ChildHandle	February 2020
2.8 A	2020 EFI_LOADED_IMAGE_PROTOCOL.LoadOptions does not mention it is related to Load Options.	February 2020
2.8 A	2025 Capsule Depex Length Declaration	February 2020
2.8 A	2026 FMP Capsule Image Header extension	February 2020
2.8 A	2029 Add missing GUIDs in Appendix N	February 2020
2.8 A	2030 Fix spec index to show the Appendix chapters	February 2020
2.8 A	2034 Depex added description	February 2020
2.8 A	2035 Fix OUT parameters marked as IN OUT	February 2020
2.8 A	2036 SetVariable errata: clarify that in-place variable update is supported	February 2020
2.8 A	2038 Configuration Tables Errata	February 2020
2.8 A	2041 EFI_EVENT_GROUP_EXIT_BOOT_SERVICES Errata	February 2020
2.8 A	2050 Incomplete list of EFI_SERVICE_BINDING_PROTOCOL protocols	February 2020
2.8 A	2049 RuntimeServicesSupported EFI variable should be a config table	February 2020
2.8 A	2051 Typo in Table - CPER IA32/X64 Bus Check Structure	February 2020
2.8 A	2053 Figure/Table Numbers are Duplicated in Appendices	February 2020
2.8	1832 Extend SERIAL_IO with DeviceTypeGuid	March 2019
2.8	1834 UEFI REST EX Protocol	March 2019
2.8	1853 Adding support for a REST style formset	March 2019
2.8	1858 New Device Path for bootable NVDIMM namespaces	March 2019
2.8	1861 New EFI_MEMORY_RANGE_CAPSULE Descriptor	March 2019
2.8	1866 GetInfo() of Adapter Information Protocol should have a provision for IHV to return no data	March 2019
2.8	1872 Peripheral-attached Memory	March 2019
2.8	1876 Remove the EBC support requirement	March 2019
2.8	1879 Clarification of REST (EX) protocol	March 2019
2.8	1908 Update of uncommitted data in the FOROM_OPEN callback	March 2019
2.8	1919 Memory Cryptography Attribute	March 2019
2.8	1920 Redfish Discover Protocol	March 2019
2.8	1921 HTTPS hostname validation	March 2019

continues on next page

Table 3 – continued from previous page

2.8	1924 Update to EFI_REST_EX_PROTOCOL.AsyncSendReceive	March 2019
2.8	1925 Clarify requirement of REST related protocols	March 2019
2.8	1926 New UEFI Spec Revision → 2.8	March 2019
2.8	1935 UEFI JSON Capsule Support	March 2019
2.8	1936 ResetSystem - support ResetData for all status scenarios.	March 2019
2.8	1937 Behavior of default values	March 2019
2.8	1941 New EFI REST JSON Structure Protocol	March 2019
2.8	1942 Adding dependency expression capability into FMP type capsules	March 2019
2.8	1947 Keyword strings of Configuration Keyword Handler Protocol Enhancements	March 2019
2.8	1953 Add document version# conventions	March 2019
2.8	1954 set (*Attributes) when GetVariable() returns EFI_BUFFER_TOO_SMALL and Attributes is non-NULL	March 2019
2.8	1956 Platform to honor ActionRequest for Action changing	March 2019
2.8	1961 Add EFI_UNSUPPORTED to EFI_RUNTIME_SERVICES calls	March 2019
2.8	1966 Add new capsule processing error codes	March 2019
2.8	1974 Add new CCIX PER Log Error Section to appendix	March 2019
2.8	1996 Firmware Processing of the Capsule Identified by EFI_JSON_CAPSULE_ID_GUID	March 2019
2.7B	1773 Clarify The EFI System Table entry for capsule image	March 2019
2.7B	1801 ExtractConfig() format may change when called multiple times	March 2019
2.7B	1835 Misleading / unclear statement about EFI-bootability of UDF media	March 2019
2.7B	1838 RGB/BGR Contradiction in 2.7 GOP	March 2019
2.7B	1841 BluetoothLE ECR - support autoreconnect	March 2019
2.7B	1842 BluetoothLE ECR - Add missing ConnectionCompleteCallback	March 2019
2.7B	1843 HTTP Example Code Update	March 2019
2.7B	1844 Replace obsoleted RFC number with new number for TCP	March 2019
2.7B	1845 Clarification on AIP types “Network boot” and “SAN MAC Address”	March 2019
2.7B	1846 EFI_LOAD_FILE2 requirement	March 2019
2.7B	1865 Adding clarification in EFI_NOT_READY for ReadKeyStrokeEx()	March 2019
2.7B	1869 Clarify FMP buffer too small behavior	March 2019
2.7B	1874 Add RFC3021 to reference in uefi.org	March 2019
2.7B	1875 Clarify platform specific elements in chapter 2.6.2	March 2019
2.7B	1878 Errata - Make DHCP server optional for HTTP boot	March 2019
2.7B	1880 Arm binding EL2 register state clarification	March 2019
2.7B	1890 EfiMemoryMappedIO Usage Clarification	March 2019
2.7B	1897 Clarification on mapping of UEFI memory attributes to ARM memory types and paging attributes	March 2019
2.7B	1899 Errata: Clarify EFI_INVALID_PARAMETER for FMP->GetImageInfo()	March 2019
2.7B	1901 GPT Protective MBR description	March 2019
2.7B	1902 CapsuleImageSize Clarification	March 2019
2.7B	1903 Root Directory File Name	March 2019
2.7B	1906 ACPI Table Pointer Installation	March 2019
2.7B	1908 Update of uncommitted data in the FORUM_OPEN callback	March 2019

continues on next page

Table 3 – continued from previous page

2.7B	1923 Syntax error in EFI iSCSI Initiator Name Protocol	March 2019
2.7B	1957 Request to add status code EFI_DEVICE_ERROR for ExtractConfig	March 2019
2.7B	1964 Print disclaimer for all future UEFI specs	March 2019
2.7B	1987 incorrect VLAN_CONFIG_SET function definition	March 2019
2.7A	1830 Label Protocol - EFI_NVDIMM_LABEL_FLAGS_LOCAL definition needs to be updated	August 2017
2.7A	1829 Label Protocol Section - Missing define for EFI_NVDIMM_LABEL_FLAGS_UPDATING	August 2017
2.7A	1823 Modifications to the examples of the PCI Option ROM image combinations	August 2017
2.7A	1822 UEFI 2.7 Organization chapter duplicated	August 2017
2.7A	1821 Modify the requirement to enable PCI Bus Mastering	August 2017
2.7A	1817 NVDIMM Label Protocol - SetCookie SerialNumber needs to be UINT32 NOT UINT64	August 2017
2.7A	1816 Clarification of Using HttpConfigData in HTTP protocol	August 2017
2.7A	1815 OpenProtocol() / EFI_ALREADY_STARTED should output existent Interface	August 2017
2.7A	1808 Clarification of using option 43 in PXE v2.1	August 2017
2.7	1779 Adjusting UEFI version to UEFI 2.7	April 2017
2.7	1771 BluetoothLE minor fix	April 2017
2.7	1762 UEFI UFS DEVICECONFIG Protocol	April 2017
2.7	1751 Update DNS Device Path	April 2017
2.7	1750 Add new data type to EFI Supplicant Protocol	April 2017
2.7	1745 NVDIMM Label Protocol	April 2017
2.7	1744 NVDIMMBlock Translation Table (BTT) Protocol {NewChapter}	April 2017
2.7	1730 HII Popup Protocol	April 2017
2.7	1726 Host and I/O defense	April 2017
2.7	1720 Have Partition driver publish addition information for MBR/GPT partition types.	April 2017
2.7	1719 Add EFI HTTP Boot Callback Protocol	April 2017
2.7	1718 Allow SetData to clear configuration in Ip4Config2/Ip6Config Protocol	April 2017
2.7	1716 Add BluetoothLE ECR	April 2017
2.7	1711 Firmware Error Record Update	April 2017
2.7	1707 Clarification of Private Authenticated Variables	April 2017
2.7	1701 Add wildcard support to RegisterKeyNotify	April 2017
2.7	1690 Reset Notification Protocol Update	April 2017
2.7	1689 Secure Boot with Externally Managed Configuration	April 2017
2.7	1685 Key Management Services (KMS) Protocol Enhancement	April 2017
2.7	1672 UEFI Variable Enhancements	April 2017
2.7	1654 New AIP Information block for wireless NIC	April 2017
2.7	1652 Add DNS device path node	April 2017
2.7	1647 UEFI binding for RISC-V	April 2017
2.7	1641 Simplify SecureBoot Revocation and Usage of VerifySignature	April 2017
2.7	1641 Simplify Secure Boot Revocation and Usage of VerifySignature	April 2017
2.7	1627 Support ASCII RegEx Patterns in EFI_REGULAR_EXPRESSION_PROTOCOL	April 2017
2.7	1627 EFI regular expression syntax type definitions	April 2017

continues on next page

Table 3 – continued from previous page

2.7	1623 New EFI_HTTP_STATUS_CODE enum for 308 Permanent Redirect	April 2017
2.7	1623 New EFI_HTTP_STATUS_CODE enum for 308 Permanent Redirect	April 2017
2.6B	1772 Clarify EFI_NOT_READY in Media State of AIP	April 2017
2.6B	1767 Incorrect structure definition for EFI_IFR_RESET_BUTTON_OP	April 2017
2.6B	1742 Clarify PK enrolling in user mode	April 2017
2.6B	1741 The memory map returned ByBS->GetMemoryMap() may- Contain impossible values.	April 2017
2.6B	1739 typos -Broken references link.	April 2017
2.6B	1729 Cleanup of ACPI 2.0 references in UEFI spec	April 2017
2.6B	1708 Typos in Image Decode and Image Ex Protocols	April 2017
2.6B	1700 Align ACPI descriptor definitions in PCI I/O and PCI Root- Bridge I/O	April 2017
2.6B	1698 Update to Mantis 1613 - GetNextVariable	April 2017
2.6B	1691 Remove/Deprecate SMM Communication ACPI Table	April 2017
2.6B	1682 HII Protocol StatusCodes	April 2017
2.6B	1678 Simplify the ACPI Table GUID declarations	April 2017
2.6B	1675 section 30.5.1 typo	April 2017
2.6B	1668 Duplicate GUID issue - mustChange the Image Decoder Pro- tocol GUID	April 2017
2.6B	1655 HTTP errata in Configure()	April 2017
2.6B	1653 Incorrect errorCode value in MTFTP6	April 2017
2.6B	1634 Update to the EFI_SIMPLE_TEXT_INPUT_PROTOCOL TPL restriction	April 2017
2.6B	1629 Errata in GetVariable description	April 2017
2.6B	1625 Clarification of HTTPBoot wire protocol “HTTPClient” VendorClass Option	April 2017
2.6B	1624 Fix spelling typo in EFI_HTTP_STATUS_CODE	April 2017
2.6B	1613 GetNextVariableName Errata	April 2017
2.6B	1612 ResetSystem Errata	April 2017
2.6B	1609 UEFI Errata - Address Security problems in the Pkcs7Verify Protocol	April 2017
2.6B	1608 Enhance EFI_IFR_NUMERIC (Step)	April 2017
2.6B	1586 Errors in appendix N for ARM ProcessorContext Informa- tion	April 2017
2.6B	1584 WIFI errata	April 2017
2.6B	1580 Correct some typos	April 2017
2.6B	1559 Clarify return value for NULL pointer in LocateProtocol() API	April 2017
2.6B	1557 secureBoot and auth variable errata	April 2017
2.6B	1556 HTTPv6Boot DHCP Options Errata	April 2017
2.6B	1555 USB Function port protocol errata	April 2017
2.6B	1554 fix to ecr 1539	April 2017
2.6B	1553 os recoveryBoot option errata	April 2017
2.6B	1551 EFIBluetoothConfiguration Protocol Errata	April 2017
2.6B	1550 Replace FTP4 dataCallback pointer-to-function-pointer with regular function pointer	April 2017
2.6A	SameContent as version 2.6, But with the Adobe “accessibility” feature activated so text-to-speech will work.	December 2016
2.6	1548 Clarify Boot procedure when file name is absent2.	January, 2016

continues on next page

Table 3 – continued from previous page

2.6	1547 Clarify requirements for setting the PK variable.	January, 2016
2.6	1544 DNS lookup API spelling	January, 2016
2.6	1543 ip4/6 Config policy errata/2.6 update	January, 2016
2.6	1542 UEFI 2.6 supplicant errata	January, 2016
2.6	1539 New EFI_HTTP_ERROR StatusCode	December, 2015
2.6	1538 UEFI TLS errata	December, 2015
2.6	1536 UEFI 2.6 Errata : IMAGE EX Protocol and EFI HII Image Decoder protocol Errata	December, 2015
2.6	1534 Editorial Comments against 2.6 Final Draft	December, 2015
2.6	1533 Bugs in the HTTP usage example	December, 2015
2.6	1523 Comments against 2.6 Draft	December, 2015
2.6	1522 AArch64 Bindings Alignment Bit errata	December, 2015
2.6	1521 Comment against UEFI.next draft - M1479	December, 2015
2.6	1519 Version for the next UEFI spec is...	December, 2015
2.6	1518 Comments against 2.6 Draft	December, 2015
2.6	1516 Editorial Comments against 2.6 Draft	December, 2015
2.6	1509 EFI_PLATFORM_TO_DRIVER_CONFIGURATION_PROTOCOL Response to unsupported ParameterTypeGuid	December, 2015
2.6	1508 Lack of flexibility and realism in exception levelChoice whenCalling runtime services	December, 2015
2.6	1507 Insufficient qualification of page attributes for AArch64	December, 2015
2.6	1502 PCI IO Define how to use the Address Translation Offset for systems that are not mapped 1:1	November, 2015
2.6	1501 Define the usage of the “Address Space Granularity” field is defined in the PCI Root IO	November, 2015
2.6	1496 Bad table reference in 13.2 EFI_PCI_ROOT_BRIDGE_IO_PROTOCOL.Configuration()	November, 2015
2.6	1494 Errata against UEFI 2.5 Properties Table	November, 2015
2.6	1493 Updates to the SD_MMC_PASS_THRU interface	November, 2015
2.6	1492 wireless macConnection protocol II errata	November, 2015
2.6	1491 supplicant errata	November, 2015
2.6	1480 Refine Progress description in EFI_KEYWORD_HANDLER_PROTOCOL	November, 2015
2.6	1479 UEFI Properties Table Clarification	November, 2015
2.6	1471 SD/eMMC PassThru Protocol update (follow up to mantis 1376)	November, 2015
2.6	1467 New API - EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL	November, 2015
2.6	1466 UEFI Ram disk protocol	November, 2015
2.6	1452 Minor edits to 0001409	November, 2015
2.6	1414 Generalisation of Communication method in Appendix O	November, 2015
2.6	1409 EFI HII ImageEX protocol and EFI HII Image Decoder protocols	November, 2015
2.6	1408 EFI HII Font EX protocol and EFI HII Font Glyph Generator protocols	November, 2015
2.6	1402 Add EFI_BROWSER_ACTION_SUBMITTED	November, 2015
2.6	1383 Adding an EraseBlocks() function to a new protocol	November, 2015
2.6	1376 SD/eMMC PassThru Protocol	November, 2015
2.6	1357 ARMCPER extensions	November, 2015
2.5A	1481 new networkConfig2 protocol data structure has a magic number	October 2015

continues on next page

Table 3 – continued from previous page

2.5A	1477 AllowCloseEvent toBeCalled within the Notification Function	October 2015
2.5A	1476 Update to Indicate thatCloseEvent UnregistersCorresponding Protocol Notification Registrations	October 2015
2.5A	1472 ATA Pass Thru Errata	October 2015
2.5A	1469 UNDI Errata - add more statistics	October 2015
2.5A	1468 Errata on UEFI Supplicant protocol	October 2015
2.5A	1451 Memory MapConsistency	October 2015
2.5A	1441 UEFI2.5A – UNDI ProtocolClarification	October 2015
2.5A	1426 UEFI 2.5 typo	October 2015
2.5A	1424 Incorrect link in Section 22.1 FMP GetImageInfo()	October 2015
2.5A	1421 Misc HTTP API typos	October 2015
2.5A	1420 Get NextHighMonotonicCountClarification	October 2015
2.5A	1419 Supplicant protocol using same GUID as TLS protocol	October 2015
2.5A	1418 Inconsistent issues in DNS	October 2015
2.5A	1417 Add HttpMethodMax to EFI_HTTP_METHOD enum	October 2015
2.5A	1410Clarifications in appendix O	October 2015
2.5A	1407 Networking errata - EFI_HTTP_STATUS typos	October 2015
2.5A	1405 Errata in table 271 in Appendix O	October 2015
2.5A	1399 Clarification for EFI_BROWSER_ACTION_ RE-QUEST_RECONNECT	October 2015
2.5A	1398 Errata update to the runtime GetVariable operation documentation	October 2015
2.5A	1388 Missed memory type fixes	October 2015
2.5A	1381 Remove informativeContent in 12.6.1	October 2015
2.5A	1365 7.4 Virtual Memory Services lists Section 2.3.2 through Section 2.3.4. incorrectly	October 2015
2.5A	1363 Short form URI device path	October 2015
2.5A	1209 UEFI networking APIChapter 2.6 requirements errors	October 2015
2.5A		October 2015
2.5	1364 Extend supplicant data type for EAP	April, 2015
2.5	1362 HTTPBoot typos/bugs	April, 2015
2.5	1360 Vendor Range for UEFI memory Types	April, 2015
2.5	1358 v2.5 amendment and v2.4 errata (missed implementation of Mantis 1089)	April, 2015
2.5	1353 SATA Device Path Node Errata	April, 2015
2.5	1352 Errata for 1263 and 1227	
2.5	1350 Keyword Strings Errata	April, 2015
2.5	1348 ERRATA - Section 10.12 EFI _ADAPTER_INFORMATION_PROTOCOLCustom Types	April, 2015
2.5	1347Boot Manager Policy Errata	April, 2015
2.5	1346 Mantis 1288 Errata	April, 2015
2.5	1345 EFI_USB2_HC_PROTOCOL Errata	April, 2015
2.5	1342 DNS6 - friendly amendment for reviewBy USWG	April, 2015
2.5	1341 DNS4 - friendly amendment toBe reviewedBy USWG	April, 2015
2.5	1339 Errata in section 7.2.3.2 Hardware Error Record Variables	April, 2015
2.5	1309 Disallow EFI_VARIABLE_AUTHENTICATION from SecureBoot Policy Variables	April, 2015
2.5	1308 Fix typo's found in the final/published UEFI 2.4 ErrataB spec	February, 2015
2.5	1304 Add IMA GE_UPDATABLE_VALID_WITH_VENDOR_CODE to FMPCheck image	February, 2015
2.5	1303 Update the UEFI version to reflect new revision	February, 2015

continues on next page

Table 3 – continued from previous page

2.5	1288 The Macro definitionConflict in EFI_SIMPLE_ TEXT_OUTPUT_PROTOCOL.SetAttribute() in UEFI 2.4B	February, 2015
2.5	1287 Errata: EFI Driver Supported EFI Version not matching the spec revision	February, 2015
2.5	1269Configuration Routing Protocol andConfiguration String Up- dates	February, 2015
2.5	1268 RAM Disk UEFI Device Path Node	February, 2015
2.5	1266 UEFI.Next Feature - IP_CONFIG2 Protocol	February, 2015
2.5	1263Customized Deployment of SecureBoot	February, 2015
2.5	1257Correct the typedef definitions for EFI_BOOT_SERVI CES/EFI_RUNTIME_SERVICES–Reiterate	February, 2015
2.5	1255 UFS Device Path Node Length	February, 2015
2.5	1254 SD Device Path	February, 2015
2.5	1251 EFI_REGULAR_EXPRESSION_PROTOCOL and EFI_IFR_MATCH2 HII op-code	February, 2015
2.5	1244 sections of the spec mis-arranged	February, 2015
2.5	1234 UEFI.Next feature - SmartCard edge protocol	February, 2015
2.5	1227 UEFI.Next feature - Platform recovery	February, 2015
2.5	1224 UEFI.Next - Adding support for No executable data areas	February, 2015
2.5	1223 UEFI.Next networking features -Chapter 2.6 requirements	February, 2015
2.5	1222 UEFI.Next feature -BMC/Service Processor Device Path	February, 2015
2.5	1221 UEFI.Next feature - REST Protocol	February, 2015
2.5	1220 UEFI.Next feature -Bluetooth	February, 2015
2.5	1219 UEFI.Next Feature - UEFI TLS API	February, 2015
2.5	1218 UEFI.Next feature - EAP2 Protocol	February, 2015
2.5	1217 UEFI.Next feature - WIFI support	February, 2015
2.5	1216 UEFI.next feature - DNS version 6	February, 2015
2.5	1215 UEFI.Next feature - DNS version 4	February, 2015
2.5	1214 UEFI.Next feature - HTTPBoot	February, 2015
2.5	1213 UEFI.Next feature - HTTP helper API	February, 2015
2.5	1212 UEFI.Next feature - HTTP API	February, 2015
2.5	1204 new UEFI USB Function I/O Protocol addition to the UEFI spec	February, 2015
2.5	1201 Exposing Memory Redundancy to OSPM	February, 2015
2.5	1199 Add NVM Express Pass Thru Protocol	February, 2015
2.5	1191 Add new SMBIOS3_TABLE_GUID in EFI_CONFIGURATION_TABLE	February, 2015
2.5	1186 AArch64BindingClarifications and errata	February, 2015
2.5	1183 New Protocol with 2 Function for PKCS7 Signature Verifi- cation Services	February, 2015
2.5	1174 errata - Error in EFI_IFR_PASSWORD logic flowchart	February, 2015
2.5	1167 Persistent Memory Type support	February, 2015
2.5	1166 hash 2 protocol errata	February, 2015
2.5	1163 InlineCryptographic Interface Protocol proposal	February, 2015
2.5	1159 Proposal for System Prep Applications	February, 2015
2.5	1158 errata -Boot managerClarification	February, 2015
2.5	1147–REDACT	February, 2015
2.5	1121 IPV6 support from UNDI	February, 2015
2.5	1109 SmartCard Reader	February, 2015
2.5	1103 Longer term NewCPEP Memory Section	February, 2015
2.5	1091Clarification of handle to host FMP	February, 2015

continues on next page

Table 3 – continued from previous page

2.5	1090 ESRT: EFI System Resource Table and Component firmware updates	February, 2015
2.5	1071 New EFI_HASH2_PROTOCOL	February, 2015
2.4C	1308 Fix typo's found in the final/published UEFI 2.4 ErrataB spec	January 2015
2.4C	1287 Errata: EFI Driver Supported EFI Version not matching the spec revision	January 2015
2.4C	1257 Correct the typedef definitions for EFI_BOOT_SERVICES/EFI_RUNTIME_SERVICES	January 2015
2.4C	1244 sections of the spec misarranged	January 2015
2.4C	1211 EFI_LOAD_OPTION Definition	January 2015
2.4C	1209 Errata - UEFI networking API Chapter 2.6 requirements	January 2015
2.4C	1205 Errata for Hii Set item	January 2015
2.4C	1200 Universal Flash Storage (UFS) Device Path	January 2015
2.4C	1198 EFI_ATA_PASS_THRU_PROTOCOL Clarification	January 2015
2.4C	1194 Add EFI_IFR_FLAG_RECONNECT_REQUIRED	January 2015
2.4C	1192 Cleanup GUID formatting issues	January 2015
2.4C	1186 AArch64 Binding Clarifications and errata	January 2015
2.4C	1185 errata - tcp api	January 2015
2.4C	1184 errata - snp mode Clarification	January 2015
2.4C	1182 Errata - UEFI URI Device path issue	January 2015
2.4C	1174 errata - Error in EFI_IFR_PASSWORD logic flowchart	January 2015
2.4C	1173 EFI_IFR_NUMERIC Errata	July 11, 2014
2.4C	1172 EfiACPI Memory NVS definition missing S4	July 11, 2014
2.4C	1170 Errata pxeBc api Clarification	July 11, 2014
2.4C	1169 Errata - volatile networking variable Cleanup	July 11, 2014
2.4C	1168 MTFTP Errata	July 11, 2014
2.4C	1165 Option rom layout errata	July 11, 2014
2.4C	1162 Typo in ReinstallProtocolInterface() EFI 1.10 Extension section	July 11, 2014
2.4C	1150 Missing LineBreakCharacter (HII Errata)	July 11, 2014
2.4C	1147 EFI_USB2_H_C_PROTOCOL.AsyncInterruptTransfer() Errata	July 11, 2014
2.4C	1141 UEFI errata - ia32/x64 vector register management	July 11, 2014
2.4C	1140 UEFI Errata - image execution info table	July 11, 2014
2.4C	1139 UEFI Errata on the storage security Command protocol	July 11, 2014
2.4C	1066 Errata—reference to missing table (90) removed	July 11, 2014
2.4C	1043 Ability to refresh the entire form [newContent]	July 11, 2014
2.4C	1042 Add Browser Action Request “reconnect”	July 11, 2014
2.4B	1146 Typos and Broken links	April 17, 2014
2.4B	1137 Typographic errors in the 2.4 ErrataB draft	April 16, 2014
2.4B	1128 URI device path node redux—supersedes (defunct) 1119	April 4, 2014
2.4B	1127 USB Errata - unnecessary restriction on UEFI interrupt transfer types	March 27, 2014
2.4B	1124 Adding text description for NVMe device node	March 27, 2014
2.4B	1122 Correct misleading language in the UEFI 2.4a specification about the EFI_ADAPTER_INFORMATION_PROTOCOL.EFI_ADAPTER_INFO_GET_SUPPORTED_TYPES function	March 27, 2014
2.4B	1120 Make time stamp handling Consistent around all of the networking API's	March 27, 2014
2.4B	1118 Network Performance Enhancements Concerning Volatile Variables	March 27, 2014

continues on next page

Table 3 – continued from previous page

2.4B	1115Clarification on the usage of XMM/FPU instructions from within a UEFI Runtime Service on an x64 processor	March 27, 2014
2.4B	1111 Errors in DisconnectController() returnCode descriptions	March 27, 2014
2.4B	1101 Errata – ReinstallProtocolInterface	March 27, 2014
2.4B	1092Clarification to PCI Option ROM Driver Loading Description	March 27, 2014
2.4B	1085 Error–added in missing text approved for 2.4A	April 17, 2014
2.4B	1014 HIIConfig Access Protocol Errata	April 3, 2014
2.4 A	1089 Short-termCER Memory Section errata	Nov. 14, 2013
2.4 A	1088 Add revision #define to EFI_FILE_PROTOCOL	Nov. 6, 2013
2.4 A	1085 Issues with Interactive password	Nov.14, 2013
2.4 A	1082 Mistake in 2.3.5.1 / 2.3.6.2 Handoff State	Nov. 6, 2013
2.4 A	1081 Update Install Table protocol to deal with duplicate tables	Nov. 6, 2013
2.4 A	1079 UEFI 2.4: Remove repetitive “the” (typo)	Nov. 6, 2013
2.4 A	1078 Adjust some text for handling EFI_BROWSER_ACTION_CHANGING	Nov. 6, 2013
2.4 A	1077 Fix wording in EVT_SIGNAL_EXIT_BOOT_SERVICES	Nov. 6, 2013
2.4 A	1076 typo in UEFI v2.3.1d and v2.4	Nov. 6, 2013
2.4 A	1075Clarifications to Table 88. Device Node Table (Device Node to TextConversion)	Nov. 6, 2013
2.4 A	1074 AddClarifications on DMA requirements for PCI_IO	Nov. 6, 2013
2.4 A	1073 Add requirement for EFI_USB_IO_PROTOCOL	Nov. 6, 2013
2.4 A	1066 Errata - ISCSI IPV6 Root PathClarification	Nov. 6, 2013
2.4 A	1064 AIP Errata	Nov. 6, 2013
2.4 A	1063Correction to GPT expression for SizeofPartitionEntry	Nov. 6, 2013
2.4 A	1062 EFI_CERT_X509_GUID does not specify theCertificate encoding	Nov. 6, 2013
2.4 A	1061 UEFI 2.4 section 2.6.2 and 2.6.3 don’t use protocol hyperlinksConsistently	Nov. 6, 2013
2.4 A	1060 SlightClarification to FMP Authentication Requirments	Nov. 6, 2013
2.4 A	1059Clarification of a return statusCode of HASH protocol	Nov. 6, 2013
2.4 A	1058Correct mistake in the system table revision	Nov. 6, 2013
2.4 A	1056 text modification to definition of EFI_FIRMWARE_IMAGE_DESCRIPTOR_VERSION 2	Nov. 6, 2013
2.4 A	1055 Disk IO 2 errata	Nov. 6, 2013
2.4 A	1054 Deprecate 6 Hash Algorithms with inconsistent usage	Nov. 6, 2013
2.4 A	1053 Reduce Name space ofCapsule Result variable to increase performance	Nov. 6, 2013
2.4 A	1035 PCI Option ROM Errata (five figures)	Nov. 6, 2013
2.4	997 Driver Health Protocol errorCodes	April 25, 2013
2.4	993 (original ticket–supersededBy 1026)	
2.4	992 Adapter Information Protocol (AIP)	April 25, 2013
2.4	991 Greater than 256 NICs support on UNDI	April 25, 2013
2.4	968 HII Forms op-code for displaying a warning message	April 25, 2013
2.4	966 Spec typos	April 25, 2013
2.4	964 Disk IO 2 Protocol to support Async IO	April 25, 2013
2.4	963 Add new device path node NVM Express devices	April 25, 2013
2.4	956 Require network drivers to return EFI_NO_MEDIA	April 25, 2013
2.4	946 ForbidCreation of non-spec variables in EFI_GLOBAL_VARIABLE namespace	April 25, 2013
2.4	920 Add a variable for indicating out ofBand key modification	April 25, 2013

continues on next page

Table 3 – continued from previous page

2.4	905 Need more granularity in EFI_RESET_TYPE to support platform specific resets	April 25, 2013
2.4	1052 UEFI 2.4 Draft April 25th -Corrections to ARM sections	May 16, 2013
2.4	1050 2.4 Draft April 25 has missing text for ECR 1009	May 16, 2013
2.4	1049 2.4 Draft April 25 has missing text for ECR 1008	May 16, 2013
2.4	1048Comment against UEFI 2.4 - NVMe related	May 16, 2013
2.4	1047Comment on Feb 25th draft - fix alignment issue	May 16, 2013
2.4	1045 PCI OpROM Device ListChanges to section 14.2	June 28, 2013
2.4	1044Corrections to Mantis 1015, Interruptible driver diagnostics	May 16, 2013
2.4	1037 Add 2.4 to the system table version	May 16, 2013
2.4	1036Comments on April 25 Draft	May 16, 2013
2.4	1033 HiiConfigAccess->ExtractConfig StatusCodes Errata	May 16, 2013
2.4	1032 HiiConfigRouting->ExtractConfig StatusCodes Errata	May 16, 2013
2.4	1031 NVMe subtypeConflict errata	April 25, 2013
2.4	1029 Method for delivery ofCapsule on disk; Method for reportingCapsule processing status	April 25, 2013
2.4	1026 (supersedes 993) Update to the AArch64 proposedBindingChange	April 25, 2013
2.4	1024Clarification to the NVMe Device Path text descriptions	April 25, 2013
2.4	1023 Definition ofCapsule format to deliver update image to firmware management protocol	April 25, 2013
2.4	1022 adapter information protocol for NIC iSCSI and FCoEBoot-Capabilities andCurrentBooot Mode.	April 25, 2013
2.4	1017 AIP Instance - FCOE SAN MAC Address	April 25, 2013
2.4	1016 AIP Instance - Image Update	April 25, 2013
2.4	1015 Interruptible driver diagnostics	April 25, 2013
2.4	1009 Enable hashes ofCertificates toBe used for revocation, and timestamp support	April 25, 2013
2.4	1008 New Random Number Generator / Entropy Protocol	April 25, 2013
2.4	1007Create a new Security Technologies section to avoidBlurring with SecureBoot	April 25, 2013
2.4	1002 Timestamp Protocol	April 25, 2013
2.3.1D	996 UEFI 2.0 version number still in the 2.3.1C spec	April 3, 2013
2.3.1D	995CSA linkChange	April 3, 2013
2.3.1D	994 Spec typos	April 3, 2013
2.3.1D	990 EFI_ATA_PASS_THRU need oneClarification if it supports ATAPI device	April 3, 2013
2.3.1D	989Clarify hot-remove responsibility of aBus Driver	April 3, 2013
2.3.1D	988 EFI_BLOCK_IO2_PROTOCOLBlocksChild from stopping while doing non-blocking I/O	April 3, 2013
2.3.1D	987 EFI_BLOCK_IO2_PROTOCOL has aCopy pasteBug describing the Token Parameter	April 3, 2013
2.3.1D	980 Errata on SNP Media detect	April 3, 2013
2.3.1D	978 Error Retun IndicatesCapsule requiresBoot Services	April 3, 2013
2.3.1D	977 missing statement	April 3, 2013
2.3.1D	976BrowserCallback text update to description	April 3, 2013
2.3.1D	975 UNDI errata to add missing memory type definitions	April 3, 2013
2.3.1D	974 UNDI IncorrectCPB function names ECR	April 3, 2013
2.3.1D	973 UNDI Mem_Map()Clarification	April 3, 2013
2.3.1D	972 ISCSI DHCP6Boot	April 3, 2013
2.3.1D	971 typo	April 3, 2013
2.3.1D	970 Typo section 28.3.8.3.41 EFI_IFR_MODAL_TAG	April 3, 2013

continues on next page

Table 3 – continued from previous page

2.3.1D	965 File IO Async extension	April 3, 2013
2.3.1D	962 Remove 2.3 table revision number	April 3, 2013
2.3.1D	960 Typo in netboot6 description	April 3, 2013
2.3.1D	959 InstallAcpiTable() does not say what to do when an attempt is made to install a duplicate table	April 3, 2013
2.3.1D	955 Clearing The Platform Key Errata	April 3, 2013
2.3.1D	954 LoadImage Errata	April 3, 2013
2.3.1D	953 Need text definitions for Device Path Media Type Subtype 6/7	April 3, 2013
2.3.1D	952 Clarification of requirements to update timestamp associated with authenticated variable	April 3, 2013
2.3.1D	950 Indeterminate Behavior for attribute modifications may Cause security issues	April 3, 2013
2.3.1D	949 PCI IO.GetBarAttributes needs adjustment - - Address Space Granularity field	April 3, 2013
2.3.1D	944 Errata - Replace RFC reference	April 3, 2013
2.3.1D	943 Errata - Proposed updates to required interfaces in Chapter 2.6	April 3, 2013
2.3.1D	942 ExportConfig() description does not make sense	April 3, 2013
2.3.1D	941 Add OEM StatusCode ranges to EFI StatusCode Ranges Table	April 3, 2013
2.3.1D	938 InstallMultipleProtocolInterface() is missing StatusCode Returned values	April 3, 2013
2.3.1D	935 Clarify Chaining requirements with regards to the Platform Key	April 3, 2013
2.3.1D	934 Missing Figures and typos	April 3, 2013
2.3.1D	930 Clarify usage of EFI Variable Varstores in HII	April 3, 2013
2.3.1D	928 Best Matching Language algorithm	April 3, 2013
2.3.1D	926 UEFI Image Verification Clarification	April 3, 2013
2.3.1D	924 New ErrorCode to handle reporting of IPV4 duplicate address detection	April 3, 2013
2.3.1D	1021 ATA_PASS_THRU on ATAPI device handle.	April 3, 2013
2.3.1D	1020 Clarify HII variable store definitions.	April 3, 2013
2.3.1D	1019 Alignment Requirements Clarification	April 3, 2013
2.3.1D	1018 HII Font Errata	April 3, 2013
2.3.1D	1013 HII Errata	April 3, 2013
2.3.1D	1012 Touchup to text of GPT	April 3, 2013
2.3.1D	1011 Typo regarding Debug Port in UEFI Spec	April 3, 2013
2.3.1D	1003 Missing “(” in section 11.7	April 3, 2013
2.3.1D	1000 Clarification to the IFR_REF4 opcode	April 3, 2013
2.3.1C	921 Length of IPv6 Device Path is incorrect	June 13, 2012
2.3.1C	917 UNDI drive does not need to be initialized as runtime driver	June 13, 2012
2.3.1C	915 For x64, Change Floating Point Default Configuration to Double-Extended Precision	June 13, 2012
2.3.1C	914 Error Descriptor Reset Flag Clarification	June 13, 2012
2.3.1C	913 Enum definition does not match what our Current Compilers implement.	June 13, 2012
2.3.1C	912 UEFI 2.3.1 Type	June 13, 2012
2.3.1C	909 Update to return Codes for AllocatePool / AllocatePages	June 13, 2012
2.3.1C	907 iSCSI Device Path error	June 13, 2012
2.3.1C	882 Indications Variable - OS/FW feature & Capability Communication	June 13, 2012
2.3.1C	882 Indications Variable - OS/FW feature & Capability Communication	June 13, 2012
2.3.1C	874 Provide a mechanism for providing keys in setup mode	June 13, 2012

continues on next page

Table 3 – continued from previous page

2.3.1C	831 PXEBootCSA Type definitionCleanup	June 13, 2012
2.3.1B	896 StartImage andConnectController returnCodes	April 10, 2012
2.3.1B	893 SMMCommunication ACPI Table Update	April 10, 2012
2.3.1B	891Component Name Protocol References	April 10, 2012
2.3.1B	890 DriveConfiguration Protocol Phantom.	April 10, 2012
2.3.1B	888 typo in EFI_USB_HC Protocol	April 10, 2012
2.3.1B	887 union is declared twice in same section	April 10, 2012
2.3.1B	885 Errata in the GPT Table structureComment	April 10, 2012
2.3.1B	884 EFI_BOOT_KEY_DATA relies on implementation-definedBehavior	April 10, 2012
2.3.1B	881 netboot6 - multicast versus unicast	April 10, 2012
2.3.1B	880 netboot6Clarification/errata	April 10, 2012
2.3.1B	879 Reference to unsupported specification in SCSIChapter (14.1)	April 10, 2012
2.3.1B	878 Updated HII “Selected Form”Behaviors to Reflect NewCall-back Results	April 10, 2012
2.3.1B	877 TableChecksum updateBy the A CPI_TABLE_PROTOCOL.InstallAcpiTable	April 10, 2012
2.3.1B	876 ToClarify EDID_OVERRIDE attribute definitions and expected operations	April 10, 2012
2.3.1B	873 Section 9.3.7 incorrectly assumes that all uses ofBBS device paths are non-UEFI	April 10, 2012
2.3.1B	872Change to SIMPLE_TEXT_INPUT_EX_PROTOCOL.RegisterKeyNotify/UnregisterKeyNotify	April 10, 2012
2.3.1B	871 Typo in InstallMultipleProtocolInterfaces	April 10, 2012
2.3.1B	870Clarify FrameBufferSize definition under EFI_GRAPHICS_OUTPUT_PROTOCOL_MODE struct	April 10, 2012
2.3.1B	869 Reference to FIPS 180 inChapter 27.3 is obsolete and incorrect	April 10, 2012
2.3.1B	867Clarify requirment for use of EFI_HASH_SERVICE_BINDING_PROTOCOL	April 10, 2012
2.3.1B	866 PK, KEK, db, dbx relationsClarification	April 10, 2012
2.3.1B	865 Modify Protective MBRBootIndicator definition	April 10, 2012
2.3.1B	864 Typo in Question-Level Validation section	April 10, 2012
2.3.1B	863 Attributes of the Globally Defined Variables	April 10, 2012
2.3.1B	862 User identity typo	April 10, 2012
2.3.1B	861 Globally Defined Variables Errata	April 10, 2012
2.3.1B	858 Superfluous and incorrect image hash description	April 10, 2012
2.3.1B	857 Absolute pointer typo	April 10, 2012
2.3.1B	855Clarification of UEFI driver signing/Code definitions	April 10, 2012
2.3.1B	853 The EFI_HASH_PROTOCOL.Hash() description needsClarification on padding responsibilities	April 10, 2012
2.3.1B	852 Various EFI_IFR_REFRESH_ID errata.	April 10, 2012
2.3.1B	851 For EFI_IFR_REFRESH opcode,Clarify RefreshInterval = 0 means no auto-refresh.	April 10, 2012
2.3.1B	850Clarification of responsibility for array allocation in EFI_HASH_PROTOCOL	April 10, 2012
2.3.1B	849 IFR EFI_IFR_MODAL_TAG_OP is also valid under EFI_IFR_FORM_MAP_OP	April 10, 2012
2.3.1B	848Clarification of semantics of SecureBoot variable	April 10, 2012
2.3.1B	847 When enrolling a PK, the platform shall not require a reboot to leave SetupMode	April 10, 2012
2.3.1B	845 EFI_SCSI_PASS_THRU_PROTOCOL replacement	April 10, 2012

continues on next page

Table 3 – continued from previous page

2.3.1B	842 Text to explain how the UEFI revision is referred	April 10, 2012
2.3.1B	836 StructureComment for EFI_IFR_TYPE_VALUE references unknown value type.	April 10, 2012
2.3.1B	828 Network Driver Options	April 10, 2012
2.3.1B	826Comments against Mantis 790	April 10, 2012
2.3.1B	825 DMTF SMCLP errata	April 10, 2012
2.3.1B	819 Mantis 715 was not fully implemented	April 10, 2012
2.3.1B	812 Errata – DUID-UUID usage	April 10, 2012
2.3.1B	809 Errata – Messaging Device PathClarification	April 10, 2012
2.3.1B	808 Errata –Boot File URL	April 10, 2012
2.3.1B	807 Give specific TPL rules to Stall()Boot services	April 10, 2012
2.3.1B	771 SHA1 and MD5 references	April 10, 2012
2.3.1A	MinorCorrections in toes to tickets 772, 785, 794, 804, also for- formattingCorrection for _WIN_CERTIFICATE_UEFI_GUID type- def’s parameters	September 7, 2011
2.3.1A	820 Driver Health Needs to have Mantis 0000169 implemented	August 17, 2011
2.3.1A	819 ECR715 was not fully implemented	August 17, 2011
2.3.1A	806 Text update to Driver Health Description -Clarify role of user interaction	August 17, 2011
2.3.1A	805Correct Wrong Palette Information in 28.3.7.2.3 example	August 17, 2011
2.3.1A	804ClarifyConstraints and alternatives when enrolling PK, KeK, db or dbx keys	August 17, 2011
2.3.1A	803 Fix AcpiExp device node text description.	August 17, 2011
2.3.1A	801ClarifyIFR Opcode Summary and Description #4	August 17, 2011
2.3.1A	800Clarify IFR Opcode Summary and Description #3	August 17, 2011
2.3.1A	797Clarify IFR Opcode Summary and Description #2	August 17, 2011
2.3.1A	796Clarify IFR Opcode Summary and Description #1	August 17, 2011
2.3.1A	795 Typo in ReadKeyStrokeEx()	August 17, 2011
2.3.1A	794 Incomplete text describingClearing of Platform Key	August 17, 2011
2.3.1A	793 Inconsistent wording about RemainingDevicePath	August 17, 2011
2.3.1A	790 Add warning to ReadKeyStrokeEx for partial key press	August 17, 2011
2.3.1A	789Clarify HII opcode definition	August 17, 2011
2.3.1A	788 SasEx entry in Table 86-Device Node TableContains optional Reserved entry that does not exist in device path	August 17, 2011
2.3.1A	786 PCI I/O Dual AddressCycle attributeClarification	August 17, 2011
2.3.1A	785 Allowing more general use of UEFI 2.3.1 Variable time-based authentication	August 17, 2011
2.3.1A	780 Errata in returnCode descriptions	August 17, 2011
2.3.1A	778 EFI_HI I_CONFIG_ACCESS_PROTOCOL.CallBack() Er- rata	August 17, 2011
2.3.1A	777 Specified signature sizes incorrect in Section 27.6.1	August 17, 2011
2.3.1A	776 Clarifycomputation of EFI_VARIABLE_ AUTHENTICA- TION_2 hash value	August 17, 2011
2.3.1A	774 Define EFI_BLOCK_IO_PROTOCOL_REVISION3	August 17, 2011
2.3.1A	773Clarify the value for opcode EFI_IFR_REFRESH_ID_OP	August 17, 2011
2.3.1A	772 Definition of EFI_IMAGE_SECURITY_DATABAE_GUID incorrect	August 17, 2011
2.3.1A	770 Remove references to UEFI 2.1 spec	August 17, 2011
2.3.1A	767 The ReadBlocks function forBlockIO andBlockIO2 need syn- chronization	August 17, 2011
2.3.1A	212 (revisit) final sentence section 28.2.15 missing final words.	April 21, 2011

continues on next page

Table 3 – continued from previous page

2.3.1	765 ECR to limit the hash and encryption algorithms used with PKSCertificates	April 5, 2011
2.3.1	762 DevicePath in the Image Execution Information Table.	April 5, 2011
2.3.1	761 Table 195. Information for Types of Storage	April 5, 2011
2.3.1	760 SuggestedChanges to 2.3.1 final draft spec	April 5, 2011
2.3.1	759 UEFI Errata - wincerts for rest of hash algorithms	April 5, 2011
2.3.1	755 Errata in Legacy MBR table and Legacy MBR GUID	April 5, 2011
2.3.1	754 USB timeout parameter mismatch.	April 5, 2011
2.3.1	751 Fix USB HC2 erroneous references to IsSlowDevice	March 11, 2011
2.3.1	750 Fix section 27.2.5 “related definitions” re: RSA public key exponent	March 11, 2011
2.3.1	749 Fix Table 10 (Global Variables) WithCorrect Attributes	March 11, 2011
2.3.1	748Clarify Standard GUID Text Representation	March 11, 2011
2.3.1	744 ProcessorContext information structure definition notClear	March 11, 2011
2.3.1	741 Errata:Corrected text for section 7.2.1.4 step 7	March 11, 2011
2.3.1	740 Errata: signatureheadersize inconsistencyCorrections	April 6, 2011
2.3.1	736 Insert SMMCommunication ACPI Table and related data structures to the UEFI Specification	April 5, 2011
2.3.1	735Clarification on Tape Header Format	March 11, 2011
2.3.1	734 SecureBoot variable	April 5, 2011
2.3.1	733 Errata: 27.6.1 signatureheadersize definition	March 11, 2011
2.3.1	732 Amendment to Mantis 711: section 7.2.1.6	March 11, 2011
2.3.1	729 Errata:Clarification of Microsoft references in appendix Q	March 11, 2011
2.3.1	728 Netboot 6 errata - DUID-UUID	March 11, 2011
2.3.1	727 Errata on returnCode for User Info Identity policy record	March 11, 2011
2.3.1	726 Errata/clean-up of EFI_DHCP4_TRANSMIT_RECEIVE_TOKEN definition	March 11, 2011
2.3.1	724 SetVariable Update 2	March 11, 2011
2.3.1	723 User Identification (UID) Errata – EFI User Manager Notify & EnrollClarification	April 5, 2011
2.3.1	722 User Identification (UID) Errata –Credential Provider EnrollClarification	April 5, 2011
2.3.1	721 User Identification (UID) Errata – SetInfoClarification	March 11, 2011
2.3.1	720 User Identification (UID) Errata –Credential Provider EnrollClarification	March 11, 2011
2.3.1	716 EFI_EXT_SCSI_PASS_THRU_PROTOCOL.GetNextTarget() IN OUT parameter Target input value shallBe 0xFFs	March 11, 2011
2.3.1	715CPER Record and section fieldClarification	March 11, 2011
2.3.1	713 Remove the errata revision from the EFI_IFR_VERSION format.	March 11, 2011
2.3.1	711 SetVariable Update	March 11, 2011
2.3.1	709 NewCallback() Action Requests Related To Individual Forms.	Feb. 3, 2011
2.3.1	708 Errata (non-blockingBLOCK IO)	April 5, 2011
2.3.1	707 Errata revision in the EFI_IFR_VERSION format	Feb. 3, 2011
2.3.1	705 REPC signature definition stillConfusing	Feb. 3, 2011
2.3.1	704 Unload() definition is wrong	Feb. 3, 2011
2.3.1	702Clarifications on Variable Storage for Questions	Feb. 3, 2011
2.3.1	696 Update System Table with this new #define for EFI_SYSTEM_TABLE_REVISION	Feb. 3, 2011
2.3.1	695 Add Port Ownership probing	Feb. 3, 2011
2.3.1	687 Update System Table with this new #define for 2.3.1	Jan. 17, 2011

continues on next page

Table 3 – continued from previous page

2.3.1	686 HII -Clarify FormsBrowser ‘standard’ user interactions.	Feb. 3, 2011
2.3.1	685 HII - New op-code to enable event initiated refresh ofBrowser-Context data	Feb. 3, 2011
2.3.1	682 [UCST] Modal Form	Feb. 3, 2011
2.3.1	681 Typo: Pg. 56	Jan. 17, 2011
2.3.1	680 Netboot6 handleClarification	Jan. 17, 2011
2.3.1	679 UEFI Authenticated Variable & Signature Database Updates	Jan. 17, 2011
2.3.1	678 Section 27.6.2: Imagehash reference needs toBe removed	Jan. 17, 2011
2.3.1	677 Section 27.2.5 & 27.6.1: Typo in X509 Signature Type	Jan. 17, 2011
2.3.1	674 Section 3.2: Missing variable type for SetupMode variable	Jan. 17, 2011
2.3.1	671 Errata: USB device path example is incorrect	Jan. 17, 2011
2.3.1	668 LUN implementations are notConsistent	Feb. 3, 2011
2.3.1	661 USB 3.0 Updates	Oct. 29, 2010
2.3.1	645 Non-blocking interface forBLOCK oriented devices (BLOCK_IO_EX transition toBLOCK_IO_2)	Oct. 29, 2010
2.3.1	634 FormsBrowser DefaultBehavior	Jan. 17, 2011
2.3.1	634 FormsBrowser DefaultBehavior	Oct. 29, 2010
2.3.1	616 Security ProtocolCommand to support encrypted HDD	Jan. 17, 2011
2.3.1	616 Security ProtocolCommand to support encrypted HDD	Oct. 29, 2010
2.3.1	612 UEFI system Partition FAT32 data Region Alignment	Oct. 29, 2010
2.3.1	484 Key Management Service Protocol	Oct. 28, 2010
2.3.1	484 Key Management Service (KMS) Protocol	Oct. 29, 2010
2.3.1	478 (REVISIT) Update to ALTCFG references	March 11, 2011
2.3 D	667Clarification to the UEFIConfiguration Table definition	Oct. 28, 2010
2.3 D	664 Appendix update for IPV6 networkBoot	Oct. 28, 2010
2.3 D	663 Update ARM PlatformBinding to allow OS loader to assume unaligned access support is enabled	Nov. 10, 2010
2.3 D	662 ARM ABI errata	Oct. 28, 2010
2.3 D	659Clarify section length definition in the error record	Oct. 28, 2010
2.3 D	653 Errata to the Appendix N (Common Platform Error Record)	Oct. 28, 2010
2.3 D	652Clarification to the TimeZone value usage	Oct. 28, 2010
2.3 D	651 update to IPSec for tunnel mode support	Oct. 28, 2010
2.3 D	650 networking support errata	Oct. 28, 2010
2.3 D	638 Add facility for dynamic IFR dynamicCross-references	Oct. 28, 2010
2.3 D	538 IPV6 PXE	Oct. 28, 2010
2.3C	640 String ReferenceCleanup	July 14, 2010
2.3C	639Callback() does not describe FORM_OPEN/FORM_CLOSEBehavior	July 14, 2010
2.3C	637Clarification for Date/Time Question usage in IFR expressions.	July 14, 2010
2.3C	636 Mistaken Reference to “Date” inside ofBoolean question description	July 14, 2010
2.3C	635 Missing GUID label forConfig Access protocol	July 14, 2010
2.3C	633 Explicitly Specify ACPI Table Signature Format	July 14, 2010
2.3C	632ClarifyBlock IO ReadBlocks and WriteBlocks functions handling of media stateChange events	July 14, 2010
2.3C	625 Minor typo in surrogateCharacter description section	July 14, 2010
2.3C	622 Identify() function errata	July 14, 2010
2.3C	621 Typos in an EFI_HII_CONFIG_ACCESS_ PROTOCOL.Callback() member	July 14, 2010
2.3C	620Carification of need for Path MTU support for IPV4 and IPV6	July 14, 2010
2.3C	613 PAUSE Key	July 14, 2010

continues on next page

Table 3 – continued from previous page

2.3C	611 LanguageCorrection requested for InstallProtocolInterface() and InstallConfigurationTable(), Ref# 583	July 14, 2010
2.3C	610 RSA data structureClarification	July 14, 2010
2.3C	609 StartImage returnCode update	July 14, 2010
2.3C	583 How do we know an EFI_HANDLE is Valid/Invalid	July 14, 2010
2.3C	508 Update networking references, incl ipv6	July 14, 2010
2.3B	608 more media detectClean-up	Feb. 24, 2010
2.3B	605Clarify user identity Find API	Feb. 24, 2010
2.3B	601 UNDI update as part of media detectChanges	Feb. 24, 2010
2.3B	600 Update toConfigAccess/ConfigRouting	Feb. 24, 2010
2.3B	598 ARP is only an IPV4Concept.	Feb. 24, 2010
2.3B	590 Media detectClean-up	Feb. 24, 2010
2.3B	589 Device path representation of IPv4/v6 text	Feb. 24, 2010
2.3B	588 UEFI User Identity - ReturnCodes	Feb. 24, 2010
2.3B	587 UEFI User Identity - NamingConsistency	Feb. 24, 2010
2.3B	586Clarification of PXE2.1 specification for IPV4 interoperability issues	Feb. 24, 2010
2.3B	585 Errata to EFI_IFR_SET op-code	Feb. 24, 2010
2.3B	584 EFI_PXE_BASE_CODE_DHCPV6_PACKET missing for pxeBc protocol	Feb. 24, 2010
2.3B	583 How do we know an EFI_HANDLE is Valid/Invalid	Feb. 24, 2010
2.3B	580 ACPI_SUPPORT_PROTOCOLClarifications related to FADT and the DSDT/FACS	Dec. 15, 2009
2.3B	578 ATA Passthrough updates / questions	Dec. 15, 2009
2.3B	577Clarifications on the user identity protocol	Dec. 15, 2009
2.3B	576Clarifications in the Routing Protocol	Dec. 15, 2009
2.3B	575 Machine hand-off/MP state modification	Feb. 24, 2010
2.3B	574 Add an “OPTIONAL” tag to a parameter in NewPackageList	Dec. 15, 2009
2.3B	573 EFI_DESCRIPTION_STRING and EFI_DESCRIPTION_BUNDLE adjustments	Feb. 24, 2010
2.3B	572 EFI_IFR_SECURITY shouldBe EFI_IFR_SECURITY_OP in Table 194	Dec. 15, 2009
2.3B	568 ATA_STATUS_BLOCK name errata	Dec. 15, 2009
2.3B	567 Various miscellaneous typos/updates	Feb. 24, 2010
2.3B	566 Minor update to HII->NewString function description	Dec. 15, 2009
2.3B	560Correct erroneous example in ExtractConfig()	Dec. 15, 2009
2.3B	559 Extraneous “default” tag in EFI_IFR_SECURITY grammar	Dec. 15, 2009
2.3B	558Clarify VLANConfig publication requirements	Dec. 15, 2009
2.3B	557Corrected Image Execution Information omission & ambiguity	Dec. 15, 2009
2.3B	556 additional IPsec errata/issues	Dec. 15, 2009
2.3B	549Binary prefixChange	Dec. 15, 2009
2.3B	547Clean-Up In HII Sections	Dec. 15, 2009
2.3B	546 typo in GOP definiton	Dec. 15, 2009
2.3B	545 Action parameter of the EFI_HI I_CONFIG_ACCESS_PROTOCOL.CallBack()	Dec. 15, 2009
2.3B	542 Device Path DescriptionChanges	Dec. 15, 2009
2.3B	540 Register name usage	Dec. 15, 2009
2.3B	539CHAP node fix for iSCSI	Dec. 15, 2009
2.3B	537 Add missing ACPI ADR Device Path Representation	Dec. 15, 2009
2.3B	536 IPsec errata	Dec. 15, 2009
2.3B	534 Size of Partition Entry restriction	Dec. 15, 2009

continues on next page

Table 3 – continued from previous page

2.3B	533 GPT editorialCleanup	Dec. 15, 2009
2.3B	532 “LegacyBIOSBootable” GPT attribute	Dec. 15, 2009
2.3B	531Clarify HII Variable Storage	Dec. 15, 2009
2.3B	519 AddConsole table (chapt 11) for EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL	Dec. 15, 2009
2.3B	518 Typos in the UEFI2.3 specification	Feb. 24, 2010
2.3B	515 Authenticated VariablesClarification	Feb. 24, 2010
2.3B	514 HIIConfiguration String SyntaxClarification	Feb. 24, 2010
2.3B	507Clarify ACPI Protocol’s position onChecksums	Dec. 15, 2009
2.3B	479 TPM guideline added to section 2.6.2	Dec. 15, 2009
2.3B	476 Text adjustment toConfigAccess &ConfigRouting	Dec. 15, 2009
2.3B	460 Section 2.6 languageChange	Dec. 15, 2009
2.3B	454 Dynamic support of media detection - network stack	Dec. 15, 2009
2.3B	431 UEFI 2.3 Feb Draft: Section 30.4	Feb. 24, 2010
2.3B	301 Errata to the Authentication Protocol	Dec. 15, 2009
2.3B	215 previously added to Device Driver (wrong), nowBusDriver (correct)	Dec. 15, 2009
2.3A	522Bugs in EFI_CERT_BLOCK_RSA_2048_SHA256, ISCSI device path,CHAP device path	Sept 15, 2009
2.3A	518 typos	Sept 15, 2009
2.3A	517 IP stack related protocol update	Sept 15, 2009
2.3A	516 User Identity ProtocolBugs	Sept 15, 2009
2.3A	513 add support for gateways in ipv4 & ipv6 device path nodes	Sept 15, 2009
2.3A	506 TCP6/MTFTP6 StatusCode Definition	Sept 15, 2009
2.3A	505 TCP4/MTFTP4 statusCodes	Sept 15, 2009
2.3A	490Correction 28.2.5.6, Table 185. Information for Types of Stor- age	Sept 15, 2009
2.3A	478 Update to ALTCFG references	Sept 15, 2009
2.3A	477 Text adjustment toConfigAccess/ConfigRouting	Sept 15, 2009
2.3	463 Update EFI_IP6_PROTOCOL.Neighbors() API	May 7, 2009
2.3	462 ExitBootServices timers deactivation	May 7, 2009
2.3	461IP4 Mode Data definition update	May 7, 2009
2.3	460Chapter 2.6 language update	May 7, 2009
2.3	457Change KeyData.PackedValue to 0x40000200, page 63.	May 7, 2009
2.3	456 How to handle PXEBoot w/o NII Section 21.3	May 7, 2009
2.3	454 Dynamic support of media detection - network stack	May 7, 2009
2.3	453 Errata to support dynamic media detection - UNDI	May 7, 2009
2.3	452 Support to dynamically detect media errata - SNP	May 7, 2009
2.3	450 Missing opcode headers and formatting, section 28.3.8.3.x.	May 7, 2009
2.3	449 Add missing EFI_IFR_GET, EFI_IFR_SET and EFI_IFR_MAP to the syntax.Section 28.2.5.7.	May 7, 2009
2.3	448 Section 28.2.5.4 Questions, Syntax, Update question-option- tag; Add EFI_IFR_READ and EFI_IFR_WRITE in the question syntax.	May 7, 2009
2.3	447Section 28.2.5.11.2 Moving Forms, Update line that starts with EFI_IFR_FORM to: EFI_IFR_FORM or EFI_IFR_FORM_MAP (and all references in EFI_IFR_REF)	May 7, 2009
2.3	446 Section 28.2.5.2 Forms, Syntax,Change 3rd line to: form := EFI_IFR_FORM form-tag-list EFI_IFR_FORM_MAP form- tag-list	May 7, 2009
2.3	445 Table 194: EFI_IFR_FORM_MAP_OP, 2ndColumn shouldBe 0x5d (not 05xd)	May 7, 2009

continues on next page

Table 3 – continued from previous page

2.3	444 Form Set Syntax: Section 28.2.5.1.1, section shouldBe sub-heading, not heading level 5; Section 28.2.5.1, Syntax, line 3, text after := is not aligned with other text on line 2, 4	May 7, 2009
2.3	443 Section 28.3.8.3.38, EFI_IFR_MAP, Prototype, line 4, out-dent 2 spaces.	May 7, 2009
2.3	442 Section 28.3.8.3.64, EFI_IFR_SET, Prototype, lines 3-8, indentBy 2 spaces	May 7, 2009
2.3	440Change the defined type of EFI_STATUs from INTN to UINTN	May 7, 2009
2.3	439 Incorrect definitions of UEFI_CONFIG_LANG and UEFI_CONFIG_LANG_2 in UEFI 2.3 Feb18 draft	Feb 25, 2009
2.3	438 UEFI 2.3 Feb 13 Draft:Chapter 28 Formatting Issues	Feb 18, 2009
2.3	437 Errata to 2.3 draft material from UEFI Spec 2_3_Draft_Jan29	Feb 18, 2009
2.3	436 UEFI 2.3 split Figure 88 into 3 figures	Feb. 12, 2009
2.3	435 Partition SignatureClarification	Feb. 12, 2009
2.3	434 UEFI 2.3 Feb Draft: 28.3.8.3.58	Feb. 12, 2009
2.3	432 UEFI 2.3 Feb Draft: Appendix M.	Feb. 12, 2009
2.3	431 UEFI 2.3 Feb Draft: Section 30.4	Feb. 12, 2009
2.3	418Change Appendix O from “UEFI ACPI Table” to “UEFI ACPI Data	Feb 18, 2009
2.3	413Correct the definition of UEFI_CONFIG_LANG	Feb 18, 2009
2.3	410 UNDIBuffer usage	Feb 18, 2009
2.3	408 ARMBindingCorrections	Feb. 12, 2009
2.3	406 Missing EFI System Table Revision In UEFI 2.3 Draft	Feb. 12, 2009
2.3	395 New “Non-removable MediaBootBehavior” section	Feb. 12, 2009
2.3	394 Omission in EFI_USB2_HC_PROTOCOL	Feb. 12, 2009
2.3	388 Add HIIcallback types (FORM_OPEN, FORM_CLOSE) when a form is opened orClosed.	Feb. 12, 2009
2.3	376 Add ARM processorBinding to UEFI	Jan. 12, 2009
2.3	326 Add Firmware Management Protocol	Feb. 12, 2009
2.2A	429 EFI_HASH_SERVICE_BINDING_PROTOCOL GUID define misses _GUID	Feb. 12, 2009
2.2A	404 RemoveConstraint form EFI_TIME.YearComment	Feb. 12, 2009
2.2A	400 FreePool() description error	Feb. 12, 2009
2.2A	393 UEFI 2.1/2.2Boot ManagerBehaviorClarification	Feb. 12, 2009
2.2A	392 MBR errata in UEFI 2.2	Feb. 12, 2009
2.2A	391 Polarity of INCONSISTENT_IF and NO_SUBMIT_IF IFR opcodes wrong	Feb. 12, 2009
2.2A	390 UEFI 2.2 Miscellaneous HII-related errata	Feb. 12, 2009
2.2A	389 UEFI 2.2 HII-Related Formatting Issues	Feb. 12, 2009
2.2A	387 UEFI 2.1/UEFI 2.2A (ch. 12)	Feb. 12, 2009
2.2A	384 Fix HII package description omission.	Feb. 12, 2009
2.2A	379 UEFI 2.1/UEFI 2.2 HII-Related Errata	Feb. 12, 2009
2.2A	378 UEFI 2.1 & UEFI 2.2 HIIcallbackClarifications	Feb. 12, 2009
2.2A	377 MissingBLTBuffer figure.	Feb. 12, 2009
2.2A	375 Extra periods errata in UEFI 2.2	Feb. 12, 2009
2.2A	374 UEFI 2.1 & UEFI 2.2A (10.7-10.10)	Feb. 12, 2009
2.2A	373 UEFI 2.2,Chs. 9.5 & 9.6.2 & 9.6.3 (Device Path) Errata	Feb. 12, 2009
2.2A	372 UEFI 2.2 remove “Draft for Review”	Feb. 12, 2009
2.2A	371 UEFI 2.1 & UEFI 2.2 Typos (ch. 10)	Feb. 12, 2009
2.2A	370 EFI_SYSTEM_TABLE Errata (UEFI 2.1/UEFI 2.2)	Feb. 12, 2009
2.2A	368 EFI_FONT_DISPLAY_INFO.FontInfo description incorrect	Feb. 12, 2009

continues on next page

Table 3 – continued from previous page

2.2A	366	UEFI 2.x: Erroneous references to EFI_BOOT_SERVICES_TABLE, EFI_RUNTIME_SERVICES_TABLE	refer- Feb. 12, 2009
2.2A	364	UEFI 2.2 Typos & Formatting Issues (ch. 9)	Feb. 12, 2009
2.2A	362	UEFI 2.2 Typos (Next)	Feb. 12, 2009
2.2A	361	UEFI 2.2 Typos & Formatting Issues	Feb. 12, 2009
2.2A	359	TPL Table	Feb. 12, 2009
2.2A	358	Missing signature for UEFI 2.2.	Feb. 12, 2009
2.2	398	Update to M348 to fix small typo	Jan. 11, 2009
2.2	397	PCICopyMem() misspelling	Jan. 11, 2009
2.2	394	Omission in EFI_USB2_HC_PROTOCOL	Jan. 11, 2009
2.2	357	Clarify EFI_IFR_DISABLE_IFBehavior with regard to dynamic values	Jan. 11, 2009
2.2	351	Fix an unaligned field in a device path	Jan. 11, 2009
2.2	350	EFI_HII_STRING_PROTOCOL Typos	Jan. 11, 2009
2.2	348	EFI_IFR_RESET_BUTTON is incorrectly listed as a question	Jan. 11, 2009
2.2	347	Replace first paragraph of the “Description” section for the ExitBootServices()	Sept. 25, 2008
2.2	346	Nest, Sections 10.11 & 10.12 Under 10.10	Sept. 25, 2008
2.2	344	Correct missing statusCodes returned section for Form() in EFI_USER_CREDENTIAL_PROTOCOL.	Sept. 25, 2008
2.2	343	Correct missing parameter for User() function in EFI_USER_CREDENTIAL_PROTOCOL	Sept. 25, 2008
2.2	340	UEFI 2.2 Editorial / Formatting Issues	Sept. 25, 2008
2.2	339	Update missing TPL restrictions	Sept. 25, 2008
2.2	337	Replace the EFI_CRYPT_HANDLE reference (in the IPSec API) with a self-contained, independent definition.	Sept. 25, 2008
2.2	335	User Authentication errata	Sept. 25, 2008
2.2	334	Standardized “Unicode” References	Jan. 11, 2009
2.2	333	Correct the incorrect ‘;’ at the end of EFI_GUID #defines	Sept. 25, 2008
2.2	332	Correct SendForm description Type, PackageGuid and FormsetGuid parameters	Sept. 25, 2008
2.2	331	Definition for EFI_BROWSER_ACTION and the related #defines were not present—Insert.	Sept. 25, 2008
2.2	330	EFI_IFR_REF:ChangeCross reference to a question	Sept. 25, 2008
2.2	327	Clarify the support in DHCP4 protocol for “Inform” (DHCPINFORM) messages.	Sept. 25, 2008
2.2	325	MinorCorrection 28.3.8.3.20	July 25, 2008
2.2	324	ATA Pass-Thru ECR Update	July 25, 2008
2.2	323	VLAN modificationBecause of IPV6	July 25, 2008
2.2	322	Chapter 2 updates for IP6 net stack	July 25, 2008
2.2	321	Enable PCIe 2.0 andBeyond support in the UEFI error records	July 25, 2008
2.2	320	Clarification for WIN_CERTIFICATE types & relationship with signature database types	July 25, 2008
2.2	319	UEFI IPSec protocol	July 25, 2008
2.2	315	EFI TCP6 Protocol	July 25, 2008
2.2	314	EFI MTFTP6 Protocol	July 25, 2008
2.2	313	EFI IPv6Configuration Protocol	July 25, 2008
2.2	312	EFI IPv6 Protocol	July 25, 2008
2.2	311	EFI DHCPv6 Protocol	July 25, 2008
2.2	310	EFI UDPv6 Protocol	July 25, 2008

continues on next page

Table 3 – continued from previous page

2.2	309 IPv6 Address display formatClarification	July 25, 2008
2.2	306 Some errata to the animation support	July 25, 2008
2.2	304 Errata to UpdateCapsule()	July 25, 2008
2.2	303 Add ability to have aCapsule that initiates a reset & doesn't return to theCaller	July 25, 2008
2.2	301 Errata to the Authentication Protocol	July 25, 2008
2.2	300 MTFTP errata	July 25, 2008
2.2	299 PIWG Firmware File/Firmware Volume Typo Errata	July 25, 2008
2.2	294 LocateDevicePath with multi-instance device path	July 25, 2008
2.2	291 HII Errata / Update	July 25, 2008
2.2	288 Additional wording fixes for GPT Entry AttributeBit 1	July 25, 2008
2.2	282 Updated Requirements Section For ATA Pass Through (M242)	July 25, 2008
2.2	279 Firmware/OS Trusted Key Exchange and Image Validation	July 25, 2008
2.2	242 UEFI ATA Pass-Through Protocol	July 25, 2008
2.2	237 UEFI User Identification Proposal (from USST)	July 25, 2008
2.2	215 new Start() RemainingDevicePath Syntax	July 25, 2008
2.2	212 UEFI HII Standards Mapping	July 25, 2008
2.2	211UEFI Setup Question / Form Access Update	July 25, 2008
2.2	210 UEFI HII Animation addition	July 25, 2008
2.2	202 EAP Management	July 25, 2008
2.2	201EAP	July 25, 2008
2.2	200 VLAN	July 25, 2008
2.2	199 FTP API	July 25, 2008
2.2	198 GUID Partition Entry AttributesClarification and Definition	July 25, 2008
2.2	169 EFI Driver Health Protocol	July 25, 2008
2.2	157 Floating-Point ABIChanges For X86, X64 & Itanium	July 25, 2008
2.1C	Re-format Revision History fromBulleted lists to one row per Mantis ticket/ EngineeringChange Request	June 5, 2008
2.1C	60 iSCSI Device Path Update	June 5, 2008
2.1C	59 Add returnCode to Diagnostics Protocol	June 5, 2008
2.1C	58 Language update for EfiReservedMemory type usage	June 5, 2008
2.1C	57Clarify text for Extended SCSI Pass Thru Protocol.GetNextTargetLun()	June 5, 2008
2.1C	56Clarification on ResetSystem	June 5, 2008
2.1C	55Clarification on UpdateCapsule	June 5, 2008
2.1C	54 ACPI Table Protocol GUID Update	June 5, 2008
2.1C	52 New GUID for Driver Diagnostics and DriverConfiguration Protocols with new GUID	June 5, 2008
2.1C	283 Minor update toClarify a typedef/returnCode in HII	June 5, 2008
2.1C	281 Runtime memory allocation	June 5, 2008
2.1C	280 Some minor errata to keyboard related topics	June 5, 2008
2.1C	278Change references to EFI_SIMPLE_INPUT_PROTOCOL into EFI_SIMPLE_TEXT_INPUT_PROTOCOL	June 5, 2008
2.1C	266 PKCS11.5 structure does notCorrectly specify the portion of theCited RFC that pertains to theCertificate struct/algorithm	June 5, 2008
2.1C	249 Latest update to UCST Errata list	June 5, 2008
2.1C	248Correction to text inChapter 8.2 of UEFI 2.1B	June 5, 2008
2.1C	246 New returnCode	June 5, 2008
2.1C	245 Remove extraneous text inChapter 29	June 5, 2008
2.1C	244 Replace references to EFI_FIRMWARE_VOLUME_INFO_PPI with EFI_PEI_FIRMWARE_VOLUME_INFO_PPI	June 5, 2008

continues on next page

Table 3 – continued from previous page

2.1C	221ImageBlock Structure name typos in 27.3.7.2	June 5, 2008
2.1C	220 Replace references to RFC 3066 to RFC 4646	June 5, 2008
2.1C	219 IA-32 and x64 stack need toBe 16-byte aligned	June 5, 2008
2.1C	218 SATA update to section 9.3.5.6	June 5, 2008
2.1C	217 EFI_PLATFORM_TO_DRIVER_CONFIGURATION_PROTOCOL.Query() Update	June 5, 2008
2.1C	216 UEFI 2.1 textCorrections	June 5, 2008
2.1C	214 Device_IO + typos	June 5, 2008
2.1C	213 UEFI HII Errata	June 5, 2008
2.1C	209 ESP number/locationClarifications	June 5, 2008
2.1C	208 Driver Protocol Names and GUIDs	June 5, 2008
2.1C	207 Updated Wording for the File Path	June 5, 2008
2.1C	206Clarify return values for extended scsi passthru protocol	June 5, 2008
2.1C	203 Platform Error Record - x64 register state errata	June 5, 2008
2.1C	193 Loaded Image device paths for EFI Drivers loaded from PCI Option ROMs	June 5, 2008
2.1C	189 Graphics Output ProtocolClarification	June 5, 2008
2.1B	51 Long physicalBlocks updates	December 11, 2007
2.1B	205Change LoadImage() parameter name from FilePath to DevicePath; endsConfusion with EFI_LOADED_IMAGE_PROTOCOL	December 11, 2007
2.1B	197 EFI Loaded Image Device Path Protocol	December 11, 2007
2.1B	190 Extensive errata form UCST including OPCODEsChanges to resolveConflicts.	December 11, 2007
2.1B	187Clarify input protocols.	December 11, 2007
2.1B	186Change PCIR struct to match PCI FW Spec 3.0	December 11, 2007
2.1B	185Change EFI term to UEFI forConsistency	December 11, 2007
2.1B	184 SNIA/DDF Wording Update	December 11, 2007
2.1B	182Clarify EFI_MTFPT4_TOKEN	December 11, 2007
2.1B	181Correct MNP GUIDCollision	December 11, 2007
2.1B	177 remove ending paragraph (editing text) in section 9.6	December 11, 2007
2.1B	175 Update to SendForm API	December 11, 2007
2.1B	174 Error record addition for dma remapping units	December 11, 2007
2.1B	173 MinorChanges to the description of two of the fields in theCommon Platform Error Record, in Appendix N	December 11, 2007
2.1B	172 Typo for ResetSystem()	December 11, 2007
2.1B	170 (Addition of) Driver Family Override Protocol	December 11, 2007
2.1B	168 Remove LOAD_OPTION_GRAPHICS	December 11, 2007
2.1B	165 Fix EFI_GRAPHICS_OUTPUT_PIXEL	December 11, 2007
2.1B	164 Update to USB2_HC_PROTOCOL Table	December 11, 2007
2.1B	162 UEFI PIWG Device Path Errata	December 11, 2007
2.1B	160Clean up references to PCIR	December 11, 2007
2.1B	158 Errata to the UEFI 2.1Configuration sections	December 11, 2007
2.1B	156 SendForm API Errata	December 11, 2007
2.1B	159 Adjust some of the #define names in the Simple Text Input Ex protocol	December 11, 2007
2.1A	UEFI 2.1 incorporating Errata through 4-27-07	April 27, 2007
2.1	Second release	January 23, 2007
2.0	First release of specification.	January 31, 2006